IF3170 - Inteligensi Artifisial

Laporan Tugas Besar 1

Dosen: Dr. Nur Ulfa Maulidevi, S.T, M.Sc.

Disusun oleh:

Kelompok 8 – aiyaiyaimalittlebutterfly

Kevin Wirya Valerian	13524019
Emilio Justin	13524043
Farrell Limjaya	13524046

SEMESTER 1

TAHUN AJARAN 2026/2027

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA - KOMPUTASI

INSTITUT TEKNOLOGI BANDUNG

DAFTAR ISI

DAFTAR ISI 2
DAFTAR TABEL 5
DAFTAR GAMBAR 8
Deskripsi Persoalan 1
- Spesifikasi 2
  - Daftar Variabel Permasalahan 2
  - Ketentuan Representasi 2
  - Objective Function & Constraints 3

Pembahasan 4
- Spesifikasi Persoalan 4
- Hill Climbing(Steepest Ascent) 5
  - Deskripsi Fungsi/Kelas 5
  - Run 1 9
  - Run 2 11
  - Run 3 13
- Hill Climbing(Sideways Move) 15
  - Deskripsi Fungsi/Kelas 15
  - Run 1 18
  - Run 2 20
  - Run 3 22
- Hill Climbing(Random Restart) 24
  - Deskripsi Fungsi/Kelas 24
  - Run 1 28
  - Run 2 31
  - Run 3 34
- Hill Climbing(Stochastic) 37
  - Deskripsi Fungsi/Kelas 37
  - Run 1 40
  - Run 2 42
  - Run 3 44
- Simulated Annealing 46
  - Deskripsi Fungsi/Kelas 46
  - Run 1 51
  - Run 2 53
  - Run 3 55
- Genetic Algorithm 57
  - Deskripsi Fungsi/Kelas 57

Run 1 - Iterasi 50, Populasi 20 63
Run 2 - Iterasi 50, Populasi 20 65
Run 3 - Iterasi 50, Populasi 20 67
Run 1 - Iterasi 100, Populasi 20 69
Run 2 - Iterasi 100, Populasi 20 71
Run 3 - Iterasi 100, Populasi 20 73
Run 1 - Iterasi 200, Populasi 20 75
Run 2 - Iterasi 200, Populasi 20 77
Run 3 - Iterasi 200, Populasi 20 79
Run 1 - Populasi 10, Iterasi 100 81
Run 2 - Populasi 10, Iterasi 100 83
Run 3 - Populasi 10, Iterasi 100 85
Run 1 - Populasi 20, Iterasi 100 87
Run 2 - Populasi 20, Iterasi 100 89
Run 3 - Populasi 20, Iterasi 100 91
Run 1 - Populasi 40, Iterasi 100 93
Run 2 - Populasi 40, Iterasi 100 95
Run 3 - Populasi 40, Iterasi 100 97
Bonus - Fungsi Objektif 99
Deskripsi Fungsi/Kelas 99
Run 1 103
Run 2 105
Run 3 107
Analisis Perbandingan Fungsi Objektif Wajib dan Bonus 109
Bonus - Multitruk 110
Deskripsi Fungsi/Kelas 110
Run 1 – 2 Truk 120
Run 2 – 2 Truk 122
Run 3 – 2 Truk 124
Run 1 – 3 Truk 126
Run 2 – 3 Truk 128
Run 3 – 3 Truk 130
Analisis Perbandingan Hasil untuk Variasi Banyak Truk 132
Analisis dari Keseluruhan Hasil Eksperimen 133
1. Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian? 133
2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain? 134
3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya? 135

4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan? 136
5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm? 137
Kesimpulan dan Saran 139
Kesimpulan 139
Saran 139
Pembagian Tugas 140
Referensi 141

DAFTAR TABEL

Tabel 2.0.1 Rincian Detail Paket pada Spesifikasi Persoalan..... 4
Tabel 2.1.1 Hasil Eksperimen Hill Climbing(Steepest Ascent)..... 8
Tabel 2.1.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Steepest Ascent HC..... 9
Tabel 2.1.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Steepest Ascent HC..... 11
Tabel 2.1.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Steepest Ascent HC..... 13
Tabel 2.2.1 Hasil Eksperimen Hill Climbing(Sideways Move)..... 16
Tabel 2.2.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Sideways Move HC.....18
Tabel 2.2.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Sideways Move HC.....20
Tabel 2.2.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Sideways Move HC.....22
Tabel 2.3.1 Hasil Eksperimen Hill Climbing(Random Restart).....26
Tabel 2.3.2 Jumlah Iterasi per Percobaan pada Eksperimen Random Restart Hill-Climbing..... 26
Tabel 2.3.3 Rincian Percobaan Run 1 Random Restart Hill-Climbing..... 28
Tabel 2.3.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Random Restart HC..... 29
Tabel 2.3.5 Rincian Percobaan Run 2 Random Restart Hill-Climbing..... 31
Tabel 2.3.6 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Random Restart HC..... 32
Tabel 2.3.7 Rincian Percobaan Run 3 Random Restart Hill-Climbing..... 34
Tabel 2.3.8 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Random Restart HC..... 35
Tabel 2.4.1 Hasil Eksperimen Hill Climbing(Stochastic).....39
Tabel 2.4.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Stochastic HC..... 40
Tabel 2.4.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Stochastic HC..... 42
Tabel 2.4.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Stochastic HC..... 44
Tabel 2.5.1 Hasil Eksperimen Simulated Annealing..... 50
Tabel 2.5.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Simulated Annealing..... 51
Tabel 2.5.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Simulated Annealing..... 53
Tabel 2.5.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Simulated Annealing..... 55
Tabel 2.6.1.1 Hasil Eksperimen Genetic Algorithm Variasi Iterasi, Populasi 20.....61
Tabel 2.6.1.2 Hasil Eksperimen Genetic Algorithm Variasi Populasi, Iterasi 100..... 62
Tabel 2.6.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i50-p20 Genetic Algorithm..... 63
Tabel 2.6.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i50-p20 Genetic Algorithm..... 65
Tabel 2.6.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i50-p20 Genetic Algorithm..... 67
Tabel 2.6.5 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i100-p20 Genetic Algorithm..... 69
Tabel 2.6.6 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i100-p20 Genetic Algorithm..... 71
Tabel 2.6.7 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i100-p20 Genetic

Algorithm.....73
Tabel 2.6.8 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i200-p20 Genetic Algorithm.....75
Tabel 2.6.9 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i200-p20 Genetic Algorithm.....77
Tabel 2.6.10 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i200-p20 Genetic Algorithm.....79
Tabel 2.6.11 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i100-p10 Genetic Algorithm.....81
Tabel 2.6.12 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i100-p10 Genetic Algorithm.....83
Tabel 2.6.13 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i100-p10 Genetic Algorithm.....85
Tabel 2.6.14 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i100-p20 Genetic Algorithm.....87
Tabel 2.6.15 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i100-p20 Genetic Algorithm.....89
Tabel 2.6.16 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i100-p20 Genetic Algorithm.....91
Tabel 2.6.17 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i100-p40 Genetic Algorithm.....93
Tabel 2.6.18 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i100-p40 Genetic Algorithm.....95
Tabel 2.6.19 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i100-p40 Genetic Algorithm.....97
Tabel 2.7.1 Hasil Eksperimen Fungsi Objektif Bonus dengan Algoritma Simulated Annealing..... 102
Tabel 2.7.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Simulated Annealing dengan Fungsi Objektif Bonus.....103
Tabel 2.7.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Simulated Annealing dengan Fungsi Objektif Bonus.....105
Tabel 2.7.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Simulated Annealing dengan Fungsi Objektif Bonus.....107
Tabel 2.8.1 Hasil Eksperimen Bonus Multitruk dengan Algoritma Simulated Annealing untuk 2 Truk.....119
Tabel 2.8.2 Hasil Eksperimen Bonus Multitruk dengan Algoritma Simulated Annealing untuk 3 Truk.....119
Tabel 2.8.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Simulated Annealing dengan 2 Truk.....120
Tabel 2.8.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Simulated Annealing dengan 2 Truk.....122
Tabel 2.8.5 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Simulated Annealing dengan 2 Truk.....124
Tabel 2.8.6 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Simulated Annealing

dengan 3 Truk..... 126
Tabel 2.8.7 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Simulated Annealing dengan 3 Truk..... 128
Tabel 2.8.8 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Simulated Annealing dengan 3 Truk..... 130
Tabel 2.8.9 Perbandingan Hasil untuk Variasi Banyak Truk..... 132
Tabel 2.9.1 Kedekatan dan Pencapaian Algoritma terhadap Global Optima..... 133
Tabel 2.9.2 Perbandingan Durasi Proses Pencarian Setiap Algoritma..... 135
Tabel 2.9.3 Perbandingan Rentang dan Simpangan Baku Hasil Akhir Eksperimen..... 136
Tabel 2.9.4 Pengaruh banyak iterasi dengan populasi tetap 20..... 137
Tabel 2.9.5 Pengaruh jumlah populasi dengan iterasi tetap 100..... 138

DAFTAR GAMBAR

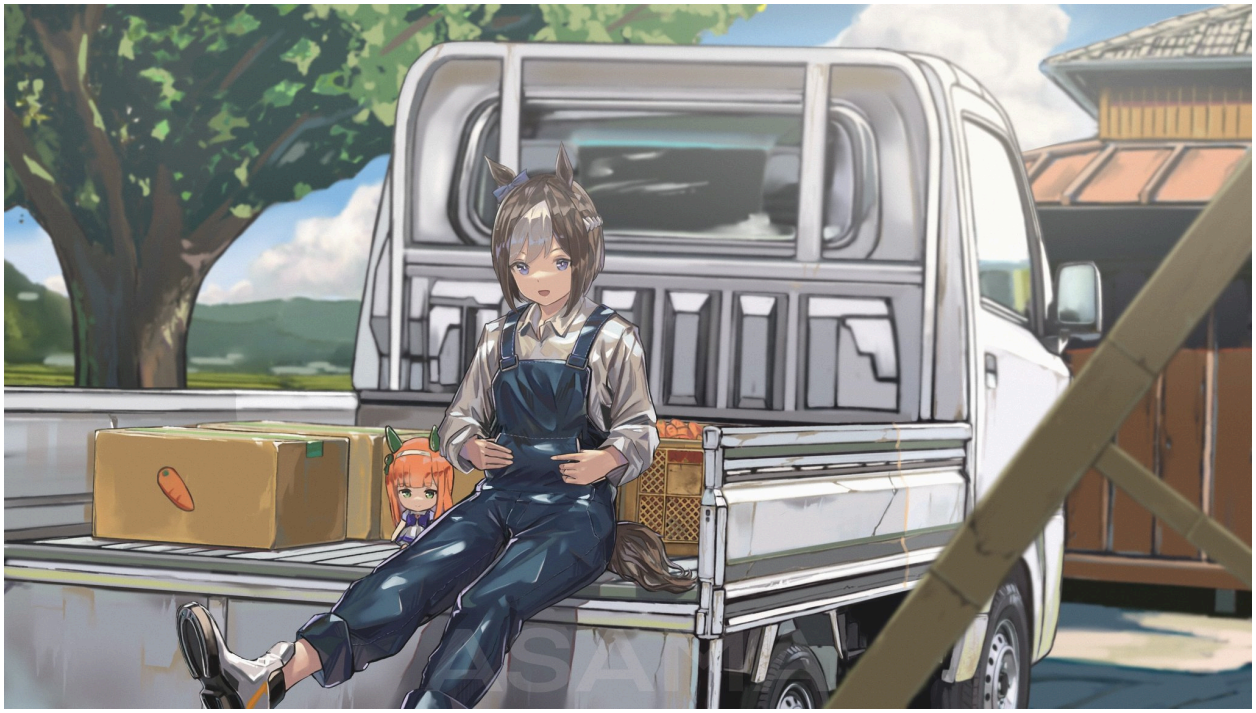
Gambar 2.1.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Steepest Ascent HC..... 10
Gambar 2.1.2 Plot Nilai Fungsi Objektif terhadap Iterasi Run 1 Steepest Ascent HC..... 10
Gambar 2.1.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Steepest Ascent HC..... 12
Gambar 2.1.4 Plot Nilai Fungsi Objektif terhadap Iterasi Run 2 Steepest Ascent HC..... 12
Gambar 2.1.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Steepest Ascent HC..... 14
Gambar 2.1.6 Plot Nilai Fungsi Objektif terhadap Iterasi Run 3 Steepest Ascent HC..... 14
Gambar 2.2.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Sideways Move HC.....19
Gambar 2.2.2 Plot Nilai Fungsi Objektif terhadap Iterasi Run 1 Sideways Move HC.....19
Gambar 2.2.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Sideways Move HC.....21
Gambar 2.2.4 Plot Nilai Fungsi Objektif terhadap Iterasi Run 2 Sideways Move HC.....21
Gambar 2.2.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Sideways Move HC.....23
Gambar 2.2.6 Plot Nilai Fungsi Objektif terhadap Iterasi Run 3 Sideways Move HC.....23
Gambar 2.3.1 Visualisasi Konfigurasi Awal dan Akhir Terbaik Run 1 Random Restart HC.....29
Gambar 2.3.2 Plot Nilai Fungsi Objektif dan Nilai Terbaik terhadap Iterasi Run 1 Random Restart Hill-Climbing.....30
Gambar 2.3.3 Visualisasi Konfigurasi Awal dan Akhir Terbaik Run 2 Random Restart HC.....32
Gambar 2.3.4 Plot Nilai Fungsi Objektif dan Nilai Terbaik terhadap Iterasi Run 2 Random Restart Hill-Climbing.....33
Gambar 2.3.5 Visualisasi Konfigurasi Awal dan Akhir Terbaik Run 3 Random Restart HC.....35
Gambar 2.3.6 Plot Nilai Fungsi Objektif dan Nilai Terbaik terhadap Iterasi Run 3 Random Restart Hill-Climbing.....36
Gambar 2.4.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Stochastic HC..... 41
Gambar 2.4.2 Plot Nilai Fungsi Objektif terhadap Iterasi Run 1 Stochastic HC..... 41
Gambar 2.4.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Stochastic HC..... 43
Gambar 2.4.4 Plot Nilai Fungsi Objektif terhadap Iterasi Run 2 Stochastic HC..... 43
Gambar 2.4.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Stochastic HC..... 45
Gambar 2.4.6 Plot Nilai Fungsi Objektif terhadap Iterasi Run 3 Stochastic HC..... 45
Gambar 2.5.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Simulated Annealing.....52
Gambar 2.5.2 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1 Simulated Annealing..... 52
Gambar 2.5.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Simulated Annealing.....54
Gambar 2.5.4 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2 Simulated Annealing..... 54
Gambar 2.5.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Simulated Annealing.....56
Gambar 2.5.6 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3 Simulated Annealing..... 56
Gambar 2.6.1 Visualisasi Konfigurasi Awal dan Akhir Run 1-i50-p20 Genetic Algorithm..... 64
Gambar 2.6.2 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i50-p20

Genetic Algorithm..... 64
Gambar 2.6.3 Visualisasi Konfigurasi Awal dan Akhir Run 2-i50-p20 Genetic Algorithm..... 66
Gambar 2.6.4 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i50-p20 Genetic Algorithm..... 66
Gambar 2.6.5 Visualisasi Konfigurasi Awal dan Akhir Run 3-i50-p20 Genetic Algorithm..... 68
Gambar 2.6.6 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i50-p20 Genetic Algorithm..... 68
Gambar 2.6.7 Visualisasi Konfigurasi Awal dan Akhir Run 1-i100-p20 Genetic Algorithm..... 70
Gambar 2.6.8 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i100-p20 Genetic Algorithm..... 70
Gambar 2.6.9 Visualisasi Konfigurasi Awal dan Akhir Run 2-i100-p20 Genetic Algorithm..... 72
Gambar 2.6.10 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i100-p20 Genetic Algorithm..... 72
Gambar 2.6.11 Visualisasi Konfigurasi Awal dan Akhir Run 3-i100-p20 Genetic Algorithm..... 74
Gambar 2.6.12 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i100-p20 Genetic Algorithm..... 74
Gambar 2.6.13 Visualisasi Konfigurasi Awal dan Akhir Run 1-i200-p20 Genetic Algorithm..... 76
Gambar 2.6.14 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i200-p20 Genetic Algorithm..... 76
Gambar 2.6.15 Visualisasi Konfigurasi Awal dan Akhir Run 2-i200-p20 Genetic Algorithm..... 78
Gambar 2.6.16 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i200-p20 Genetic Algorithm..... 78
Gambar 2.6.17 Visualisasi Konfigurasi Awal dan Akhir Run 3-i200-p20 Genetic Algorithm..... 80
Gambar 2.6.18 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i200-p20 Genetic Algorithm..... 80
Gambar 2.6.19 Visualisasi Konfigurasi Awal dan Akhir Run 1-i100-p10 Genetic Algorithm..... 82
Gambar 2.6.20 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i100-p10 Genetic Algorithm..... 82
Gambar 2.6.21 Visualisasi Konfigurasi Awal dan Akhir Run 2-i100-p10 Genetic Algorithm..... 84
Gambar 2.6.22 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i100-p10 Genetic Algorithm..... 84
Gambar 2.6.23 Visualisasi Konfigurasi Awal dan Akhir Run 3-i100-p10 Genetic Algorithm..... 86
Gambar 2.6.24 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i100-p10 Genetic Algorithm..... 86
Gambar 2.6.25 Visualisasi Konfigurasi Awal dan Akhir Run 1-i100-p20 Genetic Algorithm..... 88
Gambar 2.6.26 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i100-p20 Genetic Algorithm..... 88
Gambar 2.6.27 Visualisasi Konfigurasi Awal dan Akhir Run 2-i100-p20 Genetic Algorithm..... 90
Gambar 2.6.28 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i100-p20 Genetic Algorithm..... 90
Gambar 2.6.29 Visualisasi Konfigurasi Awal dan Akhir Run 3-i100-p20 Genetic Algorithm..... 92
Gambar 2.6.30 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i100-p20

Genetic Algorithm..... 92
Gambar 2.6.31 Visualisasi Konfigurasi Awal dan Akhir Run 1-i100-p40 Genetic Algorithm..... 94
Gambar 2.6.32 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i100-p40 Genetic Algorithm..... 94
Gambar 2.6.33 Visualisasi Konfigurasi Awal dan Akhir Run 2-i100-p40 Genetic Algorithm..... 96
Gambar 2.6.34 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i100-p40 Genetic Algorithm..... 96
Gambar 2.6.35 Visualisasi Konfigurasi Awal dan Akhir Run 3-i100-p40 Genetic Algorithm..... 98
Gambar 2.6.36 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i100-p40 Genetic Algorithm..... 98
Gambar 2.7.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Simulated Annealing dengan Fungsi Objektif Bonus..... 104
Gambar 2.7.2 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1 Simulated Annealing dengan Fungsi Objektif Bonus..... 104
Gambar 2.7.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Simulated Annealing dengan Fungsi Objektif Bonus..... 106
Gambar 2.7.4 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2 Simulated Annealing dengan Fungsi Objektif Bonus..... 106
Gambar 2.7.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Simulated Annealing dengan Fungsi Objektif Bonus..... 108
Gambar 2.7.6 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3 Simulated Annealing dengan Fungsi Objektif Bonus..... 108
Gambar 2.8.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Simulated Annealing dengan 2 Truk..... 120
Gambar 2.8.2 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1 Simulated Annealing dengan 2 Truk..... 121
Gambar 2.8.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Simulated Annealing dengan 2 Truk..... 122
Gambar 2.8.4 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2 Simulated Annealing dengan 2 Truk..... 123
Gambar 2.8.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Simulated Annealing dengan 2 Truk..... 124
Gambar 2.8.6 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3 Simulated Annealing dengan 2 Truk..... 125
Gambar 2.8.7 Visualisasi Konfigurasi Awal dan Akhir Run 1 Simulated Annealing dengan 3 Truk..... 126
Gambar 2.8.8 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1 Simulated Annealing dengan 2 Truk..... 127
Gambar 2.8.9 Visualisasi Konfigurasi Awal dan Akhir Run 2 Simulated Annealing dengan 3 Truk..... 128
Gambar 2.8.10 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2 Simulated Annealing dengan 3 Truk..... 129
Gambar 2.8.11 Visualisasi Konfigurasi Awal dan Akhir Run 3 Simulated Annealing dengan 3 Truk....

130
Gambar 2.8.12 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3 Simulated Annealing dengan 3 Truk..... 131

Deskripsi Persoalan



Gambar 1. Special Week, driver truk handal (Sumber: 真夏の银堡垒🐴C108土曜日 東地区 ホ-10a on X)

Keluarga **Mejiro** sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, **Mejiro McQueen** menawarkan posisi driver truk uji coba kepada Special Week, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama **Mejiro Express** dengan slogan "Lebih kilat dari Umamusume".

Karena **Special Week** sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.

Spesifikasi

Tugas ini bertujuan untuk membuat konfigurasi pengisian paket paling optimal untuk sebuah truk peti kemas. Optimal di sini berarti **memaksimalkan** fungsi objektif.

Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Package	id	"RusdiPackage"	Identifier
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu width, length, dan height
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu z.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu width, length, dan height
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar (seluruh bagian paket di luar).
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan key-value seperti JSON.
- Inisialisasi (*initial state*) random.
- Langkah neighbor (*move*) yang diperbolehkan tiap iterasi:
    1. Menukar dua paket, baik di dalam maupun di luar truk
    2. Memindahkan sebuah paket ke suatu koordinat yang berbeda

3. Merotasi paket 90° terhadap sumbu x, y, atau z

- Representasi solusi (*final state*) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:
  1. Bagaimana keadaan semua paket yang di dalam maupun yang di luar
  2. Bagaimana keadaan peti kemas pada truk
  3. Value pada state tersebut

Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- **Objective function**

  Permasalahan ini bertujuan untuk memaksimalkan value peti kemas pada truk.

- **Kardus tidak bisa melayang dan menembus dinding peti kemas**

  Melayang di sini didefinisikan dengan misalkan paket i setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(x_i, y_i, z_i)$$

  dan dimensi

  $$(w_i, d_i, h_i)$$

  sehingga alas paket i adalah

  $$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i, y_i \leq y < y_i + d_i\}$$

  Sebuah titik di alas memiliki support apabila:

  1. $Z_{i=0}$, artinya di alas truk, atau
  2. Terdapat paket lain j yang mempunyai bagian atas tepat di bawah titik tersebut

  Maka melayang adalah kondisi tidak ada satupun titik alas yang memiliki support.

- **Barang pecah-belah tidak boleh di bawah paket lain**

  Barang pecah-belah tidak boleh menjadi support

- **Truk tidak boleh mengangkut melebihi kapasitasnya**

Pembahasan

Spesifikasi Persoalan

- Dimensi Truk : w = 6, l = 4, h = 4
- Kapasitas Beban Maksimum : 40 unit berat
- Daftar 12 Paket

Tabel 2.0.1 Rincian Detail Paket pada Spesifikasi Persoalan

ID Paket	Dimensi Asli (w,l,h)	Volume	Value	Weight	Rasio Val/Wt	Fragile	ETA
P01	(4, 3, 2)	24	62	12	5.17	Tidak	6
P02	(3, 3, 2)	18	55	10	5.50	Tidak	3
P03	(4, 2, 2)	16	48	9	5.33	Tidak	1
P04	(3, 2, 2)	12	44	7	6.29	Ya	2
P05	(2, 3, 2)	12	40	6	6.67	Tidak	5
P06	(2, 2, 2)	8	32	5	6.40	Tidak	4
P07	(3, 2, 1)	6	29	4	7.25	Ya	1
P08	(1, 2, 3)	6	26	3	8.67	Tidak	6
P09	(2, 2, 1)	4	25	3	8.33	Tidak	2
P10	(3, 1, 1)	3	18	2	9.00	Ya	3
P11	(1, 2, 1)	2	12	1	12.00	Tidak	5
P12	(2, 5, 1)	10	46	6	7.67	Tidak	4
Total		121	437	68		3 Fragile	

Hill Climbing(Steepest Ascent)

Deskripsi Fungsi/Kelas

1. Fungsi _validate_parameters

```
def _validate_parameters(seed: int, max_iterations: int | None) -> None:
    if type(seed) is not int:
        raise ValueError("Seed harus berupa integer.")
    if max_iterations is not None and (
        type(max_iterations) is not int or max_iterations <= 0
    ):
        raise ValueError("max_iterations harus berupa integer positif atau None.")
```

Fungsi _validate_parameters bertugas memvalidasi parameter sebelum pencarian dimulai. Parameter seed harus berupa bilangan bulat, sedangkan max_iterations harus berupa bilangan bulat positif atau None. Nilai None menunjukkan bahwa pencarian tidak memiliki batas jumlah iterasi dan berjalan sampai tidak ditemukan neighbor yang meningkatkan nilai objective. Jika parameter tidak memenuhi ketentuan, fungsi menghasilkan ValueError.

2. Fungsi _best_neighbor

```
def _best_neighbor(
    problem: Problem, current: State, current_score: int | float, rng: Random,
    *, allow_sideways: bool = False,
) -> tuple[State | None, int | float]:
    """Reservoir sampling atas kandidat terbaik; tidak menyimpan semua neighbor."""
    best_neighbor = None
    best_score = current_score
    tied_best = 0
    for neighbor in iter_neighbors(problem, current):
        if neighbor == current:
            continue
        neighbor_score = score(problem, neighbor)
```

```
    if neighbor_score > best_score:
      best_neighbor = neighbor
      best_score = neighbor_score
      tied_best = 1
    elif neighbor_score == best_score and (best_neighbor is not None or allow_sideways):
      tied_best += 1
      if rng.randrange(tied_best) == 0:
        best_neighbor = neighbor
  return best_neighbor, best_score
```

Fungsi _best_neighbor memeriksa seluruh neighbor valid melalui iter_neighbors, menghitung nilainya menggunakan score, dan memilih kandidat terbaik yang memenuhi aturan penerimaan. Jika beberapa kandidat terbaik memiliki score sama, pemilihan dilakukan secara acak menggunakan *reservoir sampling* tanpa menyimpan seluruh neighbor sekaligus. Pada Steepest Ascent, allow_sideways=False sehingga hanya peningkatan score yang diterima; jika tidak ditemukan perbaikan, fungsi mengembalikan None beserta score saat ini.

3. Fungsi solve

```
def solve(
  problem: Problem, *, seed: int = 11, max_iterations: int | None = None,
  objective: str | None = None,
) -> SearchResult:
  _validate_parameters(seed, max_iterations)
  active_objective = set_objective(objective) if objective is not None else get_objective()

  initial_state = generate_initial_state(problem, Random(seed))
  rng = Random(seed + 1)

  started = perf_counter()
  current = initial_state
  current_score = score(problem, current)
  iterations = 0
  history = [{"iteration": 0, "score": current_score}]
  stop_reason = "max_iterations"

  while max_iterations is None or iterations < max_iterations:
    best_neighbor, best_score = _best_neighbor(problem, current, current_score, rng)
```

```
    iterations += 1
    if best_neighbor is not None:
      current = best_neighbor
      current_score = best_score
    history.append({"iteration": iterations, "score": current_score})

    if best_neighbor is None:
      stop_reason = "local_optimum"
      break

  duration_seconds = perf_counter() - started
  return SearchResult(
    algorithm="hc",
    initial_state=initial_state,
    final_state=current,
    final_score=current_score,
    iterations=iterations,
    duration_seconds=duration_seconds,
    seed=seed,
    stop_reason=stop_reason,
    parameters={"max_iterations": max_iterations, "objective": active_objective},
    history=history,
  )
```

Fungsi solve menjalankan Steepest Ascent mulai dari validasi parameter, pemilihan objective, dan pembangkitan initial state menggunakan Random(seed), dengan RNG pencarian terpisah berupa Random(seed + 1). Setiap iterasi memilih neighbor terbaik melalui _best_neighbor dan memperbarui state hanya jika score meningkat. Pencarian berhenti ketika tidak ditemukan perbaikan atau batas iterasi tercapai. Fungsi mencatat history sejak iterasi 0, menghitung pemeriksaan terakhir tanpa perpindahan sebagai satu iterasi, serta mengukur durasi pencarian menggunakan perf_counter() tanpa memasukkan inisialisasi, penyimpanan, dan visualisasi. Hasil dikembalikan sebagai SearchResult yang memuat state awal/akhir, score akhir, iterasi, durasi, seed, parameter, history, dan alasan berhenti.

4. Fungsi _nonnegative_limit

```
def _nonnegative_limit(value: int, name: str) -> None:
  if type(value) is not int or value < 0:
```

```
    raise ValueError(f"{name} harus berupa integer nonnegatif.")
```

Fungsi _nonnegative_limit memastikan suatu parameter batas berupa bilangan bulat nonnegatif dan menghasilkan ValueError jika tidak sesuai. Parameter name digunakan untuk menampilkan nama parameter pada pesan kesalahan. Fungsi ini berada pada modul Steepest Ascent sebagai pendukung bersama untuk memvalidasi batas sideways dan restart pada varian lainnya.

Tabel 2.1.1 Hasil Eksperimen Hill Climbing(Steepest Ascent)

Run (Seed)	Score Awal	Score Akhir	Nilai Objective Akhir	Berat Akhir	Iterasi	Alasan Berhenti	Durasi Pencarian (s)
1 (11)	226	283	283	40/40	4	local_optimum	0,320
2 (22)	243	243	243	34/40	1	local_optimum	0,081
3 (33)	246	246	246	39/40	1	local_optimum	0,078
Rata-rata	**238,33**	**257,33**	**257,33**	**37,67/40**	**2**	—	**0,160**

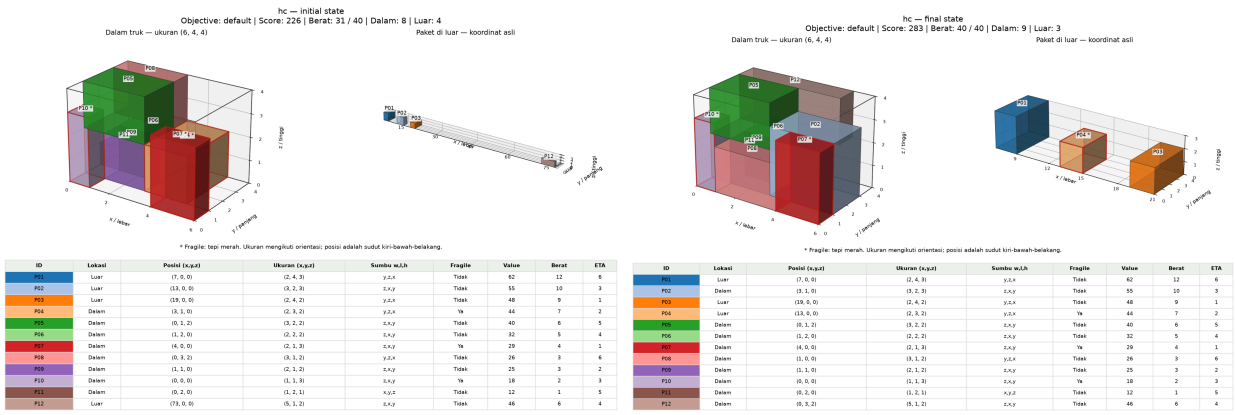
Run 1

Untuk run 1 dengan seed 11, berikut adalah hasil akhir yang didapatkan.

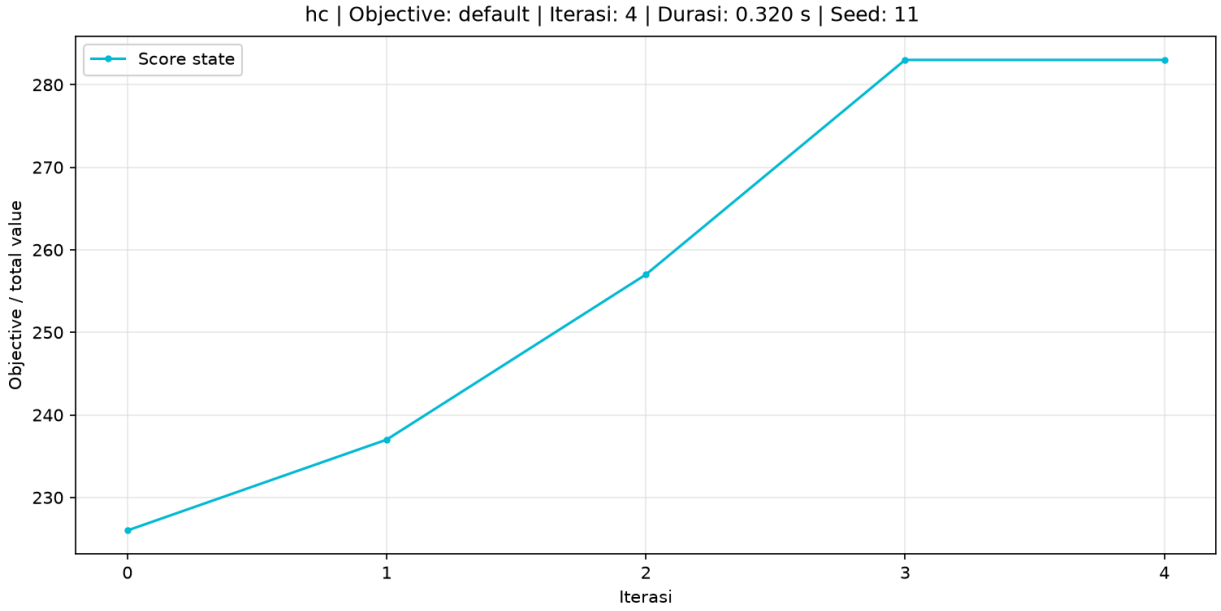
- Score Awal: 226 (berat awal: 31/40)
- Score Akhir: 283 (berat akhir: 40/40)
- Nilai Objective Function Akhir: 283
- Durasi Pencarian: 0,320 detik
- Banyak Iterasi: 4
- Alasan Berhenti: Tidak terdapat neighbor dengan score lebih tinggi (*local optimum*)

Tabel 2.1.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Steepest Ascent HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(2,4,3)	Luar	(7,0,0)	(2,4,3)
P02	Luar	(13,0,0)	(3,2,3)	Dalam	(3,1,0)	(3,2,3)
P03	Luar	(19,0,0)	(2,4,2)	Luar	(19,0,0)	(2,4,2)
P04	Dalam	(3,1,0)	(2,3,2)	Luar	(13,0,0)	(2,3,2)
P05	Dalam	(0,1,2)	(3,2,2)	Dalam	(0,1,2)	(3,2,2)
P06	Dalam	(1,2,0)	(2,2,2)	Dalam	(1,2,0)	(2,2,2)
P07	Dalam	(4,0,0)	(2,1,3)	Dalam	(4,0,0)	(2,1,3)
P08	Dalam	(0,3,2)	(3,1,2)	Dalam	(1,0,0)	(3,1,2)
P09	Dalam	(1,1,0)	(2,1,2)	Dalam	(1,1,0)	(2,1,2)
P10	Dalam	(0,0,0)	(1,1,3)	Dalam	(0,0,0)	(1,1,3)
P11	Dalam	(0,2,0)	(1,2,1)	Dalam	(0,2,0)	(1,2,1)
P12	Luar	(73,0,0)	(5,1,2)	Dalam	(0,3,2)	(5,1,2)



Gambar 2.1.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Steepest Ascent HC



Gambar 2.1.2 Plot Nilai Fungsi Objektif terhadap Iterasi Run 1 Steepest Ascent HC

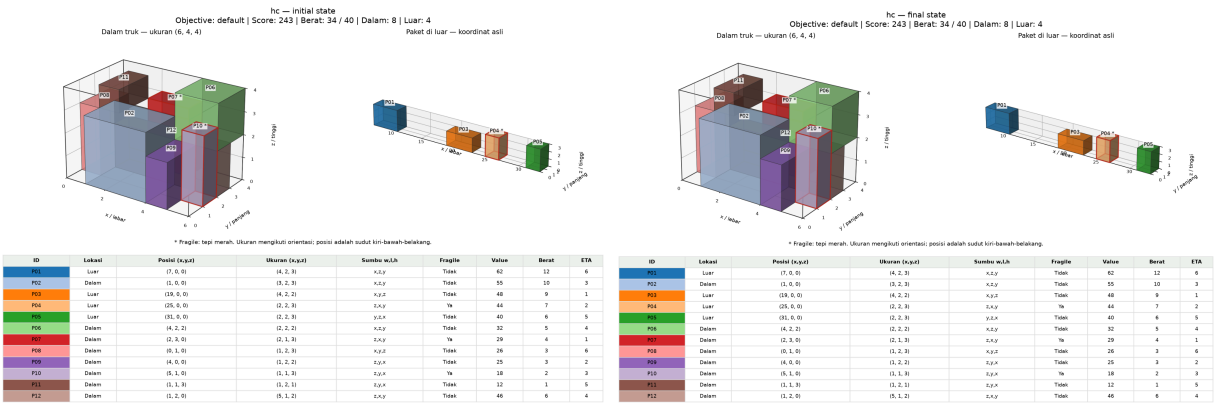
Run 2

Untuk run 2 dengan seed 22, berikut adalah hasil akhir yang didapatkan.

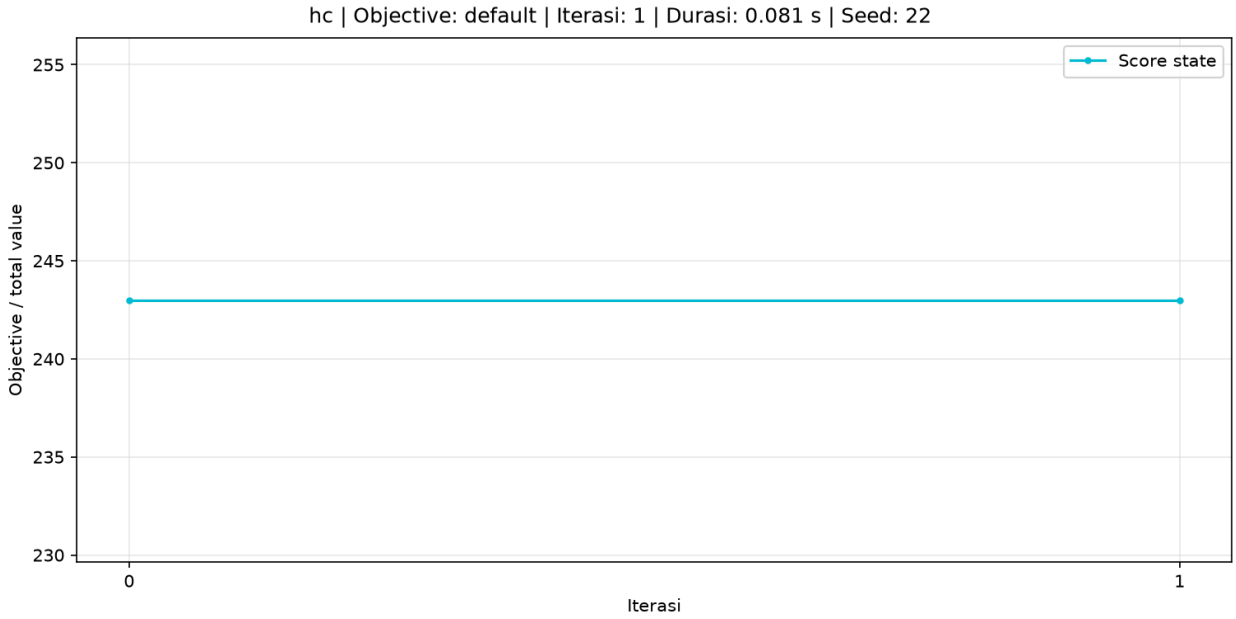
- Score Awal: 243 (berat awal: 34/40)
- Score Akhir: 243 (berat akhir: 34/40)
- Nilai Objective Function Akhir: 243
- Durasi Pencarian: 0,081 detik
- Banyak Iterasi hingga Berhenti: 1
- Alasan Berhenti: Tidak terdapat neighbor dengan score lebih tinggi (*local optimum*)

Tabel 2.1.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Steepest Ascent HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(4,2,3)	Luar	(7,0,0)	(4,2,3)
P02	Dalam	(1,0,0)	(3,2,3)	Dalam	(1,0,0)	(3,2,3)
P03	Luar	(19,0,0)	(4,2,2)	Luar	(19,0,0)	(4,2,2)
P04	Luar	(25,0,0)	(2,2,3)	Luar	(25,0,0)	(2,2,3)
P05	Luar	(31,0,0)	(2,2,3)	Luar	(31,0,0)	(2,2,3)
P06	Dalam	(4,2,2)	(2,2,2)	Dalam	(4,2,2)	(2,2,2)
P07	Dalam	(2,3,0)	(2,1,3)	Dalam	(2,3,0)	(2,1,3)
P08	Dalam	(0,1,0)	(1,2,3)	Dalam	(0,1,0)	(1,2,3)
P09	Dalam	(4,0,0)	(1,2,2)	Dalam	(4,0,0)	(1,2,2)
P10	Dalam	(5,1,0)	(1,1,3)	Dalam	(5,1,0)	(1,1,3)
P11	Dalam	(1,1,3)	(1,2,1)	Dalam	(1,1,3)	(1,2,1)
P12	Dalam	(1,2,0)	(5,1,2)	Dalam	(1,2,0)	(5,1,2)



Gambar 2.1.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Steepest Ascent HC



Gambar 2.1.4 Plot Nilai Fungsi Objektif terhadap Iterasi Run 2 Steepest Ascent HC

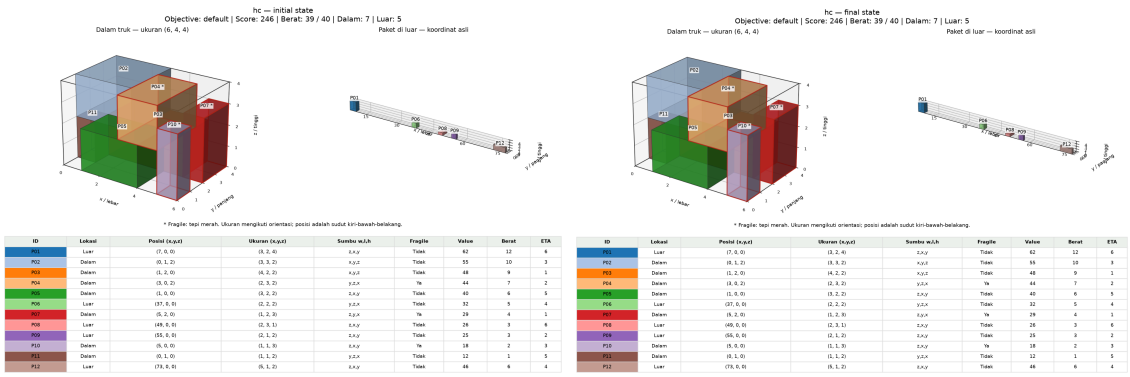
Run 3

Untuk run 3 dengan seed 33, berikut adalah hasil akhir yang didapatkan.

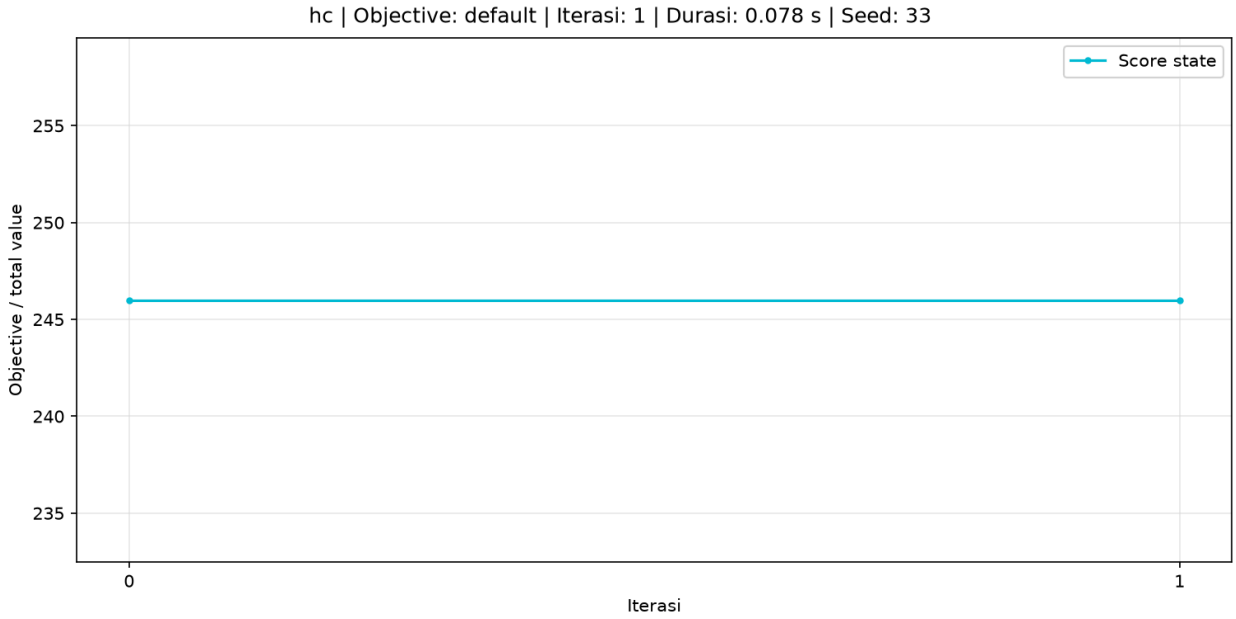
- Score Awal: 246 (berat awal: 39/40)
- Score Akhir: 246 (berat akhir: 39/40)
- Nilai Objective Function Akhir: 246
- Durasi Pencarian: 0,078 detik
- Banyak Iterasi hingga Berhenti: 1
- Alasan Berhenti: Tidak terdapat neighbor dengan score lebih tinggi (*local optimum*)

Tabel 2.1.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Steepest Ascent HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(3,2,4)	Luar	(7,0,0)	(3,2,4)
P02	Dalam	(0,1,2)	(3,3,2)	Dalam	(0,1,2)	(3,3,2)
P03	Dalam	(1,2,0)	(4,2,2)	Dalam	(1,2,0)	(4,2,2)
P04	Dalam	(3,0,2)	(2,3,2)	Dalam	(3,0,2)	(2,3,2)
P05	Dalam	(1,0,0)	(3,2,2)	Dalam	(1,0,0)	(3,2,2)
P06	Luar	(37,0,0)	(2,2,2)	Luar	(37,0,0)	(2,2,2)
P07	Dalam	(5,2,0)	(1,2,3)	Dalam	(5,2,0)	(1,2,3)
P08	Luar	(49,0,0)	(2,3,1)	Luar	(49,0,0)	(2,3,1)
P09	Luar	(55,0,0)	(2,1,2)	Luar	(55,0,0)	(2,1,2)
P10	Dalam	(5,0,0)	(1,1,3)	Dalam	(5,0,0)	(1,1,3)
P11	Dalam	(0,1,0)	(1,1,2)	Dalam	(0,1,0)	(1,1,2)
P12	Luar	(73,0,0)	(5,1,2)	Luar	(73,0,0)	(5,1,2)



Gambar 2.1.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Steepest Ascent HC



Gambar 2.1.6 Plot Nilai Fungsi Objektif terhadap Iterasi Run 3 Steepest Ascent HC

Hill Climbing(Sideways Move)

Deskripsi Fungsi/Kelas

1. Fungsi solve

```
def solve(
    problem: Problem, *, seed: int = 11, max_iterations: int | None = None,
    max_sideways_moves: int = 100, objective: str | None = None,
) -> SearchResult:
    _validate_parameters(seed, max_iterations)
    _nonnegative_limit(max_sideways_moves, "max_sideways_moves")
    active_objective = set_objective(objective) if objective is not None else get_objective()
    initial_state = generate_initial_state(problem, Random(seed))
    rng = Random(seed + 1)

    started = perf_counter()
    current = initial_state
    current_score = score(problem, current)
    iterations = sideways_moves = consecutive_sideways = 0
    history = [{"iteration": 0, "score": current_score, "consecutive_sideways": 0}]
    stop_reason = "max_iterations"

    while max_iterations is None or iterations < max_iterations:
        neighbor, neighbor_score = _best_neighbor(
            problem, current, current_score, rng,
            allow_sideways=max_sideways_moves > 0,
        )
        iterations += 1
        if neighbor is not None:
            if neighbor_score == current_score:
                consecutive_sideways += 1
                sideways_moves += 1
            else:
                consecutive_sideways = 0
            current, current_score = neighbor, neighbor_score
        history.append({
            "iteration": iterations, "score": current_score,
            "consecutive_sideways": consecutive_sideways,
        })
        if neighbor is None:
            stop_reason = "local_optimum"
            break
        if max_sideways_moves > 0 and consecutive_sideways >= max_sideways_moves:
            stop_reason = "max_sideways_moves"
            break

    duration_seconds = perf_counter() - started
```

```
  return SearchResult(
    algorithm="hc_sideways", initial_state=initial_state, final_state=current,
    final_score=current_score, iterations=iterations,
    duration_seconds=duration_seconds, seed=seed, stop_reason=stop_reason,
    parameters={
      "max_iterations": max_iterations,
      "max_sideways_moves": max_sideways_moves,
      "objective": active_objective,
    },
    history=history,
    metrics={"sideways_moves": sideways_moves, "consecutive_sideways":
consecutive_sideways},
  )
```

Fungsi solve menjalankan Hill-Climbing with Sideways Move dengan memilih neighbor terbaik yang memiliki score lebih tinggi atau sama dengan state saat ini. Proses diawali dengan validasi parameter, pemilihan objective, dan pembangkitan initial state menggunakan Random(seed), sedangkan pencarian menggunakan Random(seed + 1). Perpindahan dengan score sama menambah counter sideways_moves dan consecutive_sideways; ketika score meningkat, counter berturut-turut direset menjadi nol. Pencarian berhenti ketika batas sideways berturut-turut tercapai, tidak ditemukan neighbor yang dapat diterima, atau batas iterasi tercapai. Nilai max_sideways_moves=0 menonaktifkan perpindahan sideways. Fungsi mencatat history sejak iterasi 0 dan mengukur durasi menggunakan perf_counter() setelah inisialisasi selesai, kemudian mengembalikan SearchResult beserta metrik sideways. Pada eksperimen digunakan maksimum 100 langkah sideways berturut-turut tanpa batas iterasi tambahan, sehingga total sideways dapat melebihi 100 apabila terdapat peningkatan score yang mereset counter.

Fungsi _validate_parameters, _nonnegative_limit, dan _best_neighbor diimpor dari modul Steepest Ascent sehingga untuk deskripsi dan penjelasannya sama dengan yang ada pada Hill Climbing(Steepest Ascent).

Tabel 2.2.1 Hasil Eksperimen Hill Climbing(Sideways Move)

Run (Seed)	Score Awal	Score Akhir	Nilai Objective Akhir	Berat Akhir	Iterasi	Maks. Sideways Berturut-turut	Total Sideways	Alasan Berhenti	Durasi Pencarian (s)
1 (11)	226	283	283	40/40	103	100	100	max_sideways_moves	37,651

2 (22)	243	266	266	40/40	103	100	102	max_sideways_moves	8,283
3 (33)	246	252	252	39/40	157	100	156	max_sideways_moves	12,781
Rata-rata	**238,33**	**267,00**	**267,00**	**39,67/40**	**121**	**100**	**119,33**	—	**19,571**

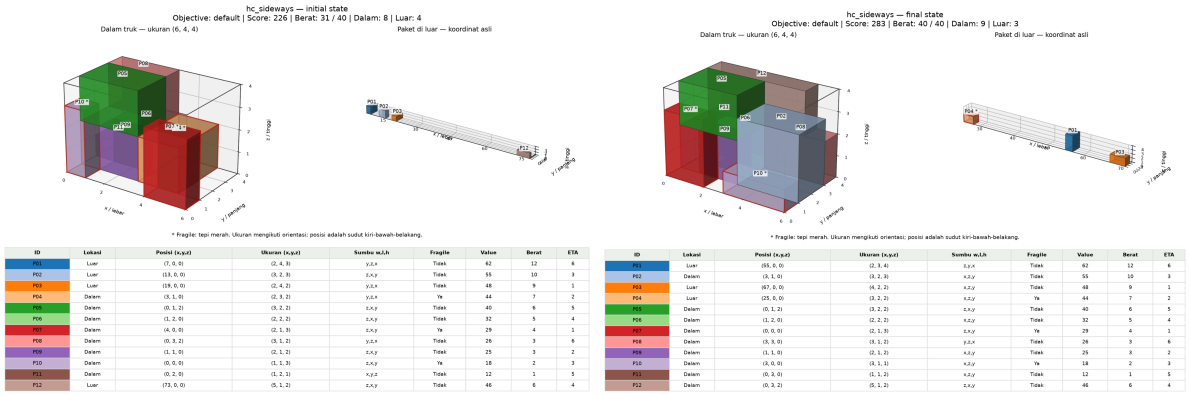
Run 1

Untuk run 1 dengan seed 11, berikut adalah hasil akhir yang didapatkan.

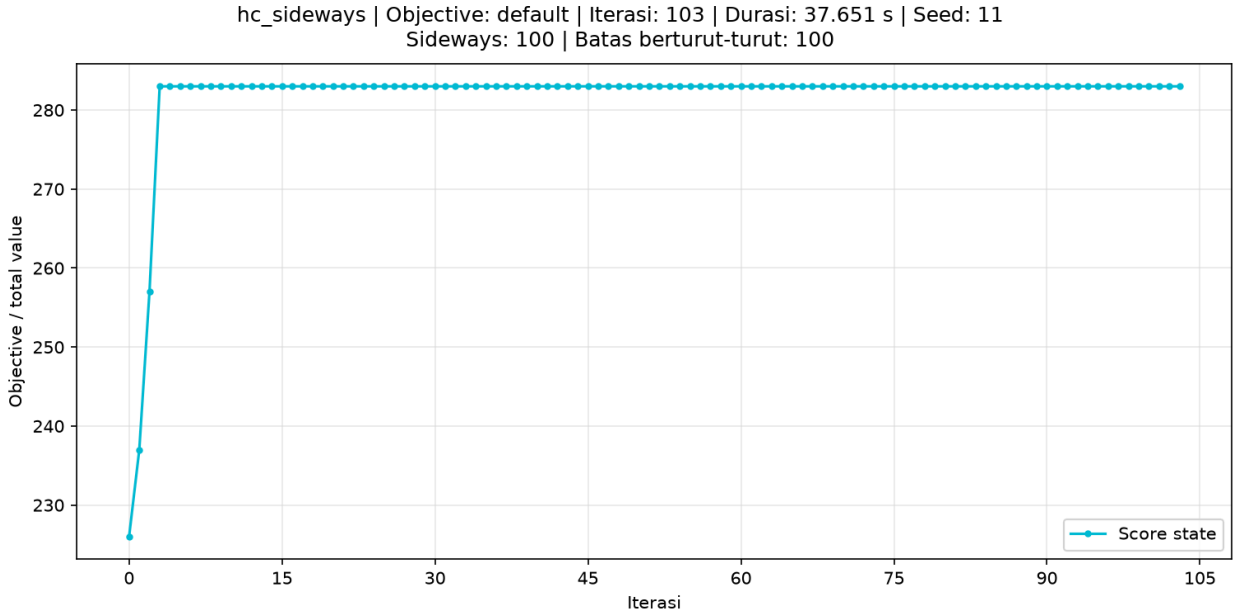
- Score Awal: 226 (berat awal: 31/40)
- Score Akhir: 283 (berat akhir: 40/40)
- Nilai Objective Function Akhir: 283
- Durasi Pencarian: 37,651 detik
- Banyak Iterasi hingga Berhenti: 103
- Maximum Sideways Move: 100 langkah berturut-turut
- Total Sideways Move: 100
- Sideways Berturut-turut pada Akhir Pencarian: 100
- Alasan Berhenti: Batas maksimum sideways move berturut-turut tercapai (*max_sideways_moves*)

Tabel 2.2.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Sideways Move HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(2,4,3)	Luar	(55,0,0)	(2,3,4)
P02	Luar	(13,0,0)	(3,2,3)	Dalam	(3,1,0)	(3,2,3)
P03	Luar	(19,0,0)	(2,4,2)	Luar	(67,0,0)	(4,2,2)
P04	Dalam	(3,1,0)	(2,3,2)	Luar	(25,0,0)	(3,2,2)
P05	Dalam	(0,1,2)	(3,2,2)	Dalam	(0,1,2)	(3,2,2)
P06	Dalam	(1,2,0)	(2,2,2)	Dalam	(1,2,0)	(2,2,2)
P07	Dalam	(4,0,0)	(2,1,3)	Dalam	(0,0,0)	(2,1,3)
P08	Dalam	(0,3,2)	(3,1,2)	Dalam	(3,3,0)	(3,1,2)
P09	Dalam	(1,1,0)	(2,1,2)	Dalam	(1,1,0)	(2,1,2)
P10	Dalam	(0,0,0)	(1,1,3)	Dalam	(3,0,0)	(3,1,1)
P11	Dalam	(0,2,0)	(1,2,1)	Dalam	(0,3,0)	(1,1,2)
P12	Luar	(73,0,0)	(5,1,2)	Dalam	(0,3,2)	(5,1,2)



Gambar 2.2.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Sideways Move HC



Gambar 2.2.2 Plot Nilai Fungsi Objektif terhadap Iterasi Run 1 Sideways Move HC

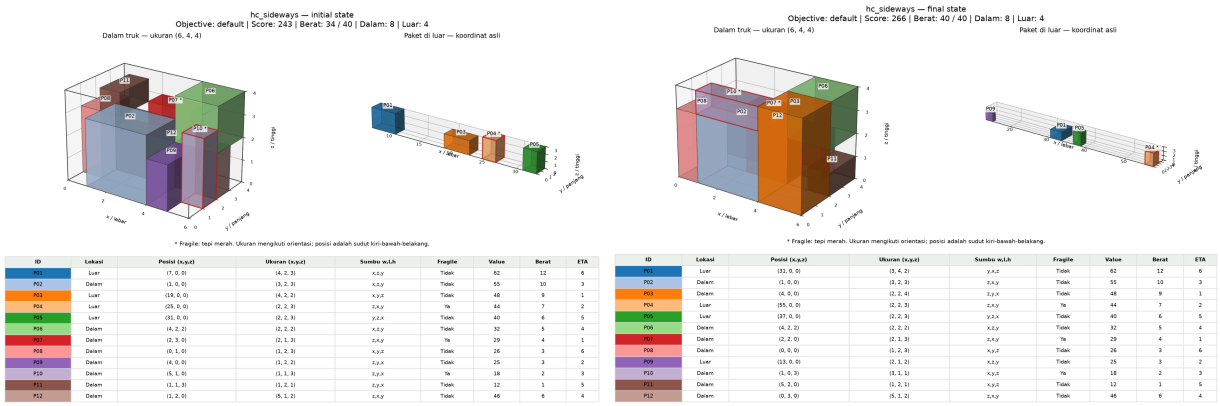
Run 2

Untuk run 2 dengan seed 22, berikut adalah hasil akhir yang didapatkan.

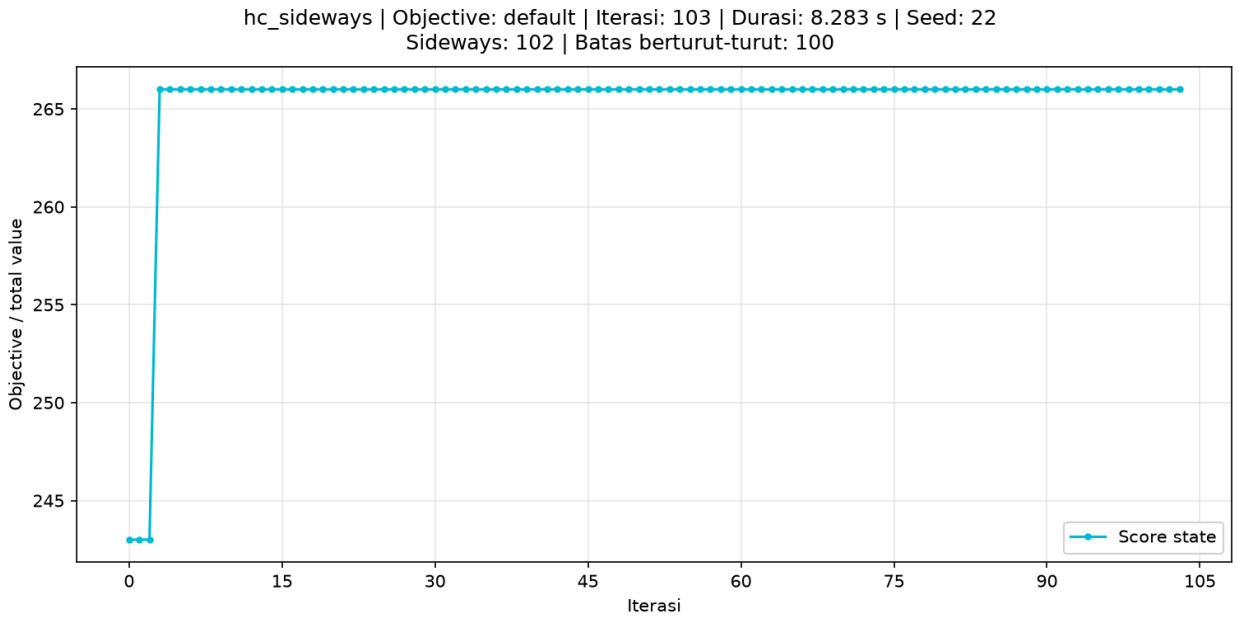
- Score Awal: 243 (berat awal: 34/40)
- Score Akhir: 266 (berat akhir: 40/40)
- Nilai Objective Function Akhir: 266
- Durasi Pencarian: 8,283 detik
- Banyak Iterasi hingga Berhenti: 103
- Maximum Sideways Move: 100 langkah berturut-turut
- Total Sideways Move: 102
- Sideways Berturut-turut pada Akhir Pencarian: 100
- Alasan Berhenti: Batas maksimum sideways move berturut-turut tercapai (*max_sideways_moves*)

Tabel 2.2.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Sideways Move HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(4,2,3)	Luar	(31,0,0)	(3,4,2)
P02	Dalam	(1,0,0)	(3,2,3)	Dalam	(1,0,0)	(3,2,3)
P03	Luar	(19,0,0)	(4,2,2)	Dalam	(4,0,0)	(2,2,4)
P04	Luar	(25,0,0)	(2,2,3)	Luar	(55,0,0)	(2,2,3)
P05	Luar	(31,0,0)	(2,2,3)	Luar	(37,0,0)	(2,2,3)
P06	Dalam	(4,2,2)	(2,2,2)	Dalam	(4,2,2)	(2,2,2)
P07	Dalam	(2,3,0)	(2,1,3)	Dalam	(2,2,0)	(2,1,3)
P08	Dalam	(0,1,0)	(1,2,3)	Dalam	(0,0,0)	(1,2,3)
P09	Dalam	(4,0,0)	(1,2,2)	Luar	(13,0,0)	(2,1,2)
P10	Dalam	(5,1,0)	(1,1,3)	Dalam	(1,0,3)	(3,1,1)
P11	Dalam	(1,1,3)	(1,2,1)	Dalam	(5,2,0)	(1,2,1)
P12	Dalam	(1,2,0)	(5,1,2)	Dalam	(0,3,0)	(5,1,2)



Gambar 2.2.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Sideways Move HC



Gambar 2.2.4 Plot Nilai Fungsi Objektif terhadap Iterasi Run 2 Sideways Move HC

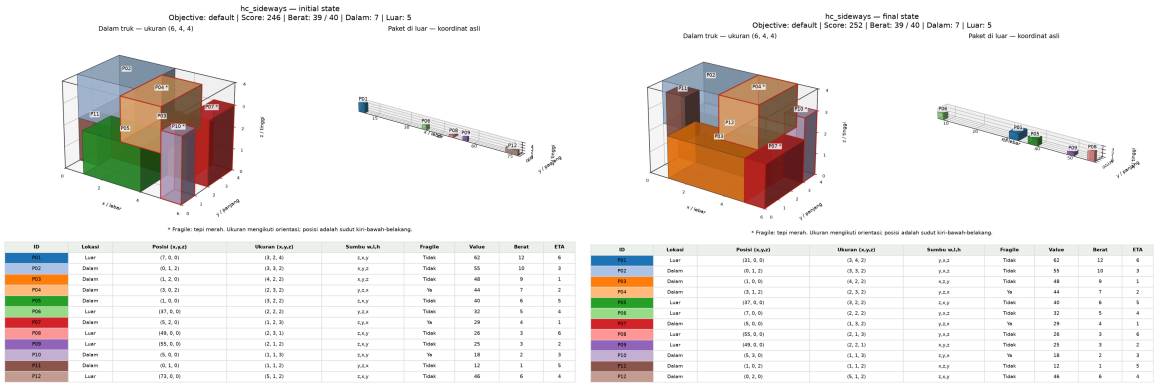
Run 3

Untuk run 3 dengan seed 33, berikut adalah hasil akhir yang didapatkan.

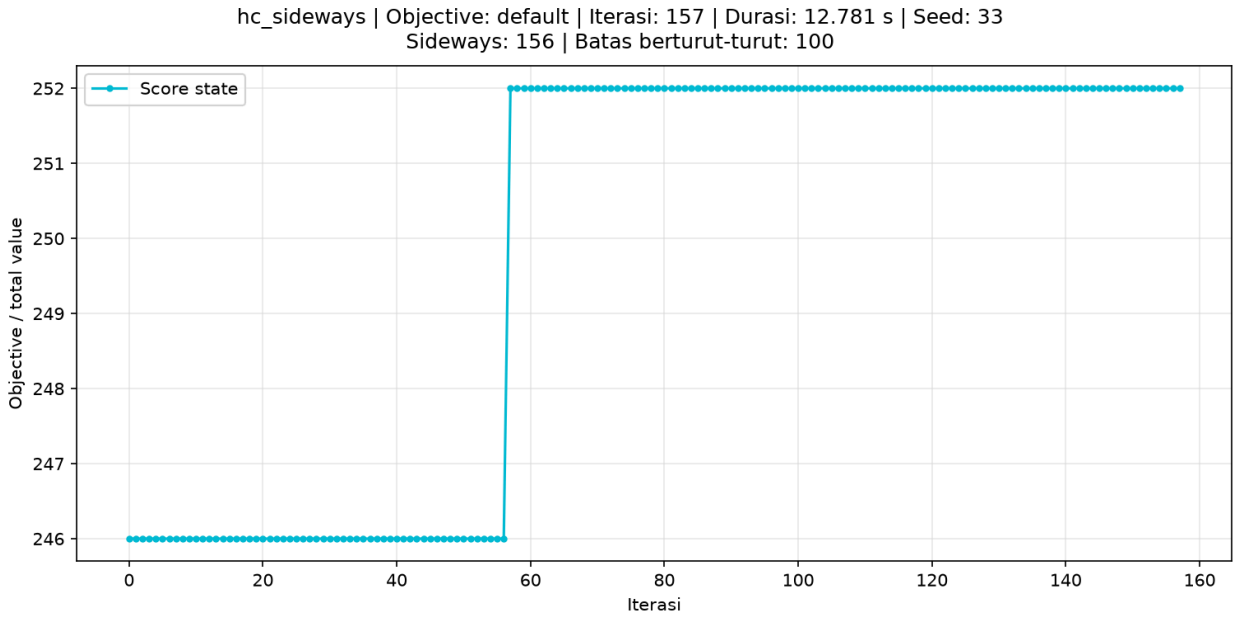
- Score Awal: 246 (berat awal: 39/40)
- Score Akhir: 252 (berat akhir: 39/40)
- Nilai Objective Function Akhir: 252
- Durasi Pencarian: 12,781 detik
- Banyak Iterasi hingga Berhenti: 157
- Maximum Sideways Move: 100 langkah berturut-turut
- Total Sideways Move: 156
- Sideways Berturut-turut pada Akhir Pencarian: 100
- Alasan Berhenti: Batas maksimum sideways move berturut-turut tercapai (*max_sideways_moves*)

Tabel 2.2.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Sideways Move HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(3,2,4)	Luar	(31,0,0)	(3,4,2)
P02	Dalam	(0,1,2)	(3,3,2)	Dalam	(0,1,2)	(3,3,2)
P03	Dalam	(1,2,0)	(4,2,2)	Dalam	(1,0,0)	(4,2,2)
P04	Dalam	(3,0,2)	(2,3,2)	Dalam	(3,1,2)	(2,3,2)
P05	Dalam	(1,0,0)	(3,2,2)	Luar	(37,0,0)	(3,2,2)
P06	Luar	(37,0,0)	(2,2,2)	Luar	(7,0,0)	(2,2,2)
P07	Dalam	(5,2,0)	(1,2,3)	Dalam	(5,0,0)	(1,3,2)
P08	Luar	(49,0,0)	(2,3,1)	Luar	(55,0,0)	(2,1,3)
P09	Luar	(55,0,0)	(2,1,2)	Luar	(49,0,0)	(2,2,1)
P10	Dalam	(5,0,0)	(1,1,3)	Dalam	(5,3,0)	(1,1,3)
P11	Dalam	(0,1,0)	(1,1,2)	Dalam	(1,0,2)	(1,1,2)
P12	Luar	(73,0,0)	(5,1,2)	Dalam	(0,2,0)	(5,1,2)



Gambar 2.2.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Sideways Move HC



Gambar 2.2.6 Plot Nilai Fungsi Objektif terhadap Iterasi Run 3 Sideways Move HC

Hill Climbing(Random Restart)

Deskripsi Fungsi/Kelas

1. Fungsi solve

```
def solve(
    problem: Problem, *, seed: int = 11, max_iterations: int | None = None,
    max_restarts: int = 10, objective: str | None = None,
) -> SearchResult:
    _validate_parameters(seed, max_iterations)
    _nonnegative_limit(max_restarts, "max_restarts")
    active_objective = set_objective(objective) if objective is not None else get_objective()
    initial_rng = Random(seed)
    initial_state = generate_initial_state(problem, initial_rng)
    rng = Random(seed + 1)

    started = perf_counter()
    initialization_seconds = 0.0
    current = best_state = initial_state
    current_score = best_score = score(problem, current)
    upper_bound = objective_upper_bound(problem, objective=active_objective)
    history = [{
        "iteration": 0, "score": current_score, "best_score": best_score,
        "restart": 0, "restart_iteration": 0,
    }]
    runs = []
    iterations = 0
    stop_reason = "max_restarts"

    for restart in range(max_restarts + 1):
        if restart:
            before_initialization = perf_counter()
            current = generate_initial_state(problem, initial_rng)
            initialization_seconds += perf_counter() - before_initialization
            current_score = score(problem, current)
            if current_score > best_score:
                best_state, best_score = current, current_score
        run_initial_score = current_score
        run_iterations = 0
        run_stop_reason = "max_iterations"

        while max_iterations is None or run_iterations < max_iterations:
            neighbor, neighbor_score = _best_neighbor(problem, current, current_score, rng)
            run_iterations += 1
            iterations += 1
            if neighbor is not None:
                current, current_score = neighbor, neighbor_score
```

```
            if current_score > best_score:
                best_state, best_score = current, current_score
        history.append({
            "iteration": iterations, "score": current_score, "best_score": best_score,
            "restart": restart, "restart_iteration": run_iterations,
        })
        if best_score >= upper_bound:
            run_stop_reason = "objective_upper_bound"
            break
        if neighbor is None:
            run_stop_reason = "local_optimum"
            break

    runs.append({
        "restart": restart, "iterations": run_iterations,
        "initial_score": run_initial_score, "final_score": current_score,
        "stop_reason": run_stop_reason,
    })
    if best_score >= upper_bound:
        stop_reason = "objective_upper_bound"
        break

duration_seconds = perf_counter() - started - initialization_seconds
return SearchResult(
    algorithm="hc_random_restart", initial_state=initial_state,
    final_state=best_state, final_score=best_score, best_state=best_state,
    iterations=iterations, duration_seconds=duration_seconds,
    seed=seed, stop_reason=stop_reason,
    parameters={
        "max_iterations": max_iterations,
        "max_restarts": max_restarts,
        "objective": active_objective,
    },
    history=history,
    metrics={
        "restarts": len(runs) - 1,
        "iterations_per_restart": [run["iterations"] for run in runs],
        "runs": runs,
    },
)
```

Fungsi solve menjalankan serangkaian pencarian Steepest Ascent dari konfigurasi awal acak yang berbeda. Setelah validasi parameter dan pemilihan objective, initial state dibangkitkan menggunakan Random(seed), sedangkan pencarian menggunakan Random(seed + 1). Setiap percobaan memilih neighbor terbaik dan menerima peningkatan score hingga tidak ditemukan

perbaikan atau batas iterasi per percobaan tercapai, kemudian pencarian diulang dari initial state baru dengan melanjutkan RNG inisialisasi. Fungsi mempertahankan best_state dan best_score dari seluruh percobaan, sehingga state akhir yang dikembalikan merupakan solusi terbaik yang ditemukan. Proses keseluruhan berhenti ketika maksimum restart tercapai atau score mencapai batas atas objective dari objective_upper_bound. History mencatat score saat ini, score terbaik, nomor restart, serta iterasi lokal dan global, sedangkan metrics menyimpan jumlah restart dan rincian tiap percobaan. Durasi diukur menggunakan perf_counter() dengan mengecualikan inisialisasi awal maupun inisialisasi ulang. Pada eksperimen digunakan maksimum **10 restart setelah percobaan awal**, sehingga terdapat paling banyak **11 percobaan**, tanpa batas iterasi tambahan per percobaan.

Fungsi _validate_parameters, _nonnegative_limit, dan _best_neighbor diimpor dari modul Steepest Ascent sehingga untuk deskripsi dan penjelasannya sama dengan yang ada pada Hill Climbing(Steepest Ascent).

Tabel 2.3.1 Hasil Eksperimen Hill Climbing(Random Restart)

Run (Seed)	Score Awal	Score Akhir	Nilai Objective Akhir	Berat Akhir	Total Iterasi	Maximum Restart	Restart Aktual	Alasan Berhenti	Durasi Pencarian (s)
1 (11)	226	283	283	40/40	20	10	10	max_restarts	1,688
2 (22)	243	283	283	40/40	20	10	10	max_restarts	1,718
3 (33)	246	276	276	39/40	16	10	10	max_restarts	1,360
Rata-rata	**238,33**	**280,67**	**280,67**	**39,67/40**	**18,67**	**10**	**10**	—	**1,589**

Tabel 2.3.2 Jumlah Iterasi per Percobaan pada Eksperimen Random Restart Hill-Climbing

Percobaan	Run 1 (Seed 11)	Run 2 (Seed 22)	Run 3 (Seed 33)
Awal	4	1	1
Restart 1	1	1	1
Restart 2	3	1	3

Restart 3	1	1	1
Restart 4	3	2	2
Restart 5	2	5	1
Restart 6	1	3	2
Restart 7	1	1	1
Restart 8	1	3	1
Restart 9	1	1	2
Restart 10	2	1	1
Total	**20**	**20**	**16**

Run 1

Untuk run 1 dengan seed 11, berikut adalah hasil akhir yang didapatkan.

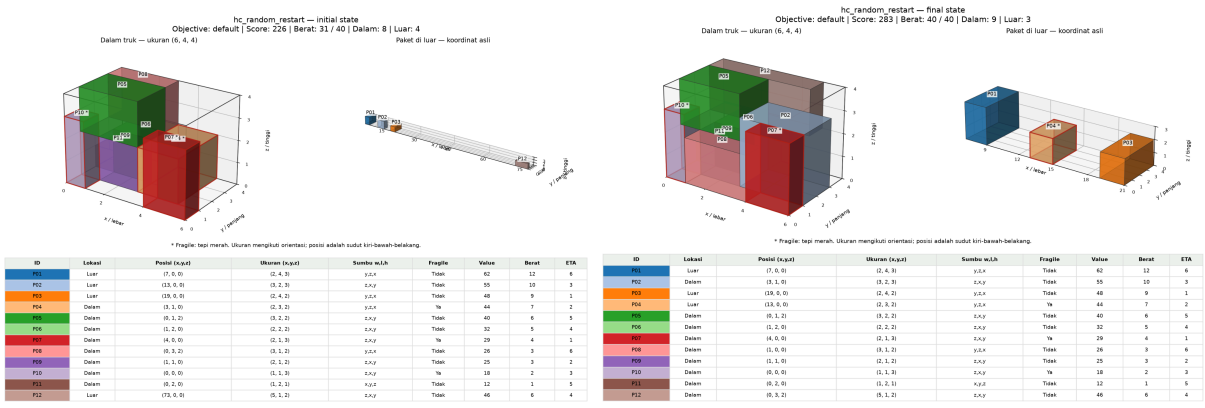
- Score Awal: 226 (berat awal: 31/40)
- Score Akhir: 283 (berat akhir: 40/40)
- Nilai Objective Function Akhir: 283
- Durasi Pencarian: 1,688 detik
- Maximum Restart: 10
- Banyak Restart Aktual: 10
- Jumlah Percobaan: 11, termasuk percobaan awal
- Total Iterasi: 20
- Alasan Berhenti: Batas maksimum restart tercapai (*max_restarts*)

Tabel 2.3.3 Rincian Percobaan Run 1 Random Restart Hill-Climbing

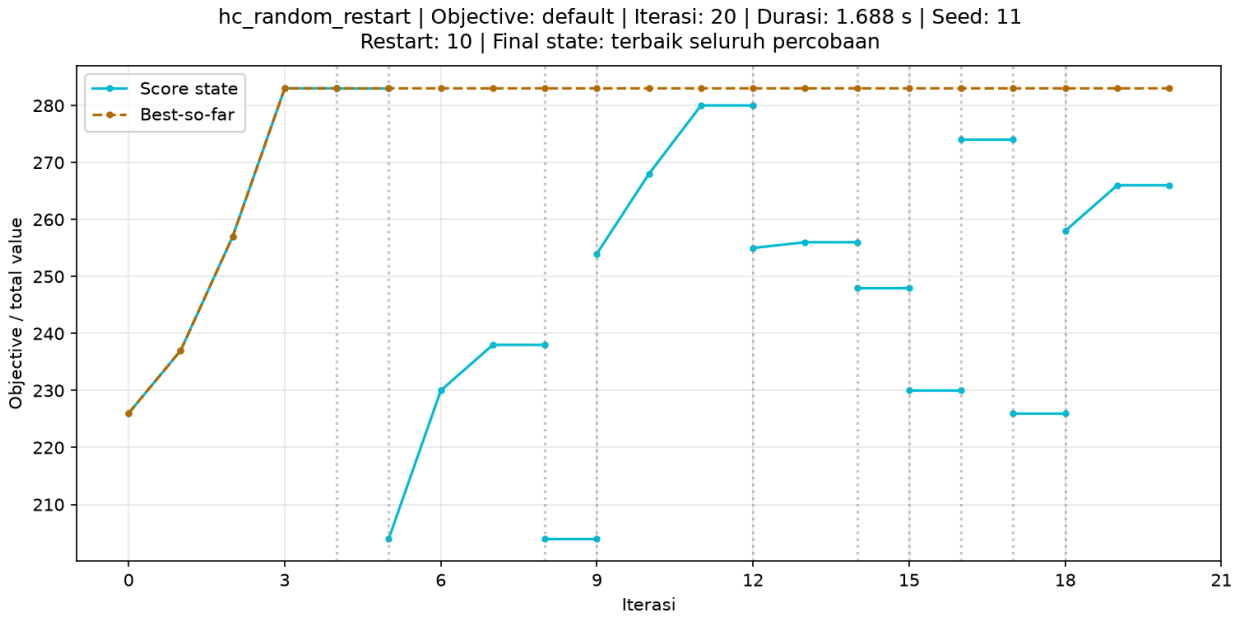
Percobaan	Score Awal	Score Akhir Percobaan	Banyak Iterasi
Awal	226	283	4
Restart 1	283	283	1
Restart 2	204	238	3
Restart 3	204	204	1
Restart 4	254	280	3
Restart 5	255	256	2
Restart 6	248	248	1
Restart 7	230	230	1
Restart 8	274	274	1
Restart 9	226	226	1
Restart 10	258	266	2
Total	—	—	**20**

Tabel 2.3.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Random Restart HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(2,4,3)	Luar	(7,0,0)	(2,4,3)
P02	Luar	(13,0,0)	(3,2,3)	Dalam	(3,1,0)	(3,2,3)
P03	Luar	(19,0,0)	(2,4,2)	Luar	(19,0,0)	(2,4,2)
P04	Dalam	(3,1,0)	(2,3,2)	Luar	(13,0,0)	(2,3,2)
P05	Dalam	(0,1,2)	(3,2,2)	Dalam	(0,1,2)	(3,2,2)
P06	Dalam	(1,2,0)	(2,2,2)	Dalam	(1,2,0)	(2,2,2)
P07	Dalam	(4,0,0)	(2,1,3)	Dalam	(4,0,0)	(2,1,3)
P08	Dalam	(0,3,2)	(3,1,2)	Dalam	(1,0,0)	(3,1,2)
P09	Dalam	(1,1,0)	(2,1,2)	Dalam	(1,1,0)	(2,1,2)
P10	Dalam	(0,0,0)	(1,1,3)	Dalam	(0,0,0)	(1,1,3)
P11	Dalam	(0,2,0)	(1,2,1)	Dalam	(0,2,0)	(1,2,1)
P12	Luar	(73,0,0)	(5,1,2)	Dalam	(0,3,2)	(5,1,2)



Gambar 2.3.1 Visualisasi Konfigurasi Awal dan Akhir Terbaik Run 1 Random Restart HC



Gambar 2.3.2 Plot Nilai Fungsi Objektif dan Nilai Terbaik terhadap Iterasi Run 1 Random Restart Hill-Climbing

Run 2

Untuk run 2 dengan seed 22, berikut adalah hasil akhir yang didapatkan.

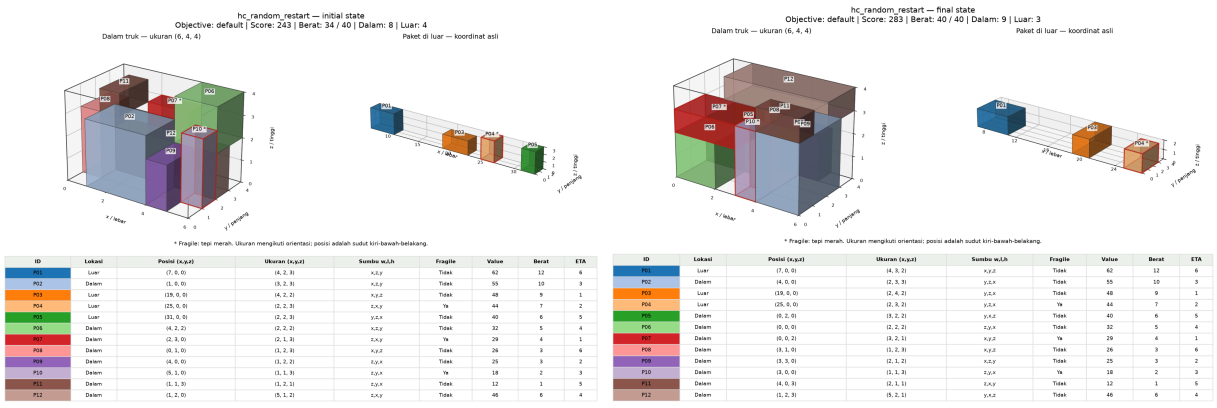
- Score Awal: 243 (berat awal: 34/40)
- Score Akhir: 283 (berat akhir: 40/40)
- Nilai Objective Function Akhir: 283
- Durasi Pencarian: 1,718 detik
- Maximum Restart: 10
- Banyak Restart Aktual: 10
- Jumlah Percobaan: 11, termasuk percobaan awal
- Total Iterasi: 20
- Alasan Berhenti: Batas maksimum restart tercapai (*max_restarts*)

Tabel 2.3.5 Rincian Percobaan Run 2 Random Restart Hill-Climbing

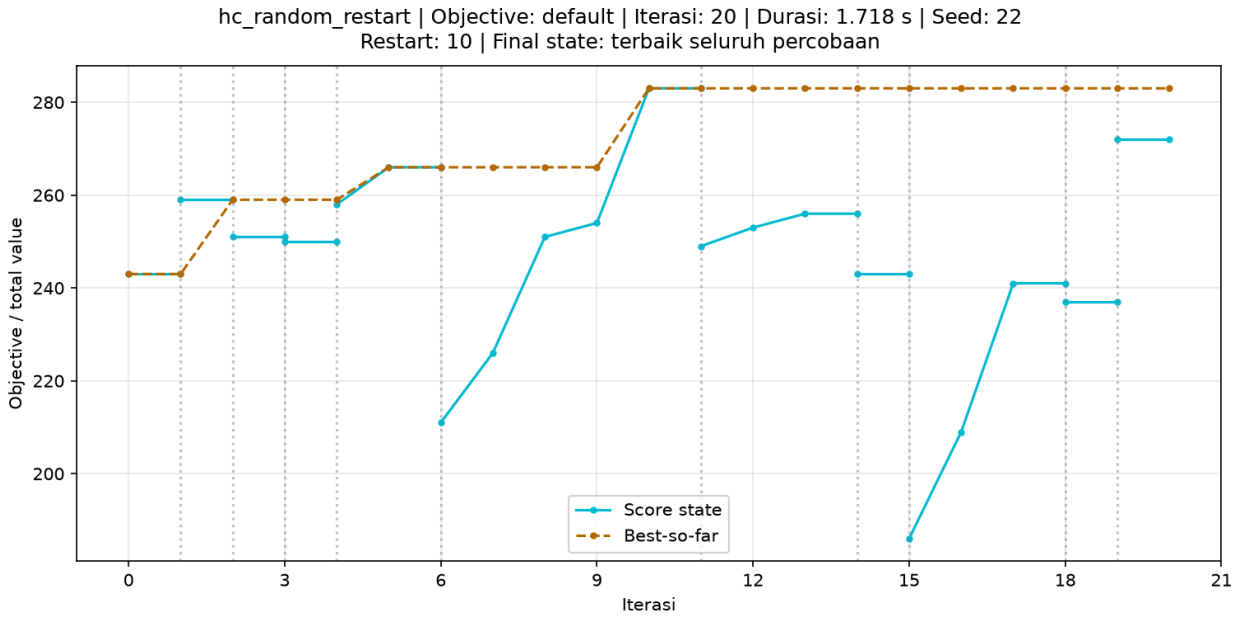
Percobaan	Score Awal	Score Akhir Percobaan	Banyak Iterasi
Awal	243	243	1
Restart 1	259	259	1
Restart 2	251	251	1
Restart 3	250	250	1
Restart 4	258	266	2
Restart 5	211	283	5
Restart 6	249	256	3
Restart 7	243	243	1
Restart 8	186	241	3
Restart 9	237	237	1
Restart 10	272	272	1
Total	—	—	**20**

Tabel 2.3.6 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Random Restart HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(4,2,3)	Luar	(7,0,0)	(4,3,2)
P02	Dalam	(1,0,0)	(3,2,3)	Dalam	(4,0,0)	(2,3,3)
P03	Luar	(19,0,0)	(4,2,2)	Luar	(19,0,0)	(2,4,2)
P04	Luar	(25,0,0)	(2,2,3)	Luar	(25,0,0)	(2,3,2)
P05	Luar	(31,0,0)	(2,2,3)	Dalam	(0,2,0)	(3,2,2)
P06	Dalam	(4,2,2)	(2,2,2)	Dalam	(0,0,0)	(2,2,2)
P07	Dalam	(2,3,0)	(2,1,3)	Dalam	(0,0,2)	(3,2,1)
P08	Dalam	(0,1,0)	(1,2,3)	Dalam	(3,1,0)	(1,2,3)
P09	Dalam	(4,0,0)	(1,2,2)	Dalam	(3,3,0)	(2,1,2)
P10	Dalam	(5,1,0)	(1,1,3)	Dalam	(3,0,0)	(1,1,3)
P11	Dalam	(1,1,3)	(1,2,1)	Dalam	(4,0,3)	(2,1,1)
P12	Dalam	(1,2,0)	(5,1,2)	Dalam	(1,2,3)	(5,2,1)



Gambar 2.3.3 Visualisasi Konfigurasi Awal dan Akhir Terbaik Run 2 Random Restart HC



Gambar 2.3.4 Plot Nilai Fungsi Objektif dan Nilai Terbaik terhadap Iterasi Run 2 Random Restart Hill-Climbing

Run 3

Untuk run 3 dengan seed 33, berikut adalah hasil akhir yang didapatkan.

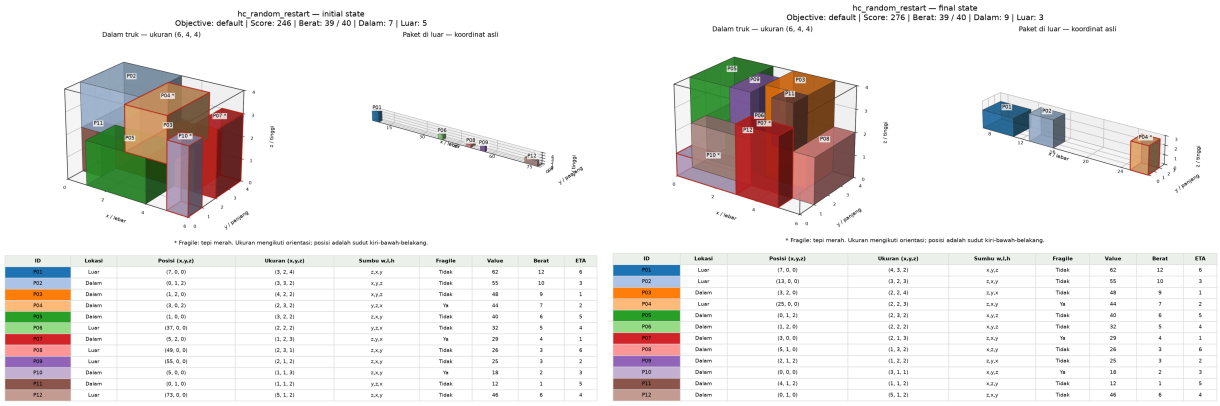
- Score Awal: 246 (berat awal: 39/40)
- Score Akhir: 276 (berat akhir: 39/40)
- Nilai Objective Function Akhir: 276
- Durasi Pencarian: 1,360 detik
- Maximum Restart: 10
- Banyak Restart Aktual: 10
- Jumlah Percobaan: 11, termasuk percobaan awal
- Total Iterasi: 16
- Alasan Berhenti: Batas maksimum restart tercapai (*max_restarts*)

Tabel 2.3.7 Rincian Percobaan Run 3 Random Restart Hill-Climbing

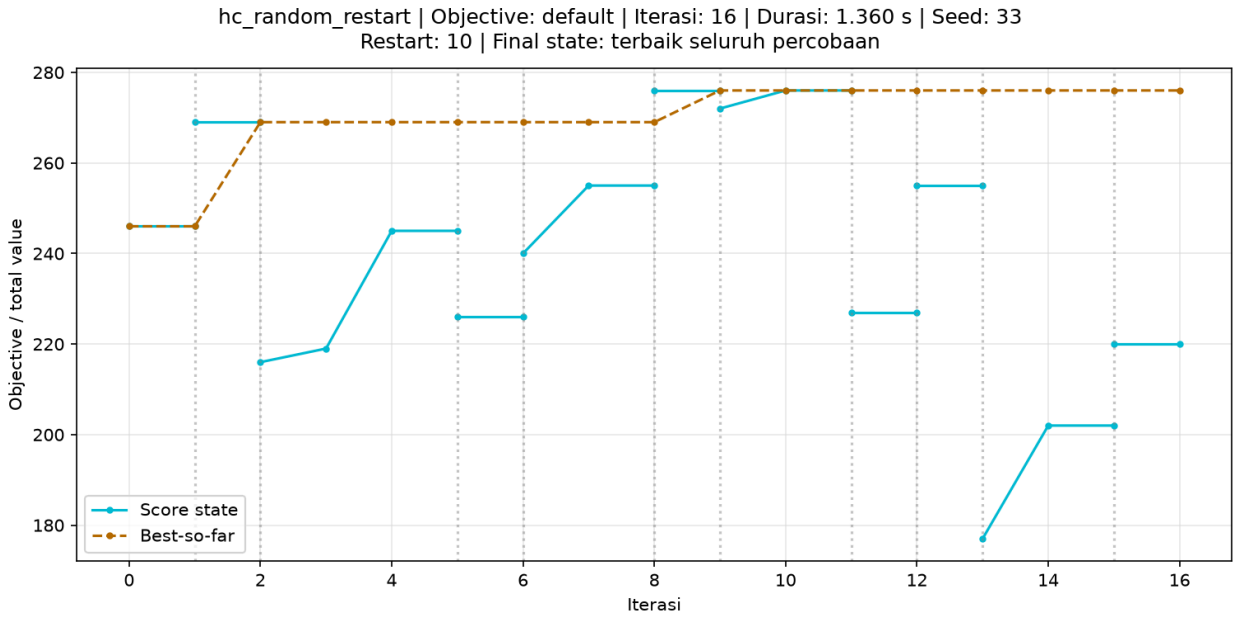
Percobaan	Score Awal	Score Akhir Percobaan	Banyak Iterasi
Awal	246	246	1
Restart 1	269	269	1
Restart 2	216	245	3
Restart 3	226	226	1
Restart 4	240	255	2
Restart 5	276	276	1
Restart 6	272	276	2
Restart 7	227	227	1
Restart 8	255	255	1
Restart 9	177	202	2
Restart 10	220	220	1
Total	—	—	**16**

Tabel 2.3.8 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Random Restart HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(3,2,4)	Luar	(7,0,0)	(4,3,2)
P02	Dalam	(0,1,2)	(3,3,2)	Luar	(13,0,0)	(3,2,3)
P03	Dalam	(1,2,0)	(4,2,2)	Dalam	(3,2,0)	(2,2,4)
P04	Dalam	(3,0,2)	(2,3,2)	Luar	(25,0,0)	(2,2,3)
P05	Dalam	(1,0,0)	(3,2,2)	Dalam	(0,1,2)	(2,3,2)
P06	Luar	(37,0,0)	(2,2,2)	Dalam	(1,2,0)	(2,2,2)
P07	Dalam	(5,2,0)	(1,2,3)	Dalam	(3,0,0)	(2,1,3)
P08	Luar	(49,0,0)	(2,3,1)	Dalam	(5,1,0)	(1,3,2)
P09	Luar	(55,0,0)	(2,1,2)	Dalam	(2,1,2)	(1,2,2)
P10	Dalam	(5,0,0)	(1,1,3)	Dalam	(0,0,0)	(3,1,1)
P11	Dalam	(0,1,0)	(1,1,2)	Dalam	(4,1,2)	(1,1,2)
P12	Luar	(73,0,0)	(5,1,2)	Dalam	(0,1,0)	(5,1,2)



Gambar 2.3.5 Visualisasi Konfigurasi Awal dan Akhir Terbaik Run 3 Random Restart HC



Gambar 2.3.6 Plot Nilai Fungsi Objektif dan Nilai Terbaik terhadap Iterasi Run 3 Random Restart Hill-Climbing

Hill Climbing(Stochastic)

Deskripsi Fungsi/Kelas

1. Fungsi solve

```
def solve(
    problem: Problem, *, seed: int = 11, max_iterations: int = 1000,
    objective: str | None = None,
) -> SearchResult:
    _validate_parameters(seed, max_iterations)
    if max_iterations is None:
        raise ValueError("max_iterations harus berupa integer positif.")
    active_objective = set_objective(objective) if objective is not None else get_objective()
    initial_state = generate_initial_state(problem, Random(seed))
    rng = Random(seed + 1)

    started = perf_counter()
    current = initial_state
    current_score = score(problem, current)
    accepted_moves = rejected_moves = sample_failures = 0
    history = [{"iteration": 0, "score": current_score, "candidate_score": None, "accepted": None}]

    for iteration in range(1, max_iterations + 1):
        neighbor = sample_neighbor(problem, current, rng)
        neighbor_score = None
        accepted = None
        if neighbor is None:
            sample_failures += 1
        else:
            neighbor_score = score(problem, neighbor)
            accepted = neighbor_score > current_score
            if accepted:
                current, current_score = neighbor, neighbor_score
                accepted_moves += 1
            else:
                rejected_moves += 1
        history.append({
            "iteration": iteration, "score": current_score,
            "candidate_score": neighbor_score, "accepted": accepted,
        })

    duration_seconds = perf_counter() - started
    return SearchResult(
        algorithm="hc_stochastic", initial_state=initial_state, final_state=current,
        final_score=current_score, iterations=max_iterations,
        duration_seconds=duration_seconds, seed=seed, stop_reason="max_iterations",
        parameters={"max_iterations": max_iterations, "objective": active_objective},
```

```
    history=history,
    metrics={
        "accepted_moves": accepted_moves, "rejected_moves": rejected_moves,
        "sample_failures": sample_failures,
    },
)
```

Fungsi solve menjalankan Stochastic Hill-Climbing dengan mengambil satu kandidat neighbor acak melalui sample_neighbor pada setiap iterasi, sesuai pseudocode materi kuliah halaman 29. Proses diawali dengan validasi parameter, pemilihan objective, dan pembangkitan initial state menggunakan Random(seed), sedangkan pencarian menggunakan Random(seed + 1). Kandidat hanya diterima apabila score-nya lebih tinggi daripada state saat ini; kandidat dengan score sama atau lebih rendah ditolak. Jika sampling gagal menghasilkan kandidat, fungsi menambah counter sample_failures dan melanjutkan pencarian. Setiap langkah tetap dihitung sebagai satu iterasi, termasuk ketika kandidat ditolak atau sampling gagal. Fungsi mencatat score saat ini, score kandidat, dan keputusan penerimaan pada history, serta menghitung jumlah langkah diterima, ditolak, dan kegagalan sampling. Durasi pencarian diukur menggunakan perf_counter() setelah inisialisasi selesai, kemudian hasil dikembalikan sebagai SearchResult. Pada eksperimen digunakan batas **1000 iterasi**, sehingga pencarian berhenti karena max_iterations dan tidak langsung berhenti ketika satu kandidat gagal memberikan perbaikan.

Fungsi _validate_parameters, _nonnegative_limit, dan _best_neighbor diimpor dari modul Steepest Ascent sehingga untuk deskripsi dan penjelasannya sama dengan yang ada pada Hill Climbing(Steepest Ascent).

Tabel 2.4.1 Hasil Eksperimen Hill Climbing(Stochastic)

Run (Seed)	Score Awal	Score Akhir	Nilai Objective Akhir	Berat Akhir	Iterasi	Perbaikan Diterima	Kandidat Ditolak	Gagal Sampling	Alasan Berhenti
1 (11)	226	283	283	40/40	1000	4	996	0	max_iterations
2 (22)	243	243	243	34/40	1000	0	1000	0	max_iterations
3 (33)	246	246	246	39/40	1000	0	1000	0	max_iterations
Rata-rata	**238,33**	**257,33**	**257,33**	**37,67/40**	**1000**	**1,33**	**998,67**	**0**	—

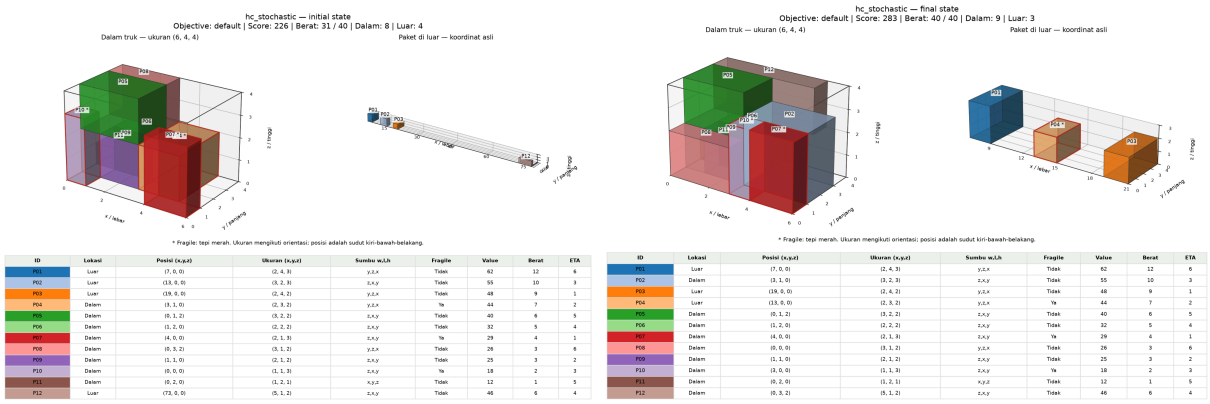
Run 1

Untuk run 1 dengan seed 11, berikut adalah hasil akhir yang didapatkan.

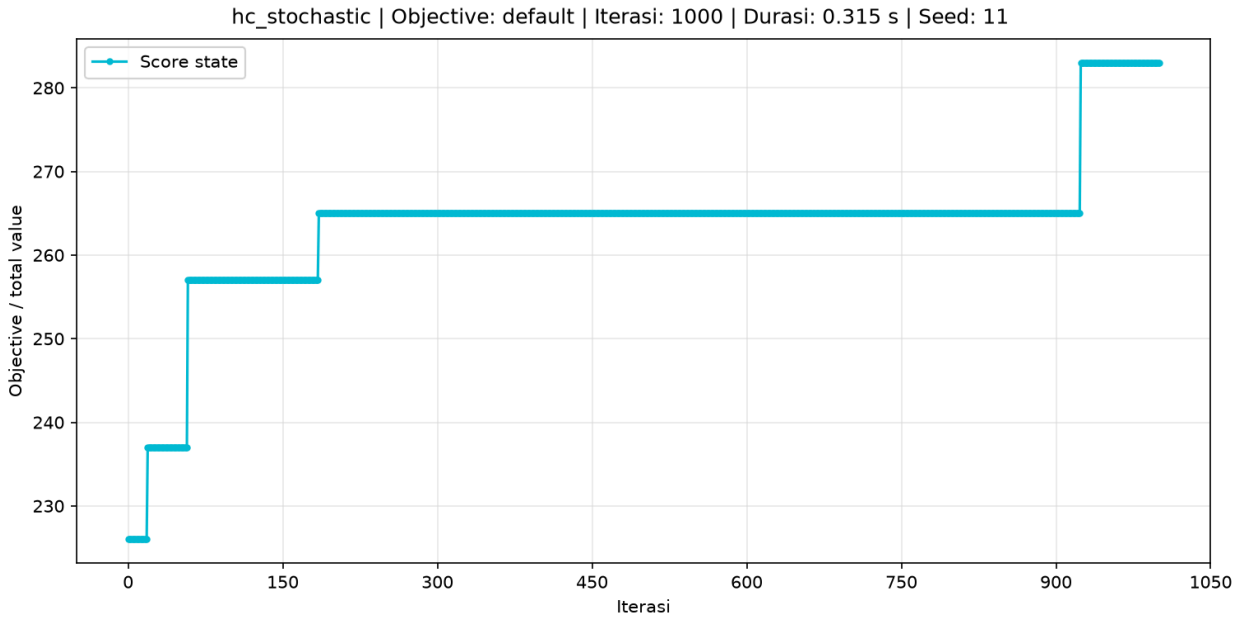
- Score Awal: **226** (berat awal: **31/40**)
- Score Akhir: **283** (berat akhir: **40/40**)
- Nilai Objective Function Akhir: **283**
- Durasi Pencarian: **0,315 detik**
- Banyak Iterasi hingga Berhenti: **1000**
- Batas Maksimum Iterasi: **1000**
- Langkah Perbaikan Diterima: **4**
- Langkah Kandidat Ditolak: **996**
- Iterasi dengan Kegagalan Sampling: **0**
- Alasan Berhenti: **Batas maksimum iterasi tercapai (*max_iterations*)**

Tabel 2.4.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Stochastic HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(2,4,3)	Luar	(7,0,0)	(2,4,3)
P02	Luar	(13,0,0)	(3,2,3)	Dalam	(3,1,0)	(3,2,3)
P03	Luar	(19,0,0)	(2,4,2)	Luar	(19,0,0)	(2,4,2)
P04	Dalam	(3,1,0)	(2,3,2)	Luar	(13,0,0)	(2,3,2)
P05	Dalam	(0,1,2)	(3,2,2)	Dalam	(0,1,2)	(3,2,2)
P06	Dalam	(1,2,0)	(2,2,2)	Dalam	(1,2,0)	(2,2,2)
P07	Dalam	(4,0,0)	(2,1,3)	Dalam	(4,0,0)	(2,1,3)
P08	Dalam	(0,3,2)	(3,1,2)	Dalam	(0,0,0)	(3,1,2)
P09	Dalam	(1,1,0)	(2,1,2)	Dalam	(1,1,0)	(2,1,2)
P10	Dalam	(0,0,0)	(1,1,3)	Dalam	(3,0,0)	(1,1,3)
P11	Dalam	(0,2,0)	(1,2,1)	Dalam	(0,2,0)	(1,2,1)
P12	Luar	(73,0,0)	(5,1,2)	Dalam	(0,3,2)	(5,1,2)



Gambar 2.4.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Stochastic HC



Gambar 2.4.2 Plot Nilai Fungsi Objektif terhadap Iterasi Run 1 Stochastic HC

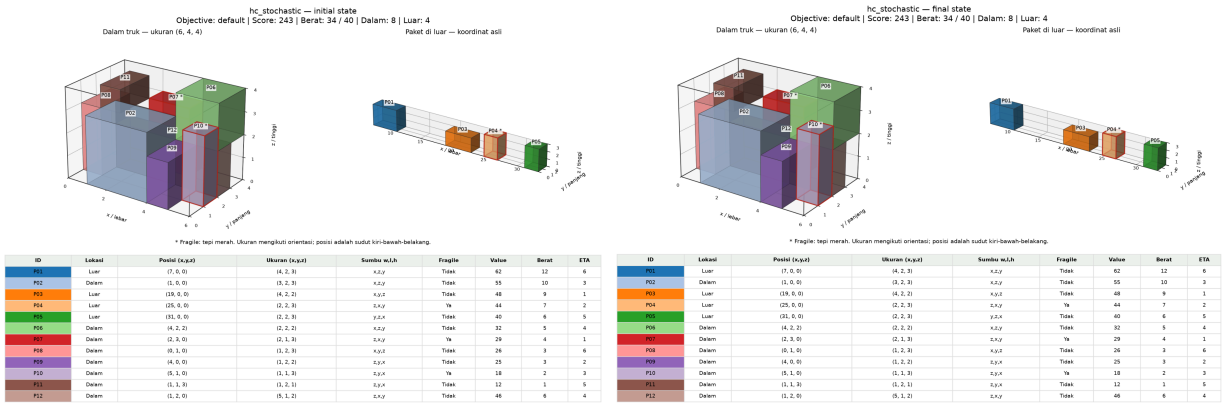
Run 2

Untuk run 2 dengan seed 22, berikut adalah hasil akhir yang didapatkan.

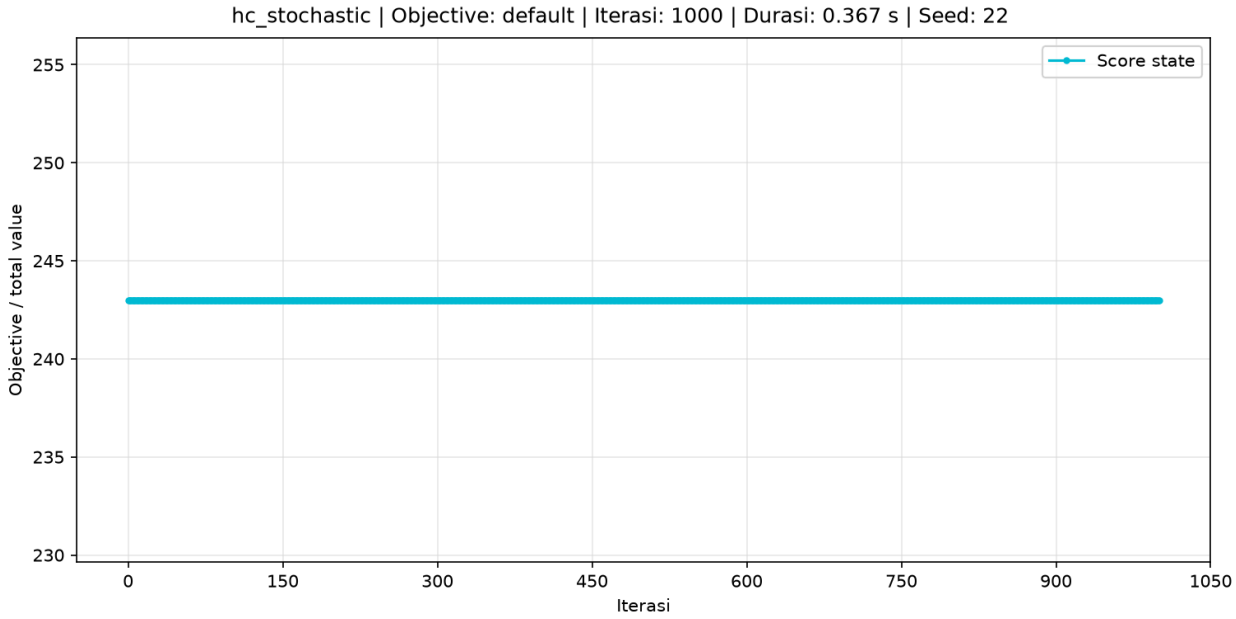
- Score Awal: **243** (berat awal: **34/40**)
- Score Akhir: **243** (berat akhir: **34/40**)
- Nilai Objective Function Akhir: **243**
- Durasi Pencarian: **0,367 detik**
- Banyak Iterasi hingga Berhenti: **1000**
- Batas Maksimum Iterasi: **1000**
- Langkah Perbaikan Diterima: **0**
- Langkah Kandidat Ditolak: **1000**
- Iterasi dengan Kegagalan Sampling: **0**
- Alasan Berhenti: **Batas maksimum iterasi tercapai (*max_iterations*)**

Tabel 2.4.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Stochastic HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(4,2,3)	Luar	(7,0,0)	(4,2,3)
P02	Dalam	(1,0,0)	(3,2,3)	Dalam	(1,0,0)	(3,2,3)
P03	Luar	(19,0,0)	(4,2,2)	Luar	(19,0,0)	(4,2,2)
P04	Luar	(25,0,0)	(2,2,3)	Luar	(25,0,0)	(2,2,3)
P05	Luar	(31,0,0)	(2,2,3)	Luar	(31,0,0)	(2,2,3)
P06	Dalam	(4,2,2)	(2,2,2)	Dalam	(4,2,2)	(2,2,2)
P07	Dalam	(2,3,0)	(2,1,3)	Dalam	(2,3,0)	(2,1,3)
P08	Dalam	(0,1,0)	(1,2,3)	Dalam	(0,1,0)	(1,2,3)
P09	Dalam	(4,0,0)	(1,2,2)	Dalam	(4,0,0)	(1,2,2)
P10	Dalam	(5,1,0)	(1,1,3)	Dalam	(5,1,0)	(1,1,3)
P11	Dalam	(1,1,3)	(1,2,1)	Dalam	(1,1,3)	(1,2,1)
P12	Dalam	(1,2,0)	(5,1,2)	Dalam	(1,2,0)	(5,1,2)



Gambar 2.4.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Stochastic HC



Gambar 2.4.4 Plot Nilai Fungsi Objektif terhadap Iterasi Run 2 Stochastic HC

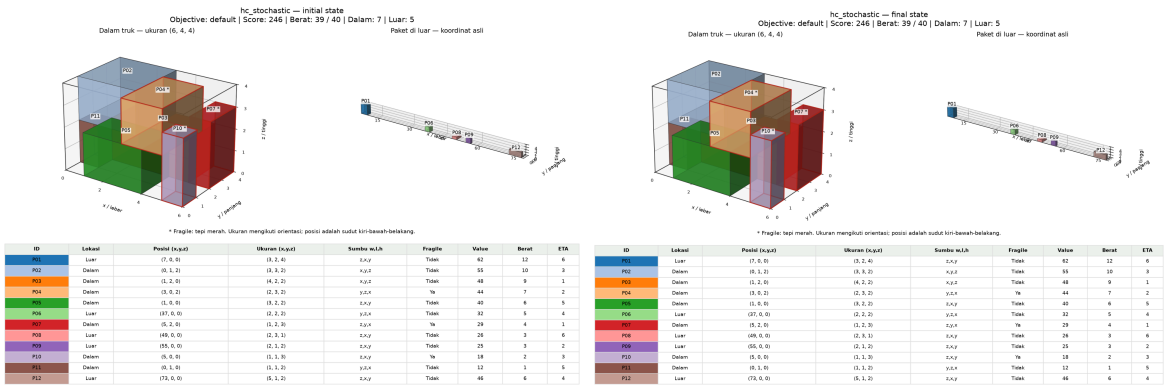
Run 3

Untuk run 3 dengan seed 33, berikut adalah hasil akhir yang didapatkan.

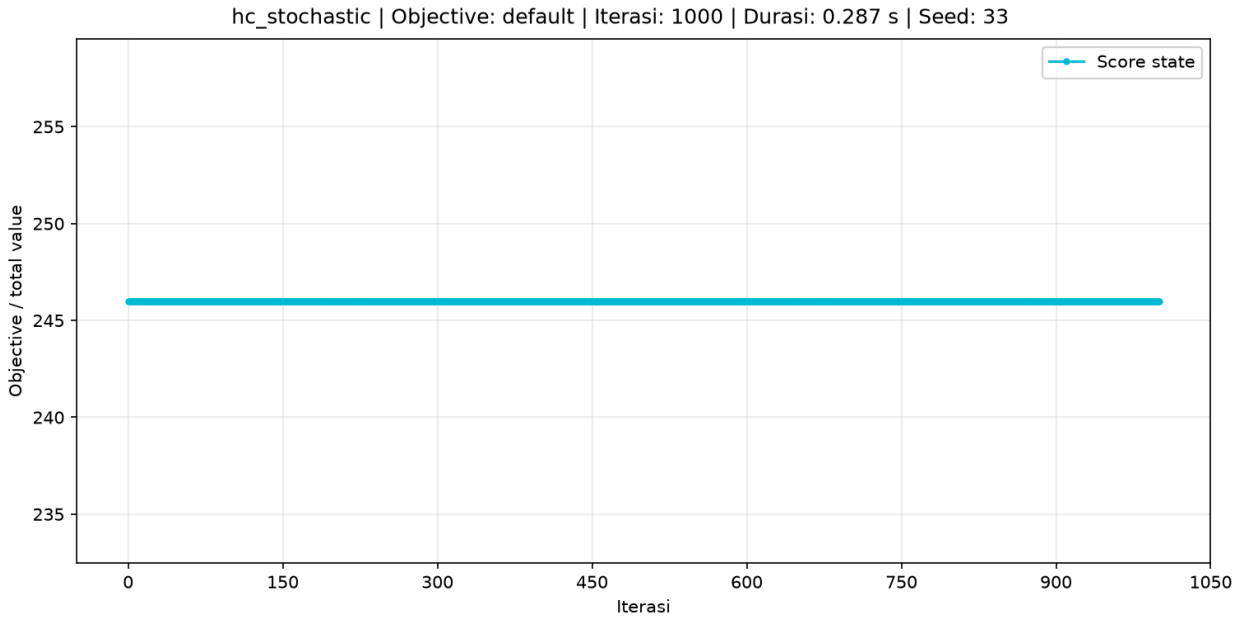
- Score Awal: **246** (berat awal: **39/40**)
- Score Akhir: **246** (berat akhir: **39/40**)
- Nilai Objective Function Akhir: **246**
- Durasi Pencarian: **0,287 detik**
- Banyak Iterasi hingga Berhenti: **1000**
- Batas Maksimum Iterasi: **1000**
- Langkah Perbaikan Diterima: **0**
- Langkah Kandidat Ditolak: **1000**
- Iterasi dengan Kegagalan Sampling: **0**
- Alasan Berhenti: **Batas maksimum iterasi tercapai (*max_iterations*)**

Tabel 2.4.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Stochastic HC

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(3,2,4)	Luar	(7,0,0)	(3,2,4)
P02	Dalam	(0,1,2)	(3,3,2)	Dalam	(0,1,2)	(3,3,2)
P03	Dalam	(1,2,0)	(4,2,2)	Dalam	(1,2,0)	(4,2,2)
P04	Dalam	(3,0,2)	(2,3,2)	Dalam	(3,0,2)	(2,3,2)
P05	Dalam	(1,0,0)	(3,2,2)	Dalam	(1,0,0)	(3,2,2)
P06	Luar	(37,0,0)	(2,2,2)	Luar	(37,0,0)	(2,2,2)
P07	Dalam	(5,2,0)	(1,2,3)	Dalam	(5,2,0)	(1,2,3)
P08	Luar	(49,0,0)	(2,3,1)	Luar	(49,0,0)	(2,3,1)
P09	Luar	(55,0,0)	(2,1,2)	Luar	(55,0,0)	(2,1,2)
P10	Dalam	(5,0,0)	(1,1,3)	Dalam	(5,0,0)	(1,1,3)
P11	Dalam	(0,1,0)	(1,1,2)	Dalam	(0,1,0)	(1,1,2)
P12	Luar	(73,0,0)	(5,1,2)	Luar	(73,0,0)	(5,1,2)



Gambar 2.4.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Stochastic HC



Gambar 2.4.6 Plot Nilai Fungsi Objektif terhadap Iterasi Run 3 Stochastic HC

Simulated Annealing

Deskripsi Fungsi/Kelas

```
def _validate_parameters(
    problem: Problem,
    seed: int,
    max_iterations: int,
    initial_temperature: int | float,
    cooling_rate: int | float,
    min_temperature: int | float,
) -> None:
    if not isinstance(problem, Problem):
        raise ValueError("problem harus berupa Problem.")
    if type(seed) is not int:
        raise ValueError("seed harus berupa integer.")
    if type(max_iterations) is not int or max_iterations <= 0:
        raise ValueError("max_iterations harus berupa bilangan bulat positif.")
    for name, value in (
        ("initial_temperature", initial_temperature),
        ("min_temperature", min_temperature),
    ):
        if type(value) not in (int, float) or not isfinite(value) or value <= 0:
            raise ValueError(f"{name} harus berupa angka positif berhingga.")
    if type(cooling_rate) not in (int, float) or not isfinite(cooling_rate) or not 0 <
cooling_rate < 1:
        raise ValueError("cooling_rate harus berada di antara 0 dan 1.")
    if min_temperature >= initial_temperature:
        raise ValueError("min_temperature harus lebih kecil dari initial_temperature.")
```

Fungsi `_validate_parameters` bertugas memvalidasi seluruh argumen konfigurasi sebelum proses pencarian Simulated Annealing dimulai. Fungsi ini memastikan bahwa objek persoalan bertipe `Problem`, nilai `seed` dan `max_iterations` adalah bilangan bulat yang valid, parameter suhu (`initial_temperature` dan `min_temperature`) bernilai positif berhingga dengan batas suhu minimum yang lebih kecil daripada suhu awal, serta laju pendinginan (`cooling_rate`) berada secara ketat pada rentang $(0, 1)$.

```
def _is_stuck(problem: Problem, state: State, current_score: int | float) -> bool:
    return not any(
        score(problem, neighbor) > current_score
        for neighbor in iter_neighbors(problem, state)
    )
```

Fungsi `_is_stuck` digunakan untuk mendeteksi apakah state saat ini sedang terjebak pada kondisi *local optimum* atau stagnan. Fungsi ini membangkitkan seluruh tetangga yang valid dari state sekarang menggunakan `iter_neighbors` dan mengembalikan nilai `True` apabila tidak ada satu pun tetangga yang memiliki skor objektif lebih tinggi daripada `current_score`.

```
def solve(
    problem: Problem, *, seed: int = 11, max_iterations: int = 1000,
    initial_temperature: float = 100.0, cooling_rate: float = 0.95,
    min_temperature: float = 0.001, objective: str | None = None,
) -> SearchResult:
    _validate_parameters(
        problem, seed, max_iterations,
        initial_temperature, cooling_rate, min_temperature,
    )
    active_objective = set_objective(objective) if objective is not None else
get_objective()
    initial_temp = float(initial_temperature)
    cooling = float(cooling_rate)
    min_temp = float(min_temperature)

    initial_state = generate_initial_state(problem, Random(seed))
    rng = Random(seed + 1)

    start_time = perf_counter()

    current = initial_state
    current_score = score(problem, current)
    best_state = current
    best_score = current_score

    history: list[dict] = [{
        "iteration": 0,
        "score": current_score,
        "best_score": best_score,
        "temperature": initial_temp,
        "delta": None,
        "log_acceptance_ratio": None,
        "acceptance_probability": None,
        "accepted": None,
    }]

    stuck_cache: dict[tuple, bool] = {}
    was_stuck = False
    stuck_episodes = 0
    stuck_iterations = 0
    iterations = 0
    stop_reason = "min_temperature"

    def schedule(t: int) -> float:
        nonlocal stop_reason
        if t > max_iterations:
            stop_reason = "max_iterations"
            return 0.0
        temperature = initial_temp * (cooling ** (t - 1))
        if temperature < min_temp:
            stop_reason = "min_temperature"
            return 0.0
        return temperature

    for t in count(1):
        T = schedule(t)
        if T == 0:
            break

        iterations = t
```

```
        next_state = sample_neighbor(problem, current, rng)

        if next_state is None:
            delta_e = None
            log_acceptance_ratio = None
            acceptance_probability = None
            accepted = None
        else:
            next_score = score(problem, next_state)
            delta_e = next_score - current_score
            log_acceptance_ratio = delta_e / T

            if delta_e > 0:
                current = next_state
                current_score = next_score
                acceptance_probability = 1.0
                accepted = True
            else:
                acceptance_probability = exp(log_acceptance_ratio)
                if rng.random() < acceptance_probability:
                    current = next_state
                    current_score = next_score
                    accepted = True
                else:
                    accepted = False

            if current_score > best_score:
                best_state = current
                best_score = current_score

        state_key = tuple(sorted(current.placements.items()))
        is_stuck = stuck_cache.get(state_key)
        if is_stuck is None:
            is_stuck = _is_stuck(problem, current, current_score)
            stuck_cache[state_key] = is_stuck
        if is_stuck:
            stuck_iterations += 1
            if not was_stuck:
                stuck_episodes += 1
        was_stuck = is_stuck

        history.append({
            "iteration": t,
            "score": current_score,
            "best_score": best_score,
            "temperature": T,
            "delta": delta_e,
            "log_acceptance_ratio": log_acceptance_ratio,
            "acceptance_probability": acceptance_probability,
            "accepted": accepted,
        })

    duration_seconds = perf_counter() - start_time

    return SearchResult(
        algorithm="sa",
        initial_state=initial_state,
        final_state=current,
        final_score=current_score,
        iterations=iterations,
```

```
        duration_seconds=duration_seconds,
        seed=seed,
        stop_reason=stop_reason,
        parameters={
            "seed": seed,
            "max_iterations": max_iterations,
            "initial_temperature": initial_temp,
            "cooling_rate": cooling,
            "min_temperature": min_temp,
            "objective": active_objective,
        },
        history=history,
        metrics={
            "stuck_episodes": stuck_episodes,
            "stuck_iterations": stuck_iterations,
            "best_score": best_score,
        },
        best_state=best_state,
    )
```

Fungsi `solve` adalah inti dari algoritma Simulated Annealing yang menjalankan pencarian mulai dari pembangkitan initial state acak hingga temperatur mencapai batas henti. Di dalamnya terdapat fungsi lokal `schedule(t)` yang menurunkan temperatur secara geometris berdasarkan rumus $T_t = T_0 \cdot \alpha^{t-1}$ dan menghentikan pencarian ketika iterasi melewati `max_iterations` atau temperatur turun di bawah `min_temperature`. Pada setiap iterasi, fungsi mengambil satu kandidat tetangga acak melalui `sample_neighbor`, menghitung perubahan nilai objektif ΔE dan selalu menerima langkah jika $\Delta E > 0$ atau menerimanya secara probabilistik dengan peluang $P = e^{\Delta E/T}$ jika $\Delta E \leq 0$. Selain itu, fungsi ini melacak state terbaik yang pernah dicapai (`best_state`), menyimpan riwayat konvergensi per iterasi pada `history`, serta menghitung frekuensi stuck menggunakan `stuck_cache` sebelum mengembalikan objek `SearchResult`.

Pada eksperimen menggunakan algoritma *simulated annealing*, digunakan *threshold* atau ambang batas sebesar **0.95**. Untuk membatasi pemrosesan yang terlalu banyak, kami mengasumsikan SA dapat berhenti maksimum pada iterasi ke-1000, tetapi umumnya secara alami akan berhenti pada iterasi ke-225 akibat suhu minimum yang diterima adalah **0.001** bukan 0 seperti yang dijelaskan di kelas.

Tabel 2.5.1 Hasil Eksperimen Simulated Annealing

Run (Seed)	Score Awal	Score Akhir	Score Terbaik	Berat Akhir	Iterasi	Alasan Berhenti	Durasi Pencarian (s)	Episode Stuck	Iterasi Stuck
1 (11)	226	236	236	36/40	225	min_ temperature	29.853	2	10
2 (22)	243	243	243	34/40	225	min_ temperature	35.140	3	58
3 (33)	246	253	253	40/40	225	min_ temperature	35.386	7	143
Rata-rata	238.3	244.0	244.00		225		33.460	4	70.33

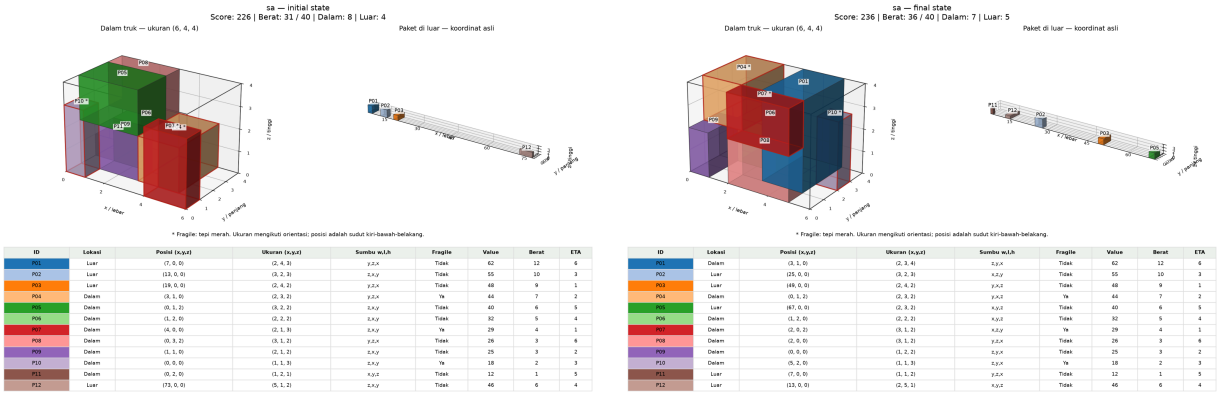
Run 1

Untuk run 1, berikut adalah hasil akhir yang didapatkan.

- Score Awal : 226 (berat awal: 31/40)
- Score Akhir : 236 (berat akhir: 36/40)
- Score Terbaik : 236
- Durasi Pencarian : 29.853 detik
- Statistik Transisi Langkah
    - Langkah perbaikan diterima : 4
    - Langkah sideways diterima : 201
    - Langkah penurunan diterima : 1
    - Langkah kandidat ditolak : 19
- Frekuensi Stuck di Local Optima :
    - Jumlah episode stuck : 2
    - Total iterasi dalam stuck : 10/225

Tabel 2.5.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Simulated Annealing

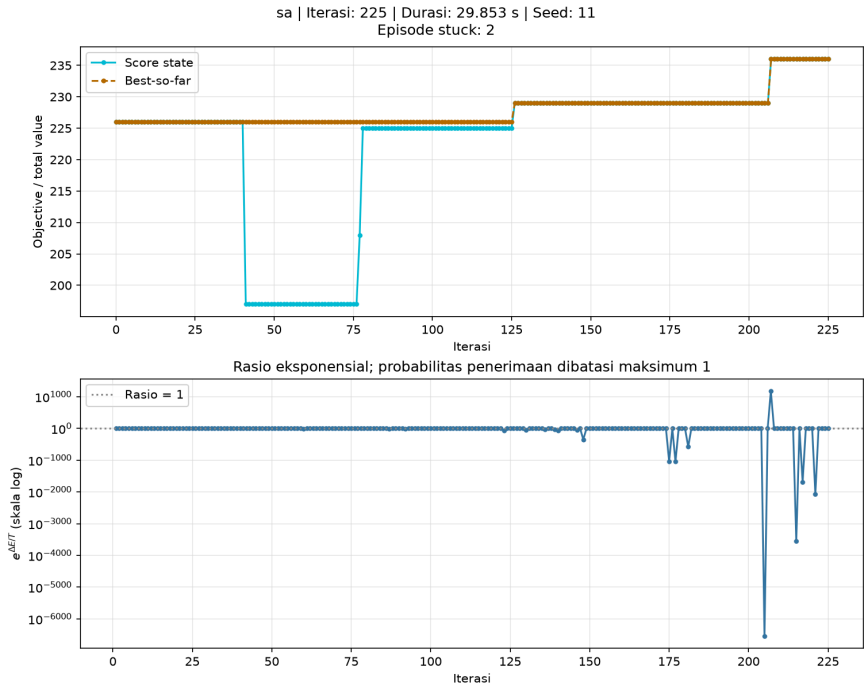
ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(2,4,3)	Dalam	(3,1,0)	(2,3,4)
P02	Luar	(13,0,0)	(3,2,3)	Luar	(25,0,0)	(3,2,3)
P03	Luar	(19,0,0)	(2,4,2)	Luar	(49,0,0)	(2,4,2)
P04	Dalam	(3,1,0)	(2,3,2)	Dalam	(0,1,2)	(2,3,2)
P05	Dalam	(0,1,2)	(3,2,2)	Luar	(67,0,0)	(2,3,2)
P06	Dalam	(1,2,0)	(2,2,2)	Dalam	(1,2,0)	(2,2,2)
P07	Dalam	(4,0,0)	(2,1,3)	Dalam	(2,0,2)	(3,1,2)
P08	Dalam	(0,3,2)	(3,1,2)	Dalam	(2,0,0)	(3,1,2)
P09	Dalam	(1,1,0)	(2,1,2)	Dalam	(0,0,0)	(1,2,2)
P10	Dalam	(0,0,0)	(1,1,3)	Dalam	(5,2,0)	(1,1,3)
P11	Dalam	(0,2,0)	(1,2,1)	Luar	(7,0,0)	(1,1,2)
P12	Luar	(73,0,0)	(5,1,2)	Luar	(13,0,0)	(2,5,1)



ID	Lokasi	Posisi (x,y,z)	Ukuran (x,y,z)	Sumbu w,l,h	Fragile	Value	Berat	ETA
P01	Luar	(7, 0, 0)	(2, 4, 3)	y,z,x	Tidak	62	12	6
P02	Luar	(13, 0, 0)	(3, 2, 3)	z,x,y	Tidak	55	10	3
P03	Luar	(19, 0, 0)	(2, 4, 2)	y,z,x	Tidak	48	9	1
P04	Dalam	(3, 1, 0)	(2, 3, 2)	y,z,x	Ya	44	7	2
P05	Dalam	(0, 1, 2)	(3, 2, 2)	z,x,y	Tidak	40	6	5
P06	Dalam	(1, 2, 0)	(2, 2, 2)	z,x,y	Tidak	32	5	4
P07	Dalam	(4, 0, 0)	(2, 1, 3)	z,x,y	Ya	29	4	1
P08	Dalam	(0, 3, 2)	(3, 1, 2)	y,z,x	Tidak	26	3	6
P09	Dalam	(1, 1, 0)	(2, 1, 2)	z,x,y	Tidak	25	3	2
P10	Dalam	(0, 0, 0)	(1, 1, 3)	z,x,y	Ya	18	2	3
P11	Dalam	(0, 2, 0)	(1, 2, 1)	x,y,z	Tidak	12	1	5
P12	Luar	(73, 0, 0)	(5, 1, 2)	z,x,y	Tidak	46	6	4

ID	Lokasi	Posisi (x,y,z)	Ukuran (x,y,z)	Sumbu w,l,h	Fragile	Value	Berat	ETA
P01	Dalam	(3, 1, 0)	(2, 3, 4)	z,y,x	Tidak	62	12	6
P02	Luar	(25, 0, 0)	(3, 2, 3)	x,z,y	Tidak	55	10	3
P03	Luar	(49, 0, 0)	(2, 4, 2)	y,x,z	Tidak	48	9	1
P04	Dalam	(0, 1, 2)	(2, 3, 2)	y,x,z	Ya	44	7	2
P05	Luar	(67, 0, 0)	(2, 3, 2)	x,y,z	Tidak	40	6	5
P06	Dalam	(1, 2, 0)	(2, 2, 2)	x,y,z	Tidak	32	5	4
P07	Dalam	(2, 0, 2)	(3, 1, 2)	x,z,y	Ya	29	4	1
P08	Dalam	(2, 0, 0)	(3, 1, 2)	y,z,x	Tidak	26	3	6
P09	Dalam	(0, 0, 0)	(1, 2, 2)	z,y,x	Tidak	25	3	2
P10	Dalam	(5, 2, 0)	(1, 1, 3)	z,y,x	Ya	18	2	3
P11	Luar	(7, 0, 0)	(1, 1, 2)	y,z,x	Tidak	12	1	5
P12	Luar	(13, 0, 0)	(2, 5, 1)	x,y,z	Tidak	46	6	4

Gambar 2.5.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Simulated Annealing



Gambar 2.5.2 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1 Simulated Annealing

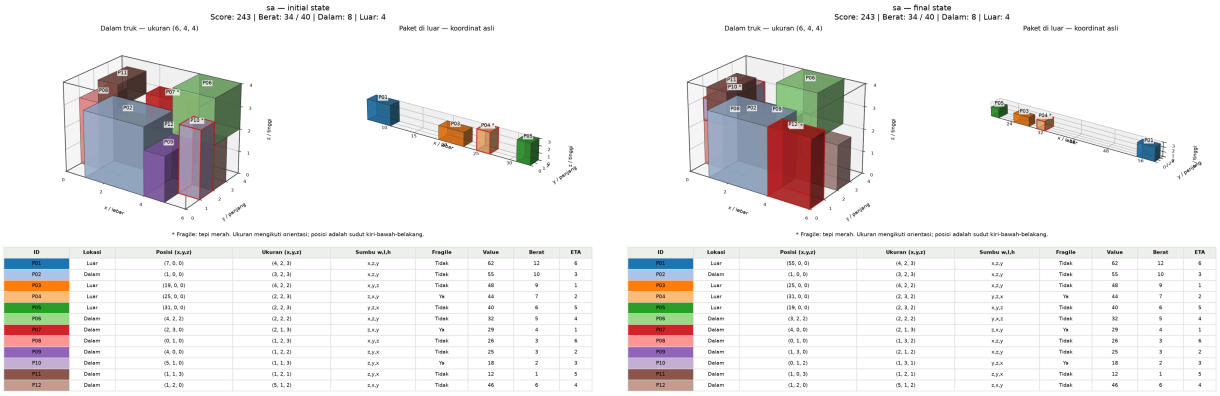
Run 2

Untuk run 2, berikut adalah hasil akhir yang didapatkan.

- Score Awal : 243 (berat awal: 34/40)
- Score Akhir : 243 (berat akhir: 34/40)
- Score Terbaik : 243
- Durasi Pencarian : 35.140 detik
- Statistik Transisi Langkah
    - Langkah perbaikan diterima : 2
    - Langkah sideways diterima : 199
    - Langkah penurunan diterima : 1
    - Langkah kandidat ditolak : 23
- Frekuensi Stuck di Local Optima
    - Jumlah episode stuck : 3
    - Total iterasi dalam stuck : 58/225

Tabel 2.5.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Simulated Annealing

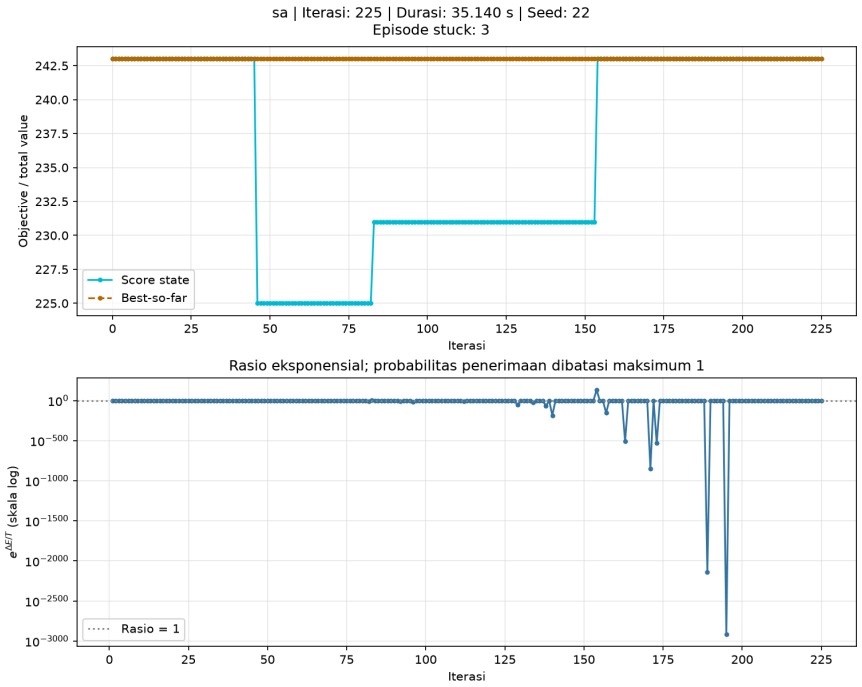
ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(4,2,3)	Luar	(55,0,0)	(4,2,3)
P02	Dalam	(1,0,0)	(3,2,3)	Dalam	(1,0,0)	(3,2,3)
P03	Luar	(19,0,0)	(4,2,2)	Luar	(25,0,0)	(4,2,2)
P04	Luar	(25,0,0)	(2,2,3)	Luar	(31,0,0)	(2,3,2)
P05	Luar	(31,0,0)	(2,2,3)	Luar	(19,0,0)	(2,3,2)
P06	Dalam	(4,2,2)	(2,2,2)	Dalam	(3,2,2)	(2,2,2)
P07	Dalam	(2,3,0)	(2,1,3)	Dalam	(4,0,0)	(2,1,3)
P08	Dalam	(0,1,0)	(1,2,3)	Dalam	(0,1,0)	(1,3,2)
P09	Dalam	(4,0,0)	(1,2,2)	Dalam	(1,3,0)	(2,1,2)
P10	Dalam	(5,1,0)	(1,1,3)	Dalam	(0,1,2)	(1,3,1)
P11	Dalam	(1,1,3)	(1,2,1)	Dalam	(1,0,3)	(1,2,1)
P12	Dalam	(1,2,0)	(5,1,2)	Dalam	(1,2,0)	(5,1,2)



ID	Lokasi	Posisi (x,y,z)	Ukuran (x,y,z)	Sumbu w,l,h	Fragile	Value	Berat	ETA
P01	Luar	(7, 0, 0)	(4, 2, 3)	x,z,y	Tidak	62	12	6
P02	Dalam	(1, 0, 0)	(3, 2, 3)	x,z,y	Tidak	55	10	3
P03	Luar	(19, 0, 0)	(4, 2, 2)	x,y,z	Tidak	48	9	1
P04	Luar	(25, 0, 0)	(2, 2, 3)	z,x,y	Ya	44	7	2
P05	Luar	(31, 0, 0)	(2, 2, 3)	y,z,x	Tidak	40	6	5
P06	Dalam	(4, 2, 2)	(2, 2, 2)	x,z,y	Tidak	32	5	4
P07	Dalam	(2, 3, 0)	(2, 1, 3)	z,x,y	Ya	29	4	1
P08	Dalam	(0, 1, 0)	(1, 2, 3)	x,y,z	Tidak	26	3	6
P09	Dalam	(4, 0, 0)	(1, 2, 2)	z,y,x	Tidak	25	3	2
P10	Dalam	(5, 1, 0)	(1, 1, 3)	z,y,x	Ya	18	2	3
P11	Dalam	(1, 1, 3)	(1, 2, 1)	z,y,x	Tidak	12	1	5
P12	Dalam	(1, 2, 0)	(5, 1, 2)	z,x,y	Tidak	46	6	4

ID	Lokasi	Posisi (x,y,z)	Ukuran (x,y,z)	Sumbu w,l,h	Fragile	Value	Berat	ETA
P01	Luar	(55, 0, 0)	(4, 2, 3)	x,z,y	Tidak	62	12	6
P02	Dalam	(1, 0, 0)	(3, 2, 3)	x,z,y	Tidak	55	10	3
P03	Luar	(25, 0, 0)	(4, 2, 2)	x,z,y	Tidak	48	9	1
P04	Luar	(31, 0, 0)	(2, 3, 2)	y,z,x	Ya	44	7	2
P05	Luar	(19, 0, 0)	(2, 3, 2)	x,y,z	Tidak	40	6	5
P06	Dalam	(3, 2, 2)	(2, 2, 2)	y,x,z	Tidak	32	5	4
P07	Dalam	(4, 0, 0)	(2, 1, 3)	z,x,y	Ya	29	4	1
P08	Dalam	(0, 1, 0)	(1, 3, 2)	x,z,y	Tidak	26	3	6
P09	Dalam	(1, 3, 0)	(2, 1, 2)	x,z,y	Tidak	25	3	2
P10	Dalam	(0, 1, 2)	(1, 3, 1)	y,z,x	Ya	18	2	3
P11	Dalam	(1, 0, 3)	(1, 2, 1)	z,y,x	Tidak	12	1	5
P12	Dalam	(1, 2, 0)	(5, 1, 2)	z,x,y	Tidak	46	6	4

Gambar 2.5.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Simulated Annealing



Gambar 2.5.4 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2 Simulated Annealing

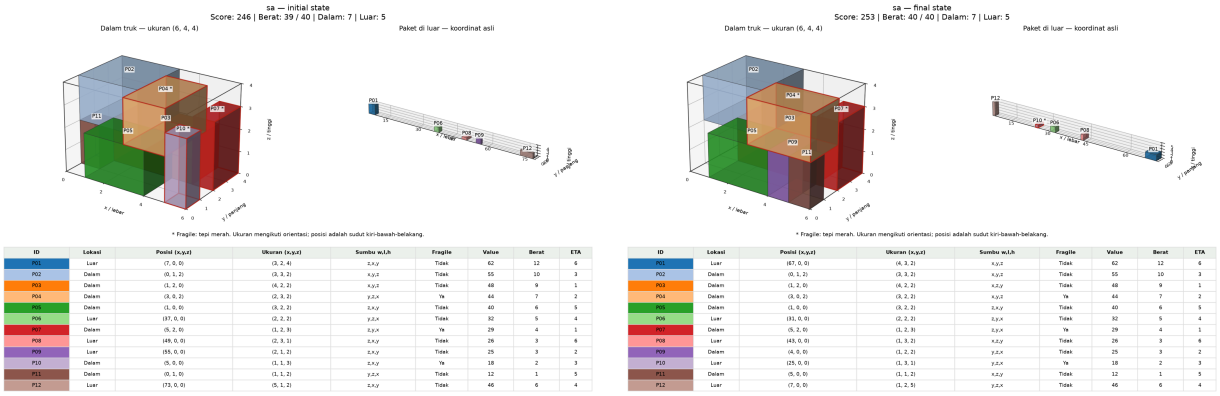
Run 3

Untuk run 3, berikut adalah hasil akhir yang didapatkan.

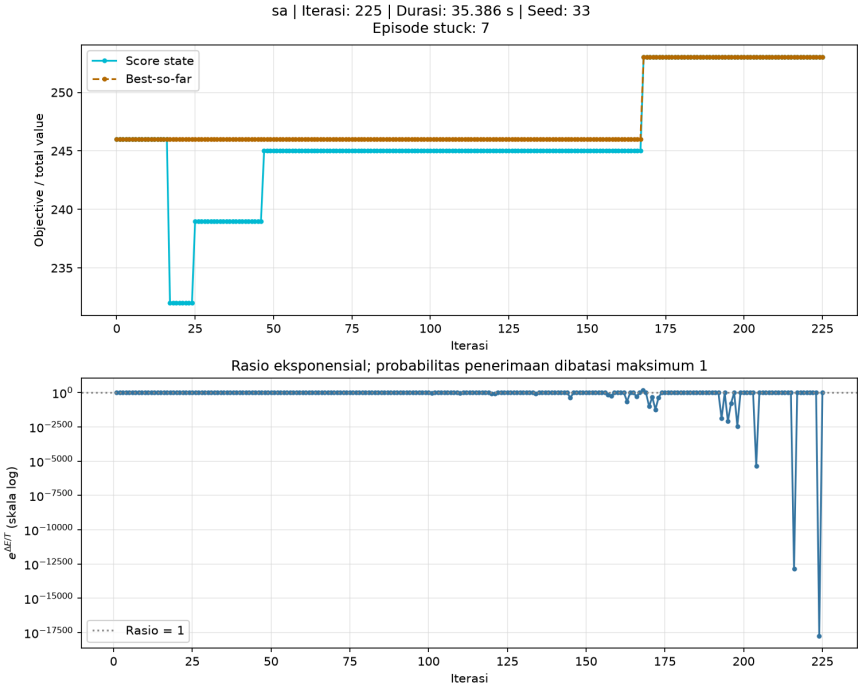
- Score Awal : 246 (berat awal: 39/40)
- Score Akhir : 253 (berat akhir: 40/40)
- Score Terbaik : 253
- Durasi Pencarian : 35.386 detik
- Statistik Transisi Langkah
    - Langkah perbaikan diterima : 3
    - Langkah sideways diterima : 195
    - Langkah penurunan diterima : 1
    - Langkah kandidat ditolak : 26
- Frekuensi Stuck di Local Optima
    - Jumlah episode stuck : 7
    - Total iterasi dalam stuck : 143/225

Tabel 2.5.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Simulated Annealing

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(3,2,4)	Luar	(67,0,0)	(4,3,2)
P02	Dalam	(0,1,2)	(3,3,2)	Dalam	(0,1,2)	(3,3,2)
P03	Dalam	(1,2,0)	(4,2,2)	Dalam	(1,2,0)	(4,2,2)
P04	Dalam	(3,0,2)	(2,3,2)	Dalam	(3,0,2)	(3,2,2)
P05	Dalam	(1,0,0)	(3,2,2)	Dalam	(1,0,0)	(3,2,2)
P06	Luar	(37,0,0)	(2,2,2)	Luar	(31,0,0)	(2,2,2)
P07	Dalam	(5,2,0)	(1,2,3)	Dalam	(5,2,0)	(1,2,3)
P08	Luar	(49,0,0)	(2,3,1)	Luar	(43,0,0)	(1,3,2)
P09	Luar	(55,0,0)	(2,1,2)	Dalam	(4,0,0)	(1,2,2)
P10	Dalam	(5,0,0)	(1,1,3)	Luar	(25,0,0)	(1,3,1)
P11	Dalam	(0,1,0)	(1,2,1)	Dalam	(5,0,0)	(1,1,2)
P12	Luar	(73,0,0)	(5,1,2)	Luar	(7,0,0)	(1,2,5)



Gambar 2.5.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Simulated Annealing



Gambar 2.5.6 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3 Simulated Annealing

Genetic Algorithm

Deskripsi Fungsi/Kelas

```
# pilih parent pake roullete wheel, peluang sebanding dengan fitness
def random_selection(population: list[State], fitness: list[int | float], rng: Random) ->
State:
    largest = max(fitness)
    if largest == 0:
        return rng.choice(population)
    return rng.choices(population, weights=[value / largest for value in fitness], k=1)[0]
```

Fungsi `random_selection` digunakan untuk memilih satu individu dari populasi sebagai *parent* menggunakan metode *roulette wheel selection*. Peluang setiap individu untuk terpilih sebanding dengan nilai fitness nya, sehingga individu dengan fitness lebih tinggi memiliki peluang lebih besar untuk menghasilkan keturunan. Nilai fitness dibagi dengan fitness terbesar sebelum digunakan sebagai bobot pemilihan, tanpa mengubah perbandingan peluang antarindividu. Jika seluruh nilai fitness bernilai nol, pemilihan dilakukan secara acak dengan peluang yang sama. Proses pemilihan menggunakan objek *rng* agar hasil pengacakan dapat dibuat ulang berdasarkan *seed*.

```
# fix child yang ngelanggar constraint setelah hasil crossover
def repair(problem: Problem, child: State) -> State:
    if not validate_state(problem, child):
        return child
    # Mulai dari slot luar unik; coba placement warisan dari bawah ke atas.
    state = State({
        package.id: Placement(position, child.placements[package.id].orientation)
        for package, position in zip(problem.packages, outside_slots(problem))
    })
    for package in sorted(problem.packages, key=lambda item:
child.placements[item.id].position[2]):
        placement = child.placements[package.id]
        if is_inside(problem, package, placement):
            placements = dict(state.placements)
            placements[package.id] = placement
            candidate = State(placements)
            if not validate_state(problem, candidate):
                state = candidate
    return state
```

Fungsi `repair` bertugas memperbaiki *child* hasil *crossover* apabila susunan paketnya melanggar *constraint*. Jika *child* sudah valid, fungsi langsung mengembalikannya tanpa perubahan. Jika terdapat pelanggaran, seluruh paket terlebih dahulu ditempatkan pada slot unik di luar truk dengan mempertahankan orientasinya. Selanjutnya, fungsi mencoba mengembalikan paket ke

posisi dan truk yang diwarisi dari parent, dengan urutan berdasarkan ketinggian dari bawah ke atas agar paket penyangga dapat ditempatkan terlebih dahulu. Setiap penempatan hanya diterima apabila state yang dihasilkan memenuhi seluruh constraint. Paket yang tidak dapat dikembalikan secara valid tetap berada di luar truk.

```
# crossover 1 titik pada urutan paket
def reproduce(problem: Problem, first: State, second: State, rng: Random) -> State:
    if len(problem.packages) < 2:
        return State(first.placements)
    cut = rng.randrange(1, len(problem.packages))
    child = State({
        package.id: (first if index < cut else second).placements[package.id]
        for index, package in enumerate(problem.packages)
    })
    return repair(problem, child)
```

Fungsi `reproduce` menghasilkan satu *child* dari dua *parent* melalui metode *one-point crossover*. Fungsi memilih satu titik potong secara acak pada urutan paket. Paket sebelum titik potong mewarisi placement dari parent pertama, sedangkan paket mulai dari titik potong hingga akhir mewarisi placement dari parent kedua. Placement yang diwariskan mencakup posisi, orientasi, dan indeks truk. *Child* kemudian diperiksa dan diperbaiki menggunakan fungsi `repair` karena penggabungan dua susunan yang valid dapat menghasilkan pelanggaran *constraint*. Jika jumlah paket kurang dari dua, fungsi mengembalikan salinan state parent pertama karena tidak terdapat titik potong yang dapat dipilih.

```
# ngambil neighbor valid
def mutate(problem: Problem, child: State, rng: Random) -> State:
    candidate = sample_neighbor(problem, child, rng)
    return child if candidate is None else candidate
```

Fungsi `mutate` digunakan untuk memberikan perubahan acak pada child dengan mengambil satu *state* tetangga yang valid melalui `sample_neighbor`. Mutasi membantu menjaga keragaman populasi dan membuka kemungkinan terbentuknya susunan paket yang berbeda dari hasil *crossover*. Jika kandidat tetangga berhasil diperoleh, fungsi mengembalikan kandidat tersebut tanpa mensyaratkan peningkatan nilai fitness. Jika tidak ada kandidat yang diperoleh, *child* dikembalikan tanpa perubahan. Peluang terjadinya mutasi diatur pada fungsi `solve` melalui konstanta `MUTATION_PROBABILITY` sebesar 0.05 atau 5% untuk setiap *child*.

```
def solve(
    problem: Problem, *, seed: int = 11, population_size: int = 20,
    generations: int = 100, objective: str | None = None,
) -> SearchResult:
    if type(seed) is not int:
        raise ValueError("Seed harus berupa integer.")
    for name, value in (("population_size", population_size), ("generations", generations)):
        if type(value) is not int or value <= 0:
            raise ValueError(f"{name} harus berupa integer positif.")
    active_objective = set_objective(objective) if objective is not None else
get_objective()

    initial_rng = Random(seed)
    population = [generate_initial_state(problem, initial_rng) for _ in
range(population_size)]
    initial_population = list(population)
    rng = Random(seed + 1)
    started = perf_counter()
    fitness = [score(problem, individual) for individual in population]
    initial_state = population[max(range(population_size), key=fitness.__getitem__)]
    history = [{"iteration": 0, "max_score": max(fitness), "mean_score": sum(fitness) /
population_size}]

    for generation in range(1, generations + 1):
        new_population = []
        for _ in range(population_size):
            first = random_selection(population, fitness, rng)
            second = random_selection(population, fitness, rng)
            child = reproduce(problem, first, second, rng)
            if rng.random() < MUTATION_PROBABILITY:
                child = mutate(problem, child, rng)
            new_population.append(child)
        population = new_population
        fitness = [score(problem, individual) for individual in population]
        history.append({
            "iteration": generation,
            "max_score": max(fitness),
            "mean_score": sum(fitness) / population_size,
        })

    final_state = population[max(range(population_size), key=fitness.__getitem__)]
    return SearchResult(
        algorithm="ga", initial_state=initial_state, final_state=final_state,
        final_score=max(fitness), iterations=generations,
        duration_seconds=perf_counter() - started, seed=seed,
        stop_reason="generation_limit",
        parameters={
            "population_size": population_size, "generations": generations,
            "mutation_probability": MUTATION_PROBABILITY,
            "selection": "roulette_wheel", "crossover": "one_point", "elitism": False,
            "objective": active_objective,
        },
        history=history, initial_population=initial_population,
        final_population=list(population),
    )
```

Fungsi `solve` merupakan fungsi utama yang menjalankan Genetic Algorithm, mulai dari validasi parameter, penentuan objective function, hingga pembentukan populasi awal secara acak

berdasarkan *seed*. Nilai *objective* setiap individu digunakan sebagai fitness, dan individu terbaik pada populasi awal dicatat sebagai `initial_state`. Pada setiap generasi, fungsi membentuk populasi baru dengan memilih dua *parent* melalui `random_selection`, menghasilkan satu *child* melalui `reproduce`, lalu menjalankan `mutate` dengan peluang 5%. Proses tersebut diulang sampai jumlah *child* sama dengan ukuran populasi, kemudian seluruh populasi lama digantikan oleh populasi baru. Nilai fitness maksimum dan rata-rata dicatat pada *history* sejak generasi awal hingga generasi terakhir. Setelah batas generasi tercapai, fungsi mengembalikan `SearchResult` yang memuat individu terbaik dari populasi terakhir, skor akhir, jumlah generasi, durasi pencarian, parameter, riwayat fitness, serta populasi awal dan akhir. Pengukuran durasi dimulai setelah populasi awal selesai dibentuk.

Pada eksperimen menggunakan algoritma *genetic algorithm*, digunakan 2 parameter pengujian, yaitu jumlah populasi dan banyak iterasi. Pengujian pertama digunakan parameter populasi sebanyak 20, dengan variasi jumlah iterasi, yaitu 50, 100, dan 200. Setiap konfigurasi dijalankan sebanyak 3 kali menggunakan *seed* 101, 102, dan 103. Pengujian kedua digunakan parameter iterasi sebanyak 100, dengan variasi jumlah populasi, yaitu sebanyak 10, 20, dan 40 individu. Masing-masing digunakan *seed* 201, 202, dan 203. Seluruh pengujian menggunakan *objective function* default dan satu truk.

Algoritma berhenti ketika jumlah generasi yang ditentukan telah tercapai, dengan alasan berhenti `generation_limit`. Skor awal merupakan nilai *objective* tertinggi pada populasi awal, sedangkan skor akhir merupakan nilai *objective* tertinggi pada populasi terakhir. Individu terbaik dari suatu generasi tidak selalu dipertahankan pada generasi berikutnya. Durasi pencarian dihitung setelah pembentukan populasi awal dan tidak mencakup penyimpanan hasil maupun pembuatan visualisasi.

Tabel 2.6.1.1 Hasil Eksperimen Genetic Algorithm Variasi Iterasi, Populasi 20

Run (Seed)	Populasi	Iterasi	Score Awal	Score Akhir	Durasi Pencarian (s)
101	20	50	272	216	0.100
102	20	50	283	216	0.092
103	20	50	283	179	0.096
Rata-rata	20	50	279.33	203.67	0.096
101	20	100	272	223	0.185
102	20	100	283	216	0.174
103	20	100	283	179	0.183
Rata-rata	20	100	279.33	206.00	0.181
101	20	200	272	223	0.362
102	20	200	283	216	0.334
103	20	200	283	179	0.345
Rata-rata	20	200	279.33	206.00	0.347

Tabel 2.6.1.2 Hasil Eksperimen Genetic Algorithm Variasi Populasi, Iterasi 100

Run (Seed)	Populasi	Iterasi	Score Awal	Score Akhir	Durasi Pencarian (s)
201	10	100	251	102	0.079
202	10	100	276	110	0.081
203	10	100	276	146	0.085
Rata-rata	10	100	267.67	119.33	0.081
201	20	100	283	225	0.190
202	20	100	276	139	0.174
203	20	100	276	171	0.191
Rata-rata	20	100	278.33	178.33	0.185
201	40	100	283	226	0.388
202	40	100	276	198	0.370
203	40	100	276	211	0.405
Rata-rata	40	100	278.33	211.67	0.388

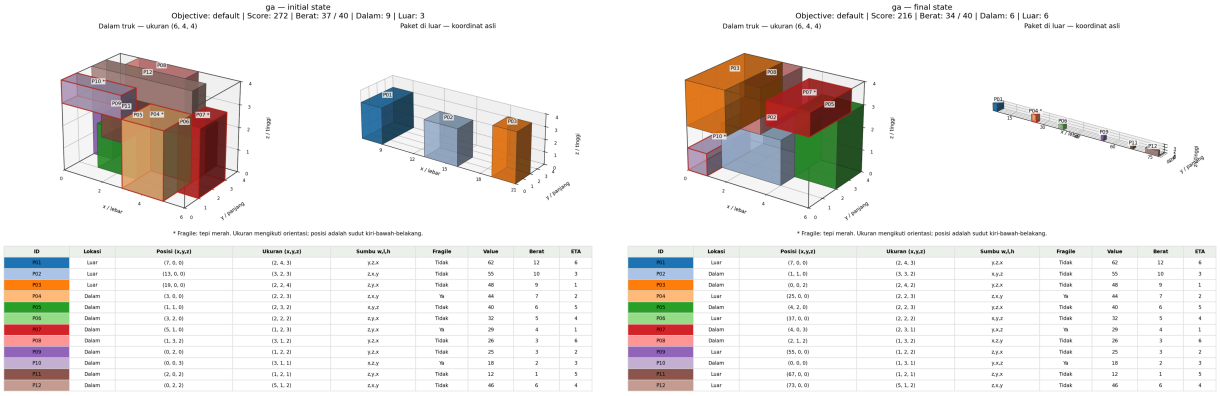
Run 1 - Iterasi 50, Populasi 20

Untuk run 1, berikut adalah hasil akhir yang didapatkan.

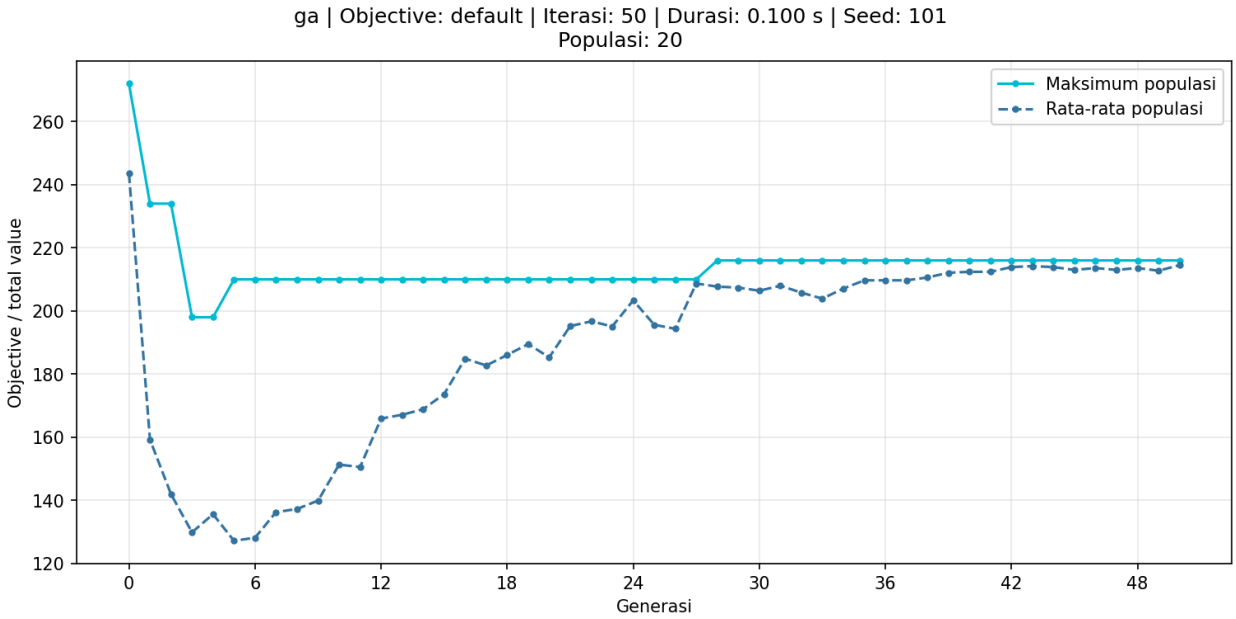
- Jumlah Populasi : 20
- Banyak Iterasi : 50
- Seed : 101
- Skor Awal : 272 (berat awal: 37/40)
- Skor Akhir : 216 (berat akhir: 34/40)
- Rata-rata Skor Populasi Akhir : 214.50
- Durasi Pencarian : 0.100 detik

Tabel 2.6.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i50-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 4, 3)	Luar	(7, 0, 0)	(2, 4, 3)
P02	Luar	(13, 0, 0)	(3, 2, 3)	Dalam	(1, 1, 0)	(3, 3, 2)
P03	Luar	(19, 0, 0)	(2, 2, 4)	Dalam	(0, 0, 2)	(2, 4, 2)
P04	Dalam	(3, 0, 0)	(2, 2, 3)	Luar	(25, 0, 0)	(2, 2, 3)
P05	Dalam	(1, 1, 0)	(2, 3, 2)	Dalam	(4, 2, 0)	(2, 2, 3)
P06	Dalam	(3, 2, 0)	(2, 2, 2)	Luar	(37, 0, 0)	(2, 2, 2)
P07	Dalam	(5, 1, 0)	(1, 2, 3)	Dalam	(4, 0, 3)	(2, 3, 1)
P08	Dalam	(1, 3, 2)	(3, 1, 2)	Dalam	(2, 1, 2)	(1, 3, 2)
P09	Dalam	(0, 2, 0)	(1, 2, 2)	Luar	(55, 0, 0)	(1, 2, 2)
P10	Dalam	(0, 0, 3)	(3, 1, 1)	Dalam	(0, 0, 0)	(1, 3, 1)
P11	Dalam	(2, 0, 2)	(1, 2, 1)	Luar	(67, 0, 0)	(1, 2, 1)
P12	Dalam	(0, 2, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(5, 1, 2)



Gambar 2.6.1 Visualisasi Konfigurasi Awal dan Akhir Run 1-i50-p20 Genetic Algorithm



Gambar 2.6.2 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i50-p20 Genetic Algorithm

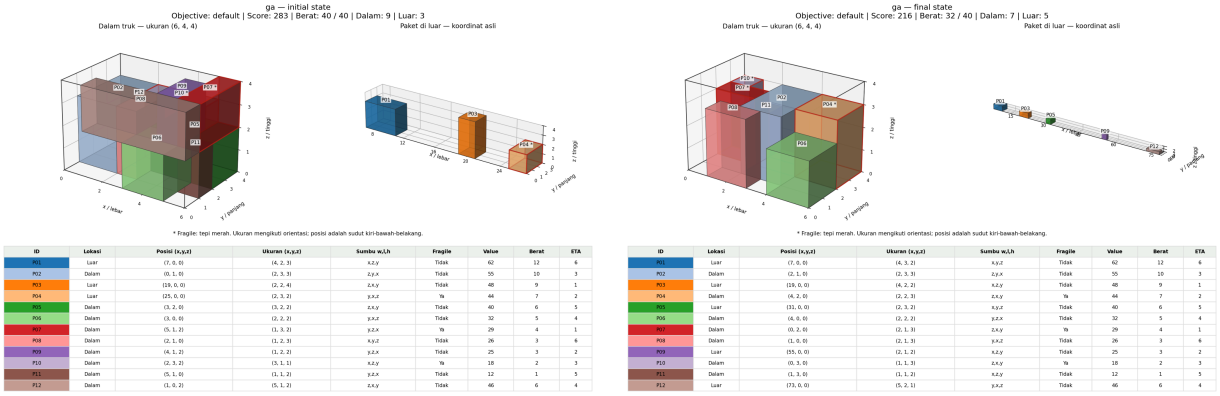
Run 2 - Iterasi 50, Populasi 20

Untuk run 2, berikut adalah hasil akhir yang didapatkan.

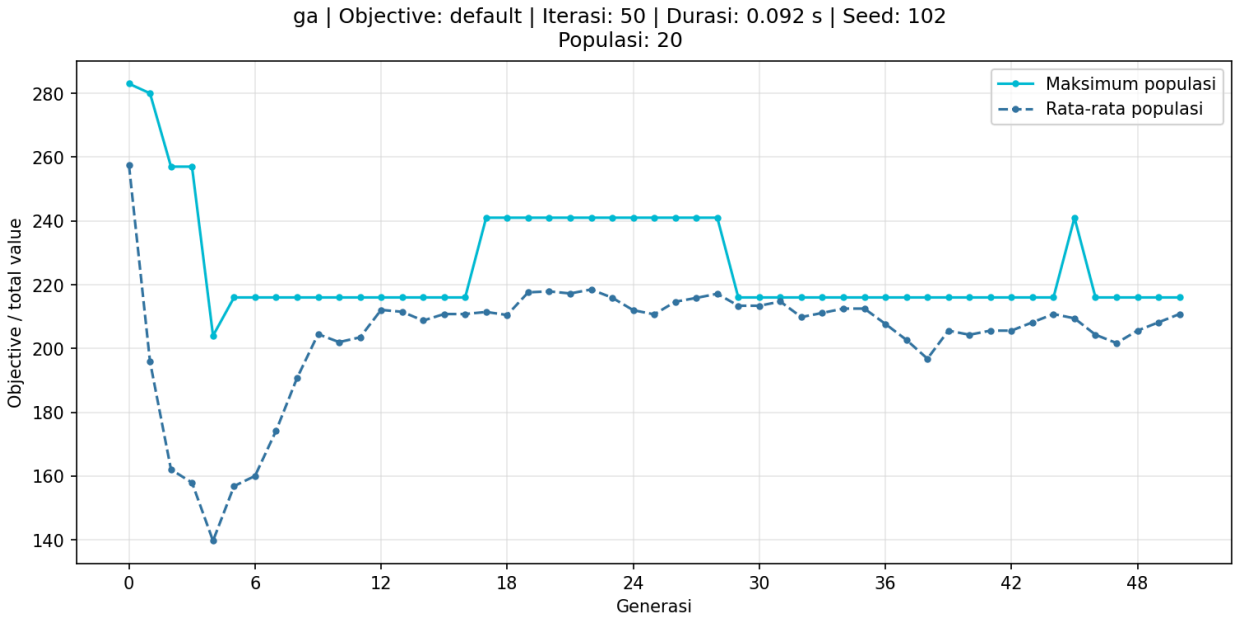
- Jumlah Populasi : 20
- Banyak Iterasi : 50
- Seed : 102
- Skor Awal : 283 (berat awal: 40/40)
- Skor Akhir : 216 (berat akhir: 32/40)
- Rata-rata Skor Populasi Akhir : 210.80
- Durasi Pencarian : 0.092 detik

Tabel 2.6.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i50-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(4, 2, 3)	Luar	(7, 0, 0)	(4, 3, 2)
P02	Dalam	(0, 1, 0)	(2, 3, 3)	Dalam	(2, 1, 0)	(2, 3, 3)
P03	Luar	(19, 0, 0)	(2, 2, 4)	Luar	(19, 0, 0)	(4, 2, 2)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Dalam	(4, 2, 0)	(2, 2, 3)
P05	Dalam	(3, 2, 0)	(3, 2, 2)	Luar	(31, 0, 0)	(2, 3, 2)
P06	Dalam	(3, 0, 0)	(2, 2, 2)	Dalam	(4, 0, 0)	(2, 2, 2)
P07	Dalam	(5, 1, 2)	(1, 3, 2)	Dalam	(0, 2, 0)	(2, 1, 3)
P08	Dalam	(2, 1, 0)	(1, 2, 3)	Dalam	(1, 0, 0)	(2, 1, 3)
P09	Dalam	(4, 1, 2)	(1, 2, 2)	Luar	(55, 0, 0)	(2, 1, 2)
P10	Dalam	(2, 3, 2)	(3, 1, 1)	Dalam	(0, 3, 0)	(1, 1, 3)
P11	Dalam	(5, 1, 0)	(1, 1, 2)	Dalam	(1, 3, 0)	(1, 1, 2)
P12	Dalam	(1, 0, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(5, 2, 1)



Gambar 2.6.3 Visualisasi Konfigurasi Awal dan Akhir Run 2-i50-p20 Genetic Algorithm



Gambar 2.6.4 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i50-p20 Genetic Algorithm

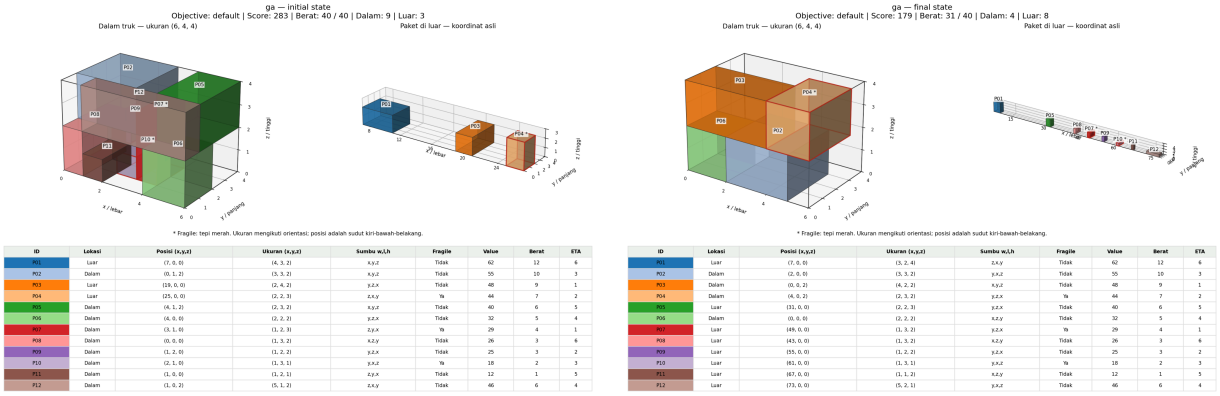
Run 3 - Iterasi 50, Populasi 20

Untuk run 3, berikut adalah hasil akhir yang didapatkan.

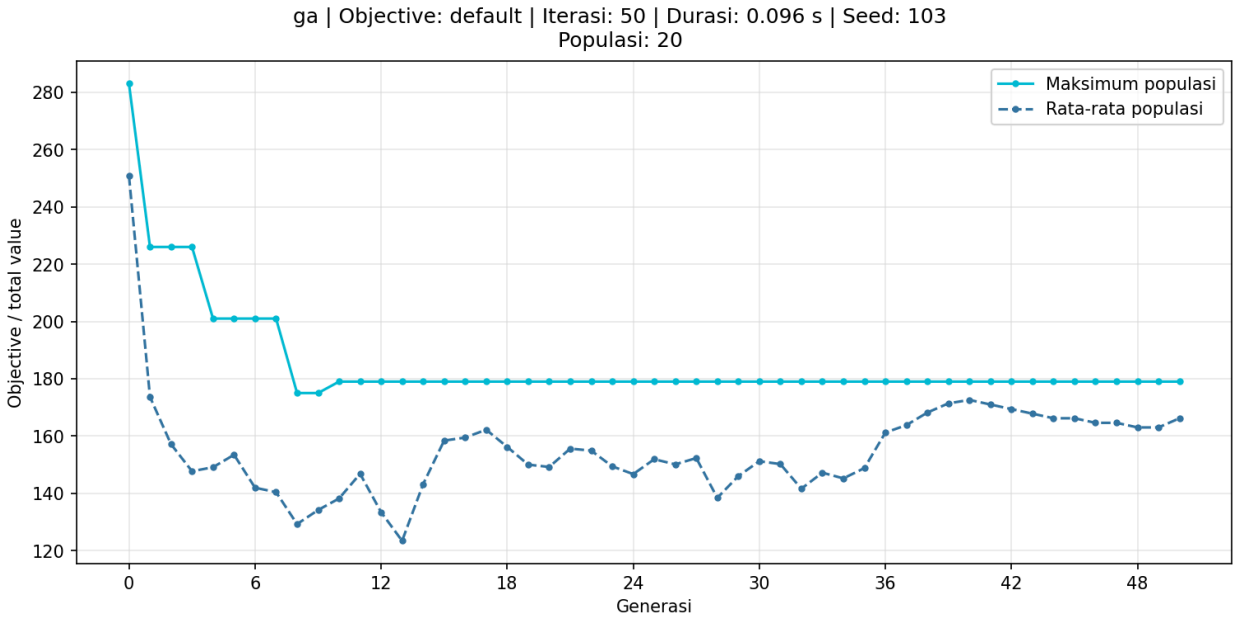
- Jumlah Populasi : 20
- Banyak Iterasi : 50
- Seed : 103
- Skor Awal : 283 (berat awal: 40/40)
- Skor Akhir : 179 (berat akhir: 31/40)
- Rata-rata Skor Populasi Akhir : 166.20
- Durasi Pencarian : 0.096 detik

Tabel 2.6.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i50-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(4, 3, 2)	Luar	(7, 0, 0)	(3, 2, 4)
P02	Dalam	(0, 1, 2)	(3, 3, 2)	Dalam	(2, 0, 0)	(3, 3, 2)
P03	Luar	(19, 0, 0)	(2, 4, 2)	Dalam	(0, 0, 2)	(4, 2, 2)
P04	Luar	(25, 0, 0)	(2, 2, 3)	Dalam	(4, 0, 2)	(2, 3, 2)
P05	Dalam	(4, 1, 2)	(2, 3, 2)	Luar	(31, 0, 0)	(2, 2, 3)
P06	Dalam	(4, 0, 0)	(2, 2, 2)	Dalam	(0, 0, 0)	(2, 2, 2)
P07	Dalam	(3, 1, 0)	(1, 2, 3)	Luar	(49, 0, 0)	(1, 3, 2)
P08	Dalam	(0, 0, 0)	(1, 3, 2)	Luar	(43, 0, 0)	(1, 3, 2)
P09	Dalam	(1, 2, 0)	(1, 2, 2)	Luar	(55, 0, 0)	(1, 2, 2)
P10	Dalam	(2, 1, 0)	(1, 3, 1)	Luar	(61, 0, 0)	(1, 3, 1)
P11	Dalam	(1, 0, 0)	(1, 2, 1)	Luar	(67, 0, 0)	(1, 1, 2)
P12	Dalam	(1, 0, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(5, 2, 1)



Gambar 2.6.5 Visualisasi Konfigurasi Awal dan Akhir Run 3-i50-p20 Genetic Algorithm



Gambar 2.6.6 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i50-p20 Genetic Algorithm

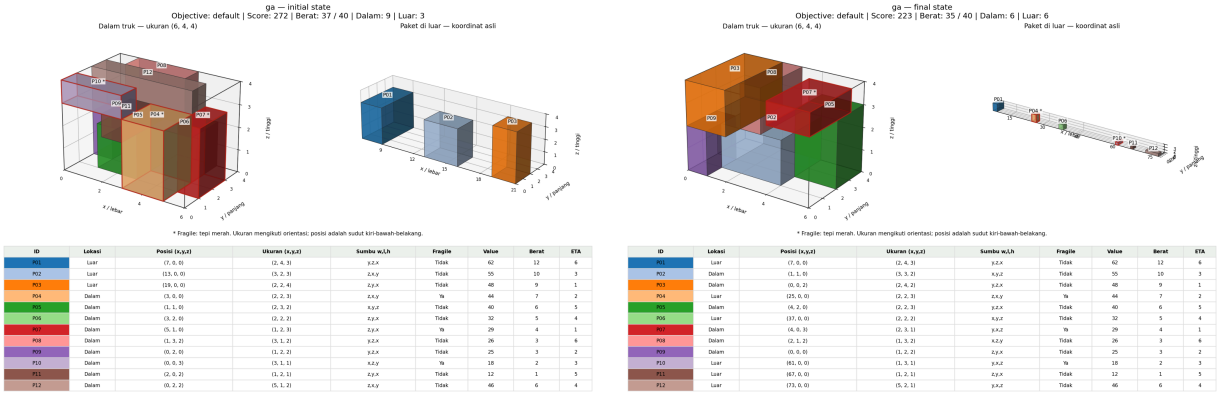
Run 1 - Iterasi 100, Populasi 20

Untuk run 1, berikut adalah hasil akhir yang didapatkan.

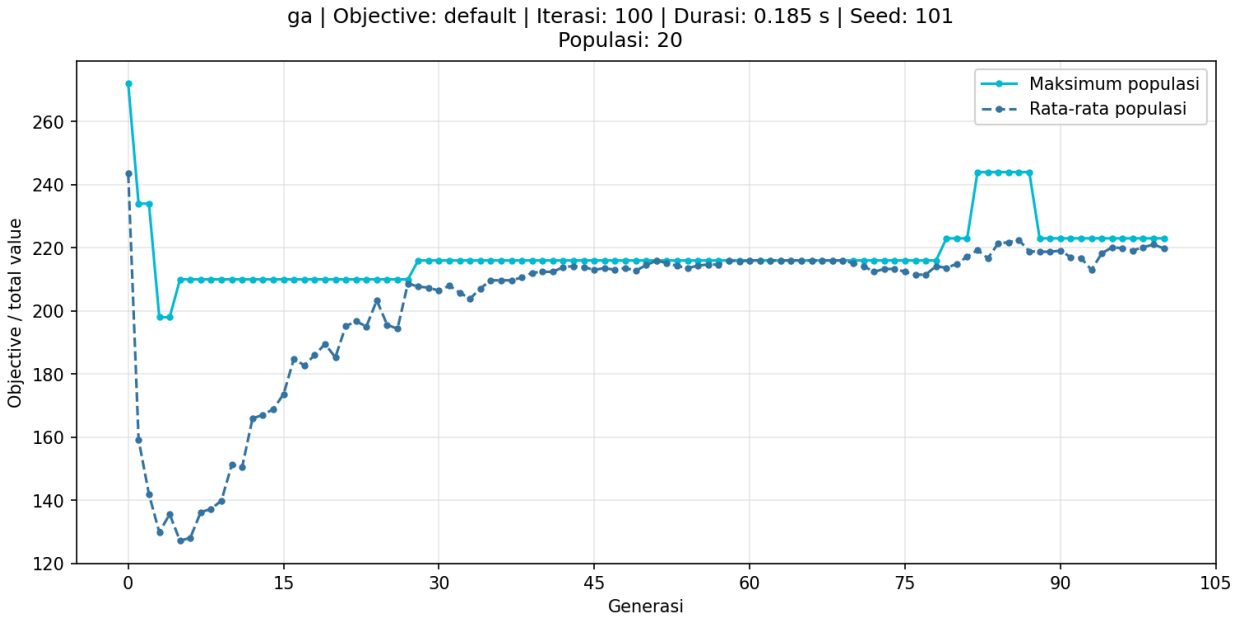
- Jumlah Populasi : 20
- Banyak Iterasi : 100
- Seed : 101
- Skor Awal : 272 (berat awal: 37/40)
- Skor Akhir : 223 (berat akhir: 35/40)
- Rata-rata Skor Populasi Akhir : 219.80
- Durasi Pencarian : 0.185 detik

Tabel 2.6.5 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i100-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 4, 3)	Luar	(7, 0, 0)	(2, 4, 3)
P02	Luar	(13, 0, 0)	(3, 2, 3)	Dalam	(1, 1, 0)	(3, 3, 2)
P03	Luar	(19, 0, 0)	(2, 2, 4)	Dalam	(0, 0, 2)	(2, 4, 2)
P04	Dalam	(3, 0, 0)	(2, 2, 3)	Luar	(25, 0, 0)	(2, 2, 3)
P05	Dalam	(1, 1, 0)	(2, 3, 2)	Dalam	(4, 2, 0)	(2, 2, 3)
P06	Dalam	(3, 2, 0)	(2, 2, 2)	Luar	(37, 0, 0)	(2, 2, 2)
P07	Dalam	(5, 1, 0)	(1, 2, 3)	Dalam	(4, 0, 3)	(2, 3, 1)
P08	Dalam	(1, 3, 2)	(3, 1, 2)	Dalam	(2, 1, 2)	(1, 3, 2)
P09	Dalam	(0, 2, 0)	(1, 2, 2)	Dalam	(0, 0, 0)	(1, 2, 2)
P10	Dalam	(0, 0, 3)	(3, 1, 1)	Luar	(61, 0, 0)	(1, 3, 1)
P11	Dalam	(2, 0, 2)	(1, 2, 1)	Luar	(67, 0, 0)	(1, 2, 1)
P12	Dalam	(0, 2, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(5, 2, 1)



Gambar 2.6.7 Visualisasi Konfigurasi Awal dan Akhir Run 1-i100-p20 Genetic Algorithm



Gambar 2.6.8 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i100-p20 Genetic Algorithm

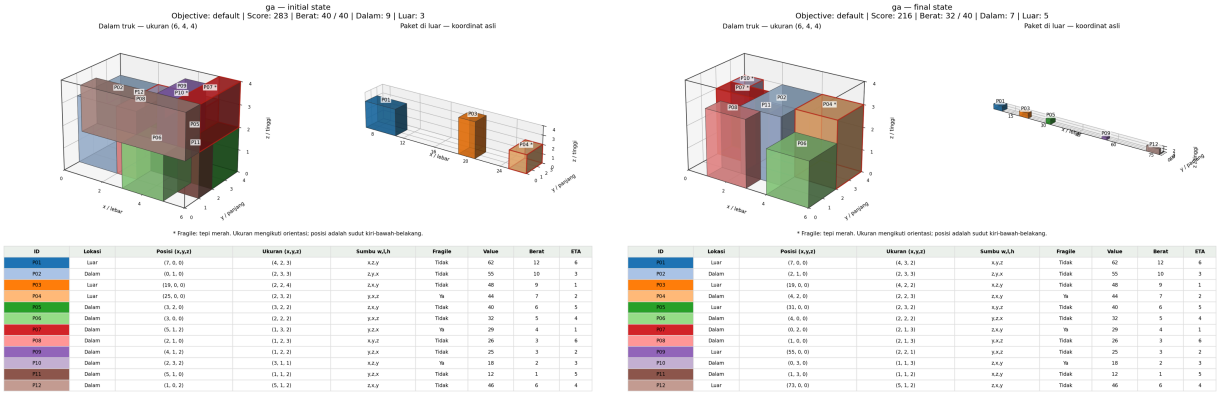
Run 2 - Iterasi 100, Populasi 20

Untuk run 2, berikut adalah hasil akhir yang didapatkan.

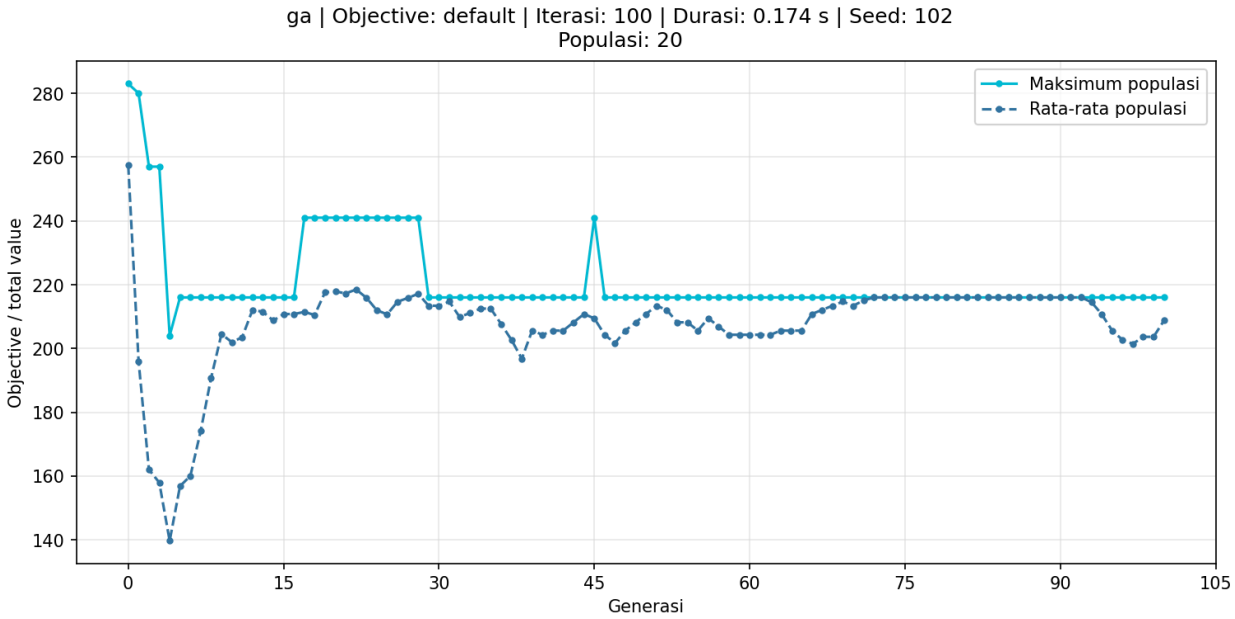
- Jumlah Populasi : 20
- Banyak Iterasi : 100
- Seed : 102
- Skor Awal : 283 (berat awal: 40/40)
- Skor Akhir : 216 (berat akhir: 32/40)
- Rata-rata Skor Populasi Akhir : 208.75
- Durasi Pencarian : 0.174 detik

Tabel 2.6.6 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i100-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(4, 2, 3)	Luar	(7, 0, 0)	(4, 3, 2)
P02	Dalam	(0, 1, 0)	(2, 3, 3)	Dalam	(2, 1, 0)	(2, 3, 3)
P03	Luar	(19, 0, 0)	(2, 2, 4)	Luar	(19, 0, 0)	(4, 2, 2)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Dalam	(4, 2, 0)	(2, 2, 3)
P05	Dalam	(3, 2, 0)	(3, 2, 2)	Luar	(31, 0, 0)	(2, 3, 2)
P06	Dalam	(3, 0, 0)	(2, 2, 2)	Dalam	(4, 0, 0)	(2, 2, 2)
P07	Dalam	(5, 1, 2)	(1, 3, 2)	Dalam	(0, 2, 0)	(2, 1, 3)
P08	Dalam	(2, 1, 0)	(1, 2, 3)	Dalam	(1, 0, 0)	(2, 1, 3)
P09	Dalam	(4, 1, 2)	(1, 2, 2)	Luar	(55, 0, 0)	(2, 2, 1)
P10	Dalam	(2, 3, 2)	(3, 1, 1)	Dalam	(0, 3, 0)	(1, 1, 3)
P11	Dalam	(5, 1, 0)	(1, 1, 2)	Dalam	(1, 3, 0)	(1, 1, 2)
P12	Dalam	(1, 0, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(5, 1, 2)



Gambar 2.6.9 Visualisasi Konfigurasi Awal dan Akhir Run 2-i100-p20 Genetic Algorithm



Gambar 2.6.10 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i100-p20 Genetic Algorithm

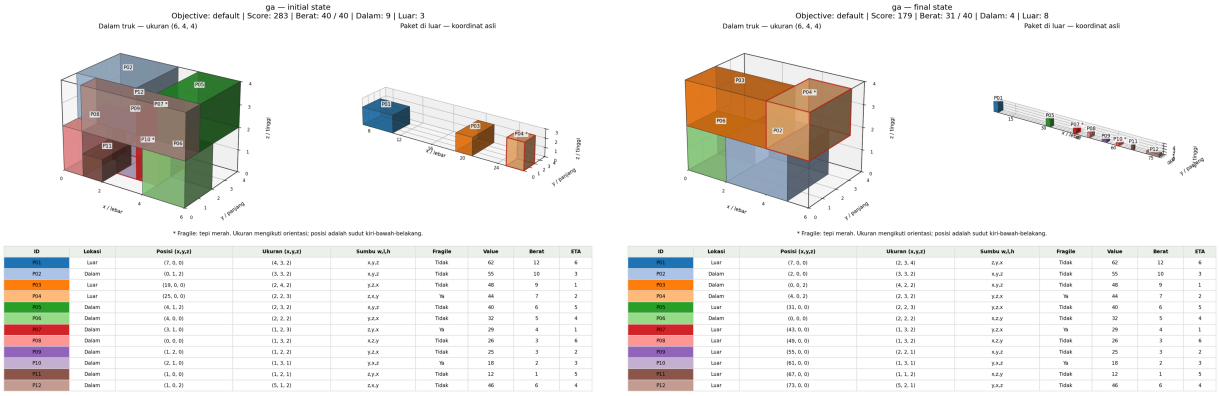
Run 3 - Iterasi 100, Populasi 20

Untuk run 3, berikut adalah hasil akhir yang didapatkan.

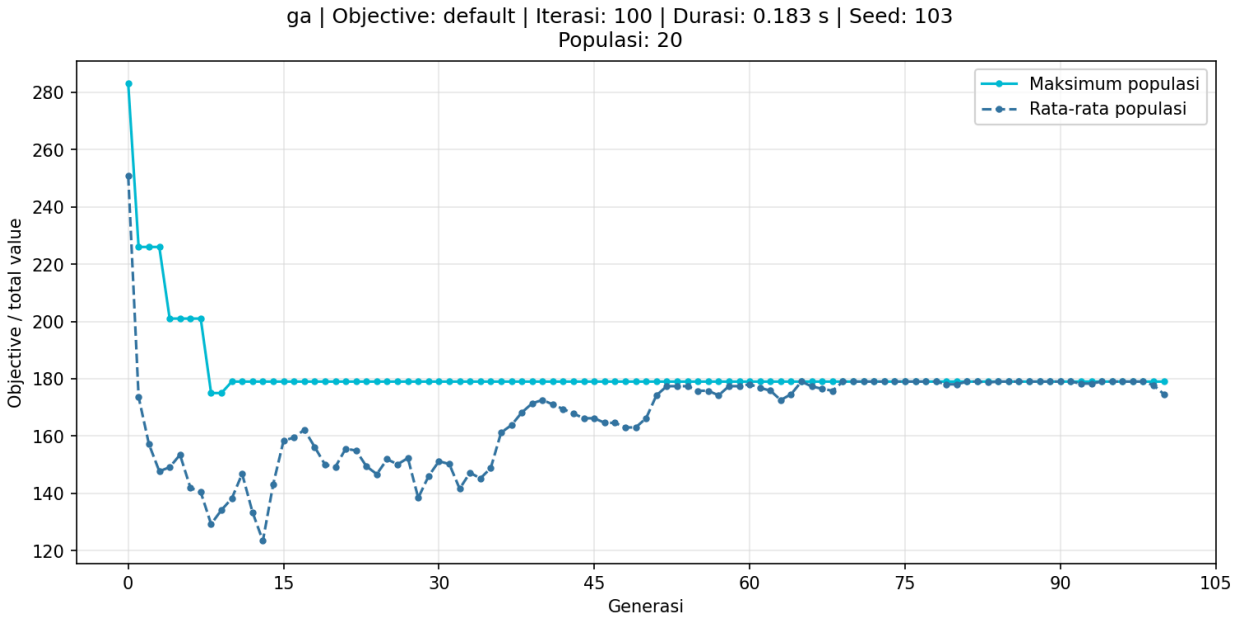
- Jumlah Populasi : 20
- Banyak Iterasi : 100
- Seed : 103
- Skor Awal : 283 (berat awal: 40/40)
- Skor Akhir : 179 (berat akhir: 31/40)
- Rata-rata Skor Populasi Akhir : 174.60
- Durasi Pencarian : 0.183 detik

Tabel 2.6.7 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i100-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(4, 3, 2)	Luar	(7, 0, 0)	(2, 3, 4)
P02	Dalam	(0, 1, 2)	(3, 3, 2)	Dalam	(2, 0, 0)	(3, 3, 2)
P03	Luar	(19, 0, 0)	(2, 4, 2)	Dalam	(0, 0, 2)	(4, 2, 2)
P04	Luar	(25, 0, 0)	(2, 2, 3)	Dalam	(4, 0, 2)	(2, 3, 2)
P05	Dalam	(4, 1, 2)	(2, 3, 2)	Luar	(31, 0, 0)	(2, 2, 3)
P06	Dalam	(4, 0, 0)	(2, 2, 2)	Dalam	(0, 0, 0)	(2, 2, 2)
P07	Dalam	(3, 1, 0)	(1, 2, 3)	Luar	(43, 0, 0)	(1, 3, 2)
P08	Dalam	(0, 0, 0)	(1, 3, 2)	Luar	(49, 0, 0)	(1, 3, 2)
P09	Dalam	(1, 2, 0)	(1, 2, 2)	Luar	(55, 0, 0)	(2, 2, 1)
P10	Dalam	(2, 1, 0)	(1, 3, 1)	Luar	(61, 0, 0)	(1, 3, 1)
P11	Dalam	(1, 0, 0)	(1, 2, 1)	Luar	(67, 0, 0)	(1, 1, 2)
P12	Dalam	(1, 0, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(5, 2, 1)



Gambar 2.6.11 Visualisasi Konfigurasi Awal dan Akhir Run 3-i100-p20 Genetic Algorithm



Gambar 2.6.12 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i100-p20 Genetic Algorithm

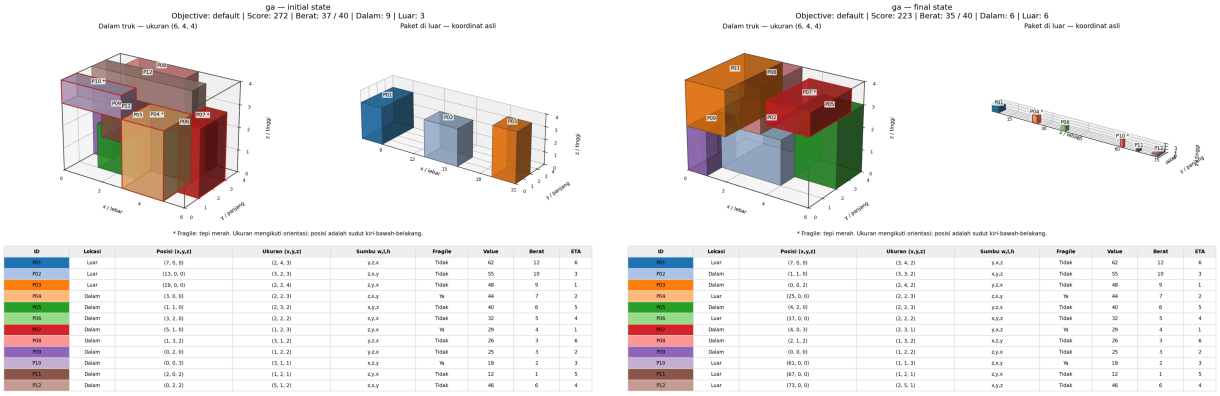
Run 1 - Iterasi 200, Populasi 20

Untuk run 1, berikut adalah hasil akhir yang didapatkan.

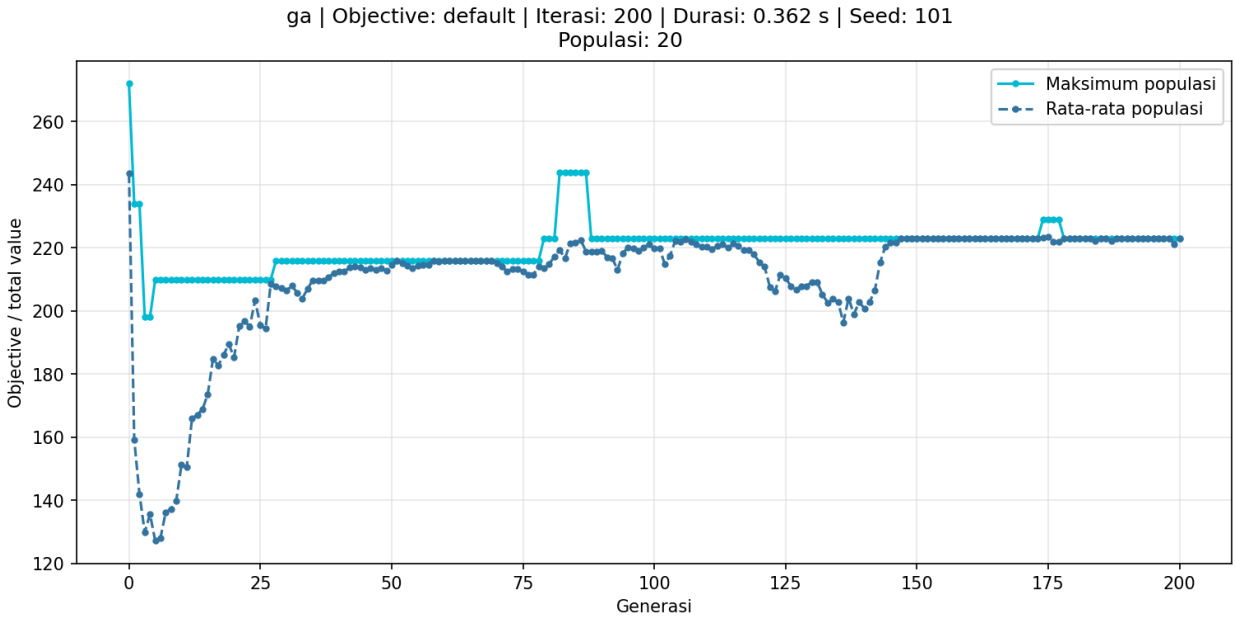
- Jumlah Populasi : 20
- Banyak Iterasi : 200
- Seed : 101
- Skor Awal : 272 (berat awal: 37/40)
- Skor Akhir : 223 (berat akhir: 25/40)
- Rata-rata Skor Populasi Akhir : 223.00
- Durasi Pencarian : 0.362 detik

Tabel 2.6.8 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i200-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 4, 3)	Luar	(7, 0, 0)	(3, 4, 2)
P02	Luar	(13, 0, 0)	(3, 2, 3)	Dalam	(1, 1, 0)	(3, 3, 2)
P03	Luar	(19, 0, 0)	(2, 2, 4)	Dalam	(0, 0, 2)	(2, 4, 2)
P04	Dalam	(3, 0, 0)	(2, 2, 3)	Luar	(25, 0, 0)	(2, 2, 3)
P05	Dalam	(1, 1, 0)	(2, 3, 2)	Dalam	(4, 2, 0)	(2, 2, 3)
P06	Dalam	(3, 2, 0)	(2, 2, 2)	Luar	(37, 0, 0)	(2, 2, 2)
P07	Dalam	(5, 1, 0)	(1, 2, 3)	Dalam	(4, 0, 3)	(2, 3, 1)
P08	Dalam	(1, 3, 2)	(3, 1, 2)	Dalam	(2, 1, 2)	(1, 3, 2)
P09	Dalam	(0, 2, 0)	(1, 2, 2)	Dalam	(0, 0, 0)	(1, 2, 2)
P10	Dalam	(0, 0, 3)	(3, 1, 1)	Luar	(61, 0, 0)	(1, 1, 3)
P11	Dalam	(2, 0, 2)	(1, 2, 1)	Luar	(67, 0, 0)	(1, 2, 1)
P12	Dalam	(0, 2, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(2, 5, 1)



Gambar 2.6.13 Visualisasi Konfigurasi Awal dan Akhir Run 1-i200-p20 Genetic Algorithm



Gambar 2.6.14 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i200-p20 Genetic Algorithm

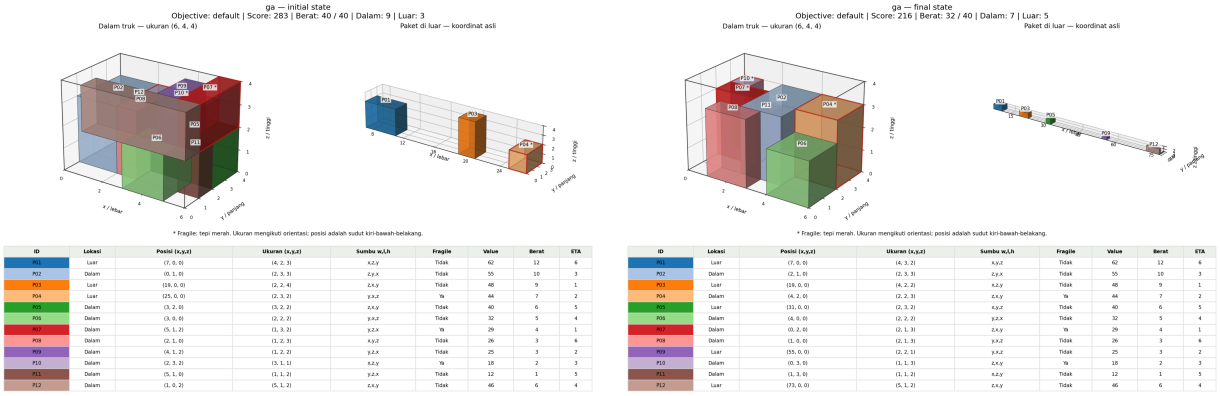
Run 2 - Iterasi 200, Populasi 20

Untuk run 2, berikut adalah hasil akhir yang didapatkan.

- Jumlah Populasi : 20
- Banyak Iterasi : 200
- Seed : 102
- Skor Awal : 283 (berat awal: 40/40)
- Skor Akhir : 216 (berat akhir: 32/40)
- Rata-rata Skor Populasi Akhir : 216.00
- Durasi Pencarian : 0.334 detik

Tabel 2.6.9 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i200-p20 Genetic Algorithm

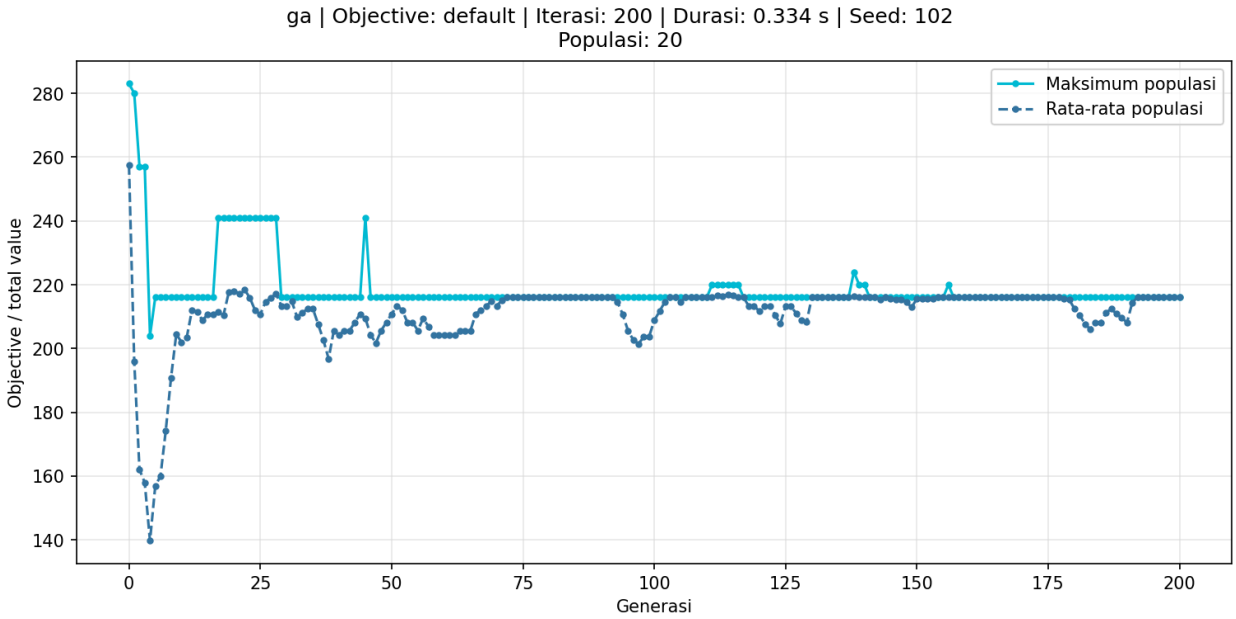
ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(4, 2, 3)	Luar	(7, 0, 0)	(4, 3, 2)
P02	Dalam	(0, 1, 0)	(2, 3, 3)	Dalam	(2, 1, 0)	(2, 3, 3)
P03	Luar	(19, 0, 0)	(2, 2, 4)	Luar	(19, 0, 0)	(4, 2, 2)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Dalam	(4, 2, 0)	(2, 2, 3)
P05	Dalam	(3, 2, 0)	(3, 2, 2)	Luar	(31, 0, 0)	(2, 3, 2)
P06	Dalam	(3, 0, 0)	(2, 2, 2)	Dalam	(4, 0, 0)	(2, 2, 2)
P07	Dalam	(5, 1, 2)	(1, 3, 2)	Dalam	(0, 2, 0)	(2, 1, 3)
P08	Dalam	(2, 1, 0)	(1, 2, 3)	Dalam	(1, 0, 0)	(2, 1, 3)
P09	Dalam	(4, 1, 2)	(1, 2, 2)	Luar	(55, 0, 0)	(2, 2, 1)
P10	Dalam	(2, 3, 2)	(3, 1, 1)	Dalam	(0, 3, 0)	(1, 1, 3)
P11	Dalam	(5, 1, 0)	(1, 1, 2)	Dalam	(1, 3, 0)	(1, 1, 2)
P12	Dalam	(1, 0, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(5, 1, 2)



ID	Lokasi	Posisi (x,y,z)	Ukuran (x,y,z)	Sumbu w,l,h	Fragile	Value	Berat	ETA
P01	Luar	(7, 0, 0)	(4, 2, 3)	x,z,y	Tidak	62	12	6
P02	Dalam	(0, 1, 0)	(2, 3, 3)	z,y,x	Tidak	55	10	3
P03	Luar	(19, 0, 0)	(2, 2, 4)	z,x,y	Tidak	48	9	1
P04	Luar	(25, 0, 0)	(2, 3, 2)	y,x,z	Ya	44	7	2
P05	Dalam	(3, 2, 0)	(3, 2, 2)	z,x,y	Tidak	40	6	5
P06	Dalam	(3, 0, 0)	(2, 2, 2)	y,x,z	Tidak	32	5	4
P07	Dalam	(5, 1, 2)	(1, 3, 2)	y,z,x	Ya	29	4	1
P08	Dalam	(2, 1, 0)	(1, 2, 3)	x,y,z	Tidak	26	3	6
P09	Dalam	(4, 1, 2)	(1, 2, 2)	y,z,x	Tidak	25	3	2
P10	Dalam	(2, 3, 2)	(3, 1, 1)	x,z,y	Ya	18	2	3
P11	Dalam	(5, 1, 0)	(1, 1, 2)	y,z,x	Tidak	12	1	5
P12	Dalam	(1, 0, 2)	(5, 1, 2)	z,x,y	Tidak	46	6	4

ID	Lokasi	Posisi (x,y,z)	Ukuran (x,y,z)	Sumbu w,l,h	Fragile	Value	Berat	ETA
P01	Luar	(7, 0, 0)	(4, 3, 2)	x,y,z	Tidak	62	12	6
P02	Dalam	(2, 1, 0)	(2, 3, 3)	z,y,x	Tidak	55	10	3
P03	Luar	(19, 0, 0)	(4, 2, 2)	x,z,y	Tidak	48	9	1
P04	Dalam	(4, 2, 0)	(2, 2, 3)	z,x,y	Ya	44	7	2
P05	Luar	(31, 0, 0)	(2, 3, 2)	x,y,z	Tidak	40	6	5
P06	Dalam	(4, 0, 0)	(2, 2, 2)	y,z,x	Tidak	32	5	4
P07	Dalam	(0, 2, 0)	(2, 1, 3)	z,x,y	Ya	29	4	1
P08	Dalam	(1, 0, 0)	(2, 1, 3)	y,x,z	Tidak	26	3	6
P09	Luar	(55, 0, 0)	(2, 2, 1)	y,x,z	Tidak	25	3	2
P10	Dalam	(0, 3, 0)	(1, 1, 3)	z,x,y	Ya	18	2	3
P11	Dalam	(1, 3, 0)	(1, 1, 2)	x,z,y	Tidak	12	1	5
P12	Luar	(73, 0, 0)	(5, 1, 2)	z,x,y	Tidak	46	6	4

Gambar 2.6.15 Visualisasi Konfigurasi Awal dan Akhir Run 2-i200-p20 Genetic Algorithm



Gambar 2.6.16 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i200-p20 Genetic Algorithm

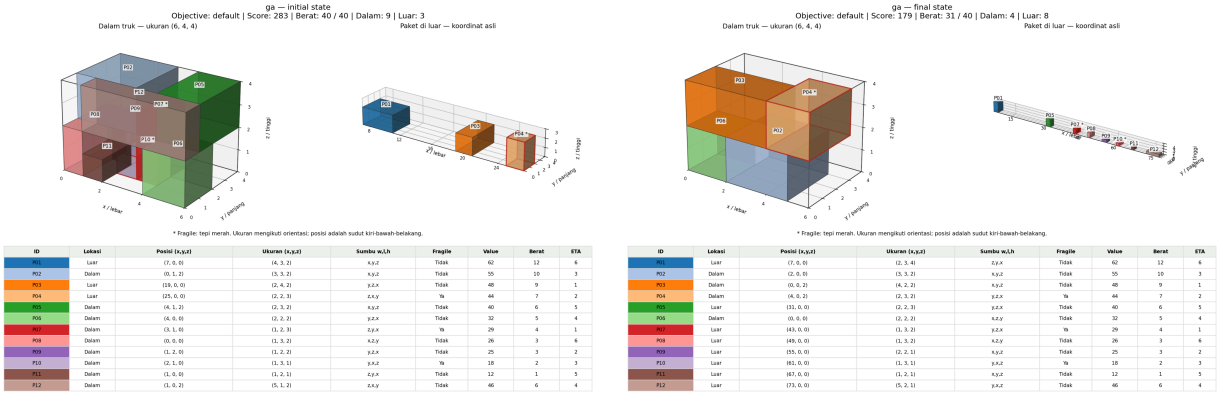
Run 3 - Iterasi 200, Populasi 20

Untuk run 3, berikut adalah hasil akhir yang didapatkan.

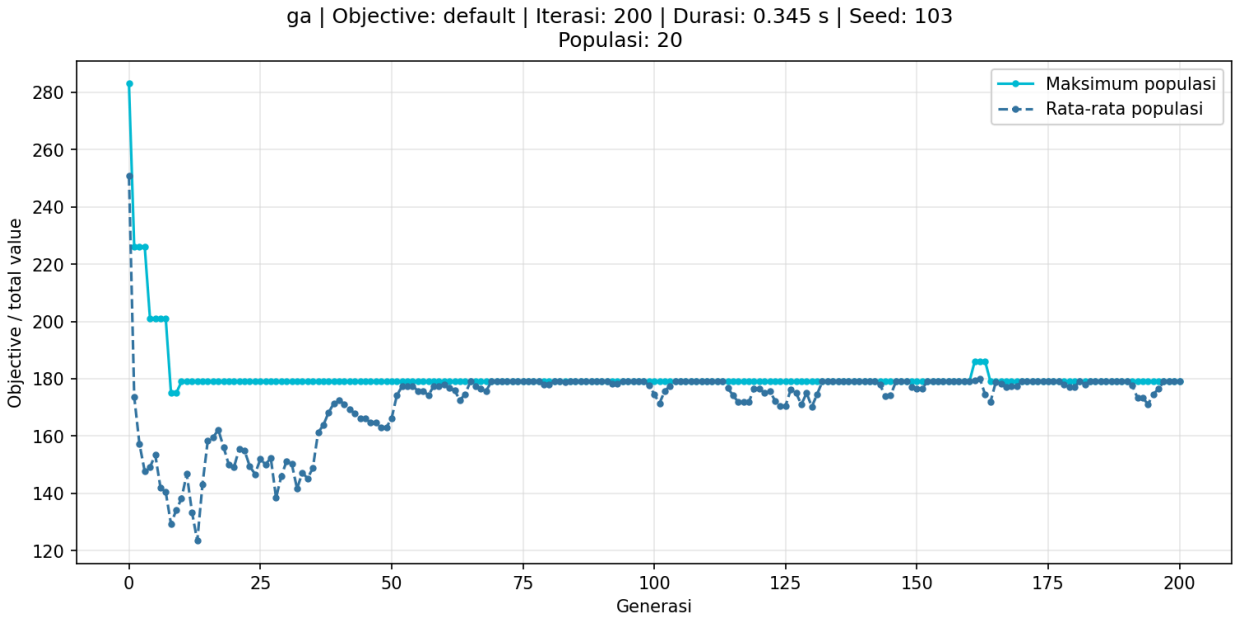
- Jumlah Populasi : 20
- Banyak Iterasi : 200
- Seed : 103
- Skor Awal : 283 (berat awal: 40/40)
- Skor Akhir : 179 (berat akhir: 31/40)
- Rata-rata Skor Populasi Akhir : 179.00
- Durasi Pencarian : 0.345 detik

Tabel 2.6.10 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i200-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(4, 3, 2)	Luar	(7, 0, 0)	(2, 3, 4)
P02	Dalam	(0, 1, 2)	(3, 3, 2)	Dalam	(2, 0, 0)	(3, 3, 2)
P03	Luar	(19, 0, 0)	(2, 4, 2)	Dalam	(0, 0, 2)	(4, 2, 2)
P04	Luar	(25, 0, 0)	(2, 2, 3)	Dalam	(4, 0, 2)	(2, 3, 2)
P05	Dalam	(4, 1, 2)	(2, 3, 2)	Luar	(31, 0, 0)	(2, 2, 3)
P06	Dalam	(4, 0, 0)	(2, 2, 2)	Dalam	(0, 0, 0)	(2, 2, 2)
P07	Dalam	(3, 1, 0)	(1, 2, 3)	Luar	(43, 0, 0)	(1, 3, 2)
P08	Dalam	(0, 0, 0)	(1, 3, 2)	Luar	(49, 0, 0)	(1, 3, 2)
P09	Dalam	(1, 2, 0)	(1, 2, 2)	Luar	(55, 0, 0)	(2, 2, 1)
P10	Dalam	(2, 1, 0)	(1, 3, 1)	Luar	(61, 0, 0)	(1, 3, 1)
P11	Dalam	(1, 0, 0)	(1, 2, 1)	Luar	(67, 0, 0)	(1, 2, 1)
P12	Dalam	(1, 0, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(5, 2, 1)



Gambar 2.6.17 Visualisasi Konfigurasi Awal dan Akhir Run 3-i200-p20 Genetic Algorithm



Gambar 2.6.18 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i200-p20 Genetic Algorithm

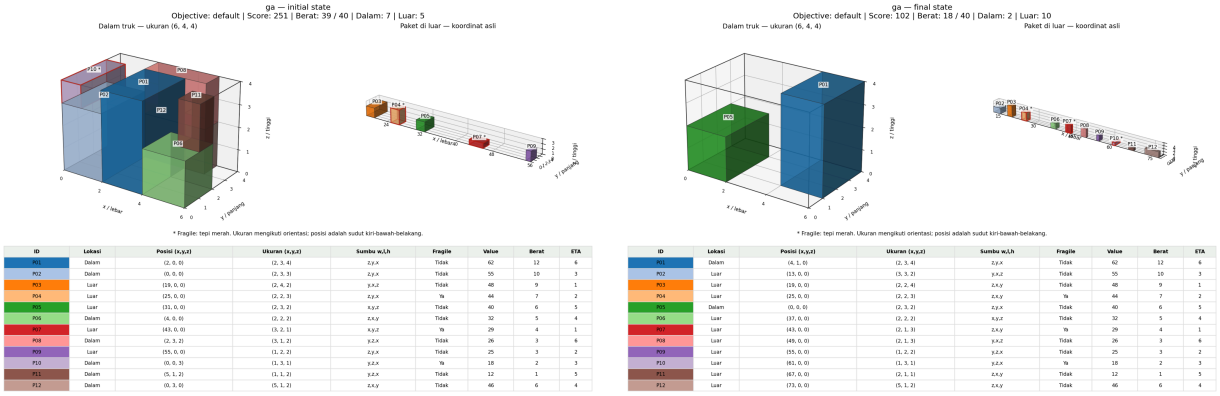
Run 1 - Populasi 10, Iterasi 100

Untuk run 1, berikut adalah hasil akhir yang didapatkan.

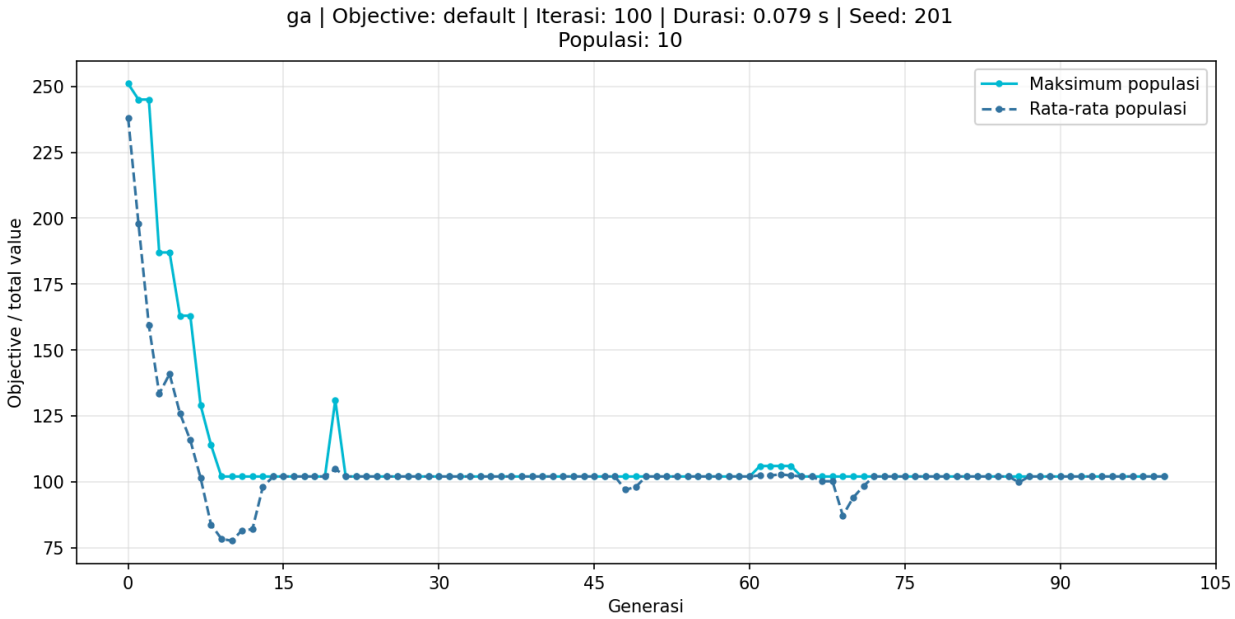
- Jumlah Populasi : 10
- Banyak Iterasi : 100
- Seed : 201
- Skor Awal : 251 (berat awal: 39/40)
- Skor Akhir : 102 (berat akhir: 18/40)
- Rata-rata Skor Populasi Akhir : 102.00
- Durasi Pencarian : 0.079 detik

Tabel 2.6.11 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i100-p10 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Dalam	(2, 0, 0)	(2, 3, 4)	Dalam	(4, 1, 0)	(2, 3, 4)
P02	Dalam	(0, 0, 0)	(2, 3, 3)	Luar	(13, 0, 0)	(3, 3, 2)
P03	Luar	(19, 0, 0)	(2, 4, 2)	Luar	(19, 0, 0)	(2, 2, 4)
P04	Luar	(25, 0, 0)	(2, 2, 3)	Luar	(25, 0, 0)	(2, 2, 3)
P05	Luar	(31, 0, 0)	(2, 3, 2)	Dalam	(0, 0, 0)	(2, 3, 2)
P06	Dalam	(4, 0, 0)	(2, 2, 2)	Luar	(37, 0, 0)	(2, 2, 2)
P07	Luar	(43, 0, 0)	(3, 2, 1)	Luar	(43, 0, 0)	(2, 1, 3)
P08	Dalam	(2, 3, 2)	(3, 1, 2)	Luar	(49, 0, 0)	(2, 1, 3)
P09	Luar	(55, 0, 0)	(1, 2, 2)	Luar	(55, 0, 0)	(1, 2, 2)
P10	Dalam	(0, 0, 3)	(1, 3, 1)	Luar	(61, 0, 0)	(1, 3, 1)
P11	Dalam	(5, 1, 2)	(1, 1, 2)	Luar	(67, 0, 0)	(2, 1, 1)
P12	Dalam	(0, 3, 0)	(5, 1, 2)	Luar	(73, 0, 0)	(5, 1, 2)



Gambar 2.6.19 Visualisasi Konfigurasi Awal dan Akhir Run 1-i100-p10 Genetic Algorithm



Gambar 2.6.20 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i100-p10 Genetic Algorithm

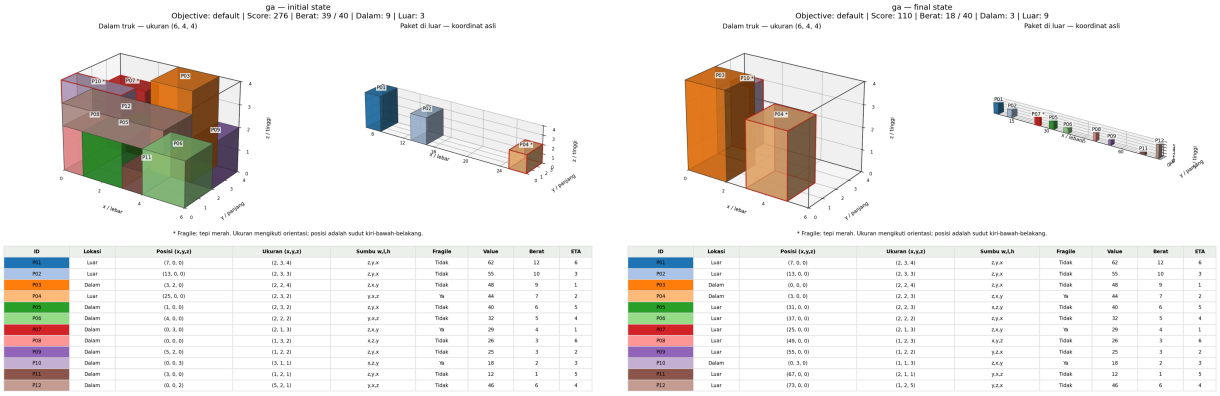
Run 2 - Populasi 10, Iterasi 100

Untuk run 2, berikut adalah hasil akhir yang didapatkan.

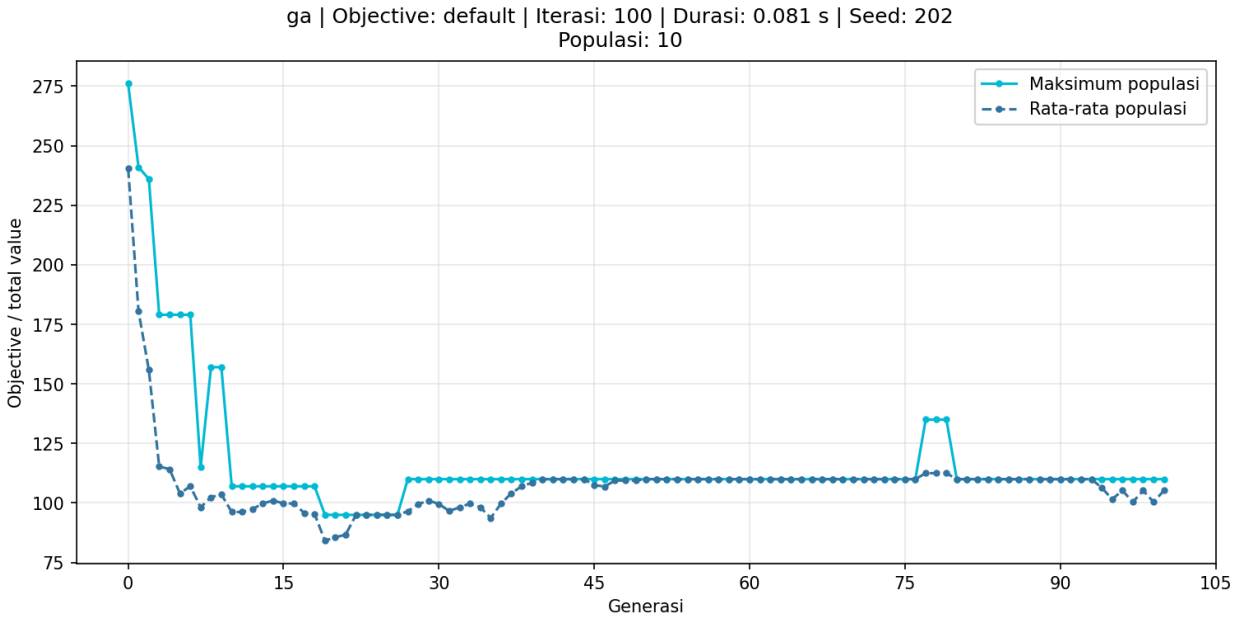
- Jumlah Populasi : 10
- Banyak Iterasi : 100
- Seed : 202
- Skor Awal : 276 (berat awal: 39/40)
- Skor Akhir : 110 (berat akhir: 18/40)
- Rata-rata Skor Populasi Akhir : 105.20
- Durasi Pencarian : 0.081 detik

Tabel 2.6.12 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i100-p10 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 3, 4)	Luar	(7, 0, 0)	(2, 3, 4)
P02	Luar	(13, 0, 0)	(2, 3, 3)	Luar	(13, 0, 0)	(2, 3, 3)
P03	Dalam	(3, 2, 0)	(2, 2, 4)	Dalam	(0, 0, 0)	(2, 2, 4)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Dalam	(3, 0, 0)	(2, 2, 3)
P05	Dalam	(1, 0, 0)	(2, 3, 2)	Luar	(31, 0, 0)	(2, 2, 3)
P06	Dalam	(4, 0, 0)	(2, 2, 2)	Luar	(37, 0, 0)	(2, 2, 2)
P07	Dalam	(0, 3, 0)	(2, 1, 3)	Luar	(25, 0, 0)	(2, 1, 3)
P08	Dalam	(0, 0, 0)	(1, 3, 2)	Luar	(49, 0, 0)	(1, 2, 3)
P09	Dalam	(5, 2, 0)	(1, 2, 2)	Luar	(55, 0, 0)	(1, 2, 2)
P10	Dalam	(0, 0, 3)	(3, 1, 1)	Dalam	(0, 3, 0)	(1, 1, 3)
P11	Dalam	(3, 0, 0)	(1, 2, 1)	Luar	(67, 0, 0)	(2, 1, 1)
P12	Dalam	(0, 0, 2)	(5, 2, 1)	Luar	(73, 0, 0)	(1, 2, 5)



Gambar 2.6.21 Visualisasi Konfigurasi Awal dan Akhir Run 2-i100-p10 Genetic Algorithm



Gambar 2.6.22 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i100-p10 Genetic Algorithm

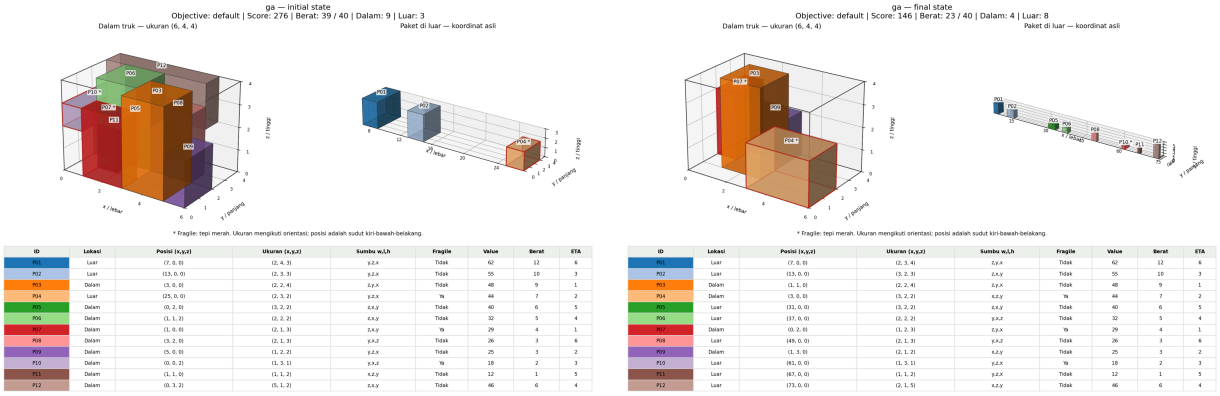
Run 3 - Populasi 10, Iterasi 100

Untuk run 3, berikut adalah hasil akhir yang didapatkan.

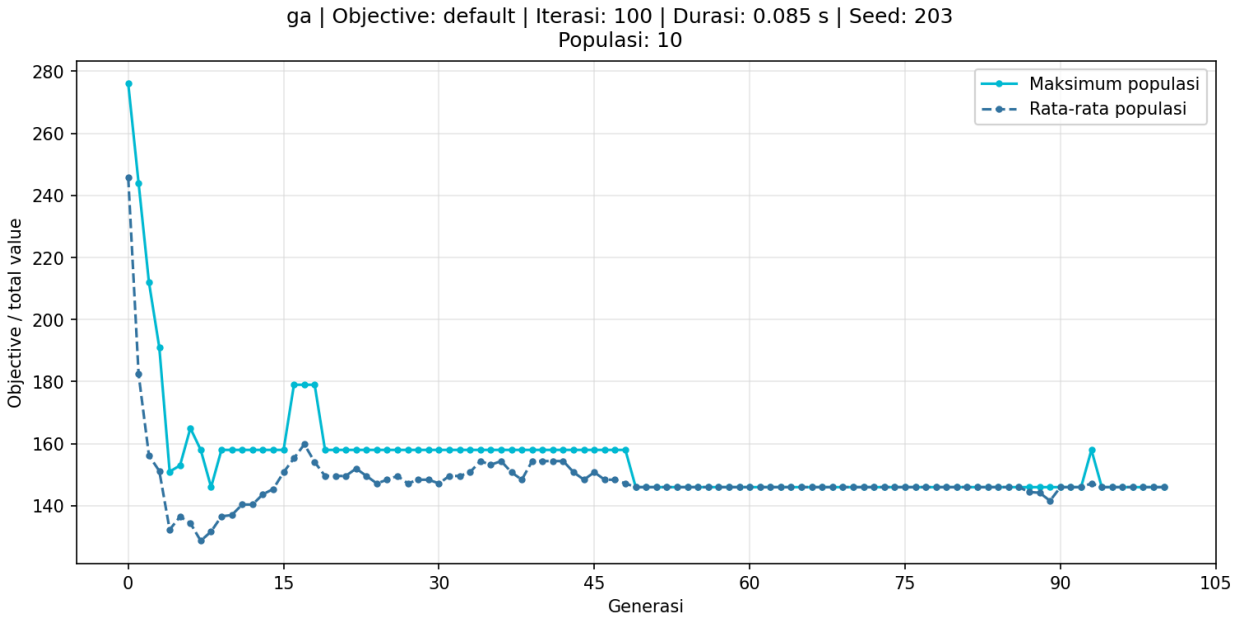
- Jumlah Populasi : 10
- Banyak Iterasi : 100
- Seed : 203
- Skor Awal : 276 (berat awal: 39/40)
- Skor Akhir : 146 (berat akhir: 23/40)
- Rata-rata Skor Populasi Akhir : 146.00
- Durasi Pencarian : 0.085 detik

Tabel 2.6.13 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i100-p10 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 4, 3)	Luar	(7, 0, 0)	(2, 3, 4)
P02	Luar	(13, 0, 0)	(2, 3, 3)	Luar	(13, 0, 0)	(3, 2, 3)
P03	Dalam	(3, 0, 0)	(2, 2, 4)	Dalam	(1, 1, 0)	(2, 2, 4)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Dalam	(3, 0, 0)	(3, 2, 2)
P05	Dalam	(0, 2, 0)	(3, 2, 2)	Luar	(31, 0, 0)	(3, 2, 2)
P06	Dalam	(1, 1, 2)	(2, 2, 2)	Luar	(37, 0, 0)	(2, 2, 2)
P07	Dalam	(1, 0, 0)	(2, 1, 3)	Dalam	(0, 2, 0)	(1, 2, 3)
P08	Dalam	(3, 2, 0)	(2, 1, 3)	Luar	(49, 0, 0)	(2, 1, 3)
P09	Dalam	(5, 0, 0)	(1, 2, 2)	Dalam	(1, 3, 0)	(2, 1, 2)
P10	Dalam	(0, 0, 2)	(1, 3, 1)	Luar	(61, 0, 0)	(1, 3, 1)
P11	Dalam	(1, 1, 0)	(1, 1, 2)	Luar	(67, 0, 0)	(1, 1, 2)
P12	Dalam	(0, 3, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(2, 1, 5)



Gambar 2.6.23 Visualisasi Konfigurasi Awal dan Akhir Run 3-i100-p10 Genetic Algorithm



Gambar 2.6.24 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i100-p10 Genetic Algorithm

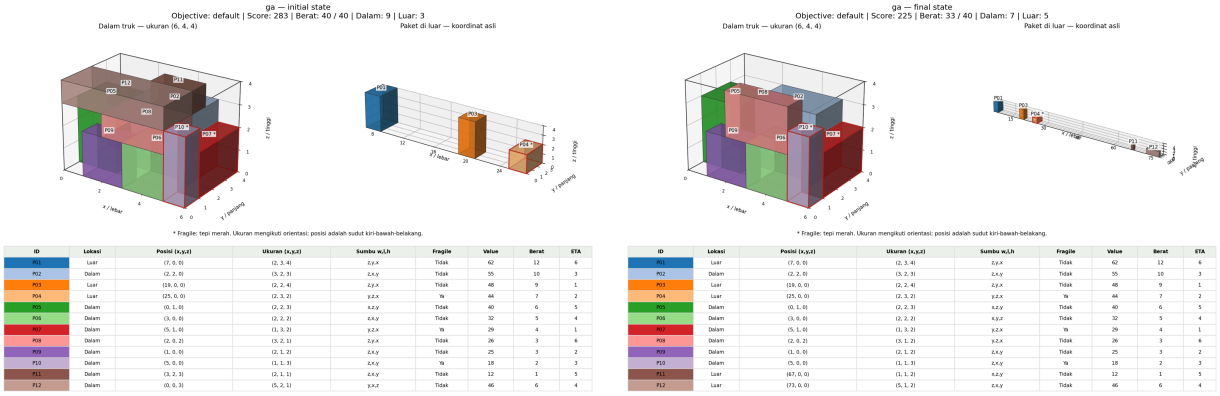
Run 1 - Populasi 20, Iterasi 100

Untuk run 1, berikut adalah hasil akhir yang didapatkan.

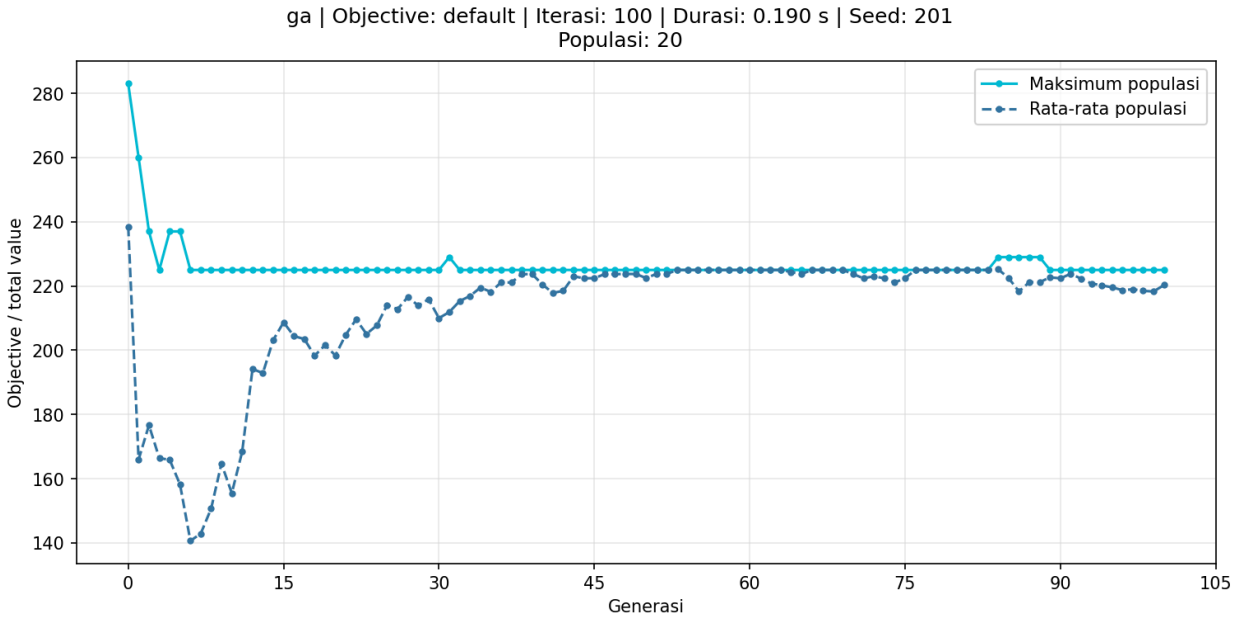
- Jumlah Populasi : 20
- Banyak Iterasi : 100
- Seed : 201
- Skor Awal : 283 (berat awal: 40/40)
- Skor Akhir : 225 (berat akhir: 33/40)
- Rata-rata Skor Populasi Akhir : 220.30
- Durasi Pencarian : 0.190 detik

Tabel 2.6.14 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i100-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 3, 4)	Luar	(7, 0, 0)	(2, 3, 4)
P02	Dalam	(2, 2, 0)	(3, 2, 3)	Dalam	(2, 2, 0)	(3, 2, 3)
P03	Luar	(19, 0, 0)	(2, 2, 4)	Luar	(19, 0, 0)	(2, 2, 4)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Luar	(25, 0, 0)	(2, 3, 2)
P05	Dalam	(0, 1, 0)	(2, 2, 3)	Dalam	(0, 1, 0)	(2, 2, 3)
P06	Dalam	(3, 0, 0)	(2, 2, 2)	Dalam	(3, 0, 0)	(2, 2, 2)
P07	Dalam	(5, 1, 0)	(1, 3, 2)	Dalam	(5, 1, 0)	(1, 3, 2)
P08	Dalam	(2, 0, 2)	(3, 2, 1)	Dalam	(2, 0, 2)	(3, 1, 2)
P09	Dalam	(1, 0, 0)	(2, 1, 2)	Dalam	(1, 0, 0)	(2, 1, 2)
P10	Dalam	(5, 0, 0)	(1, 1, 3)	Dalam	(5, 0, 0)	(1, 1, 3)
P11	Dalam	(3, 2, 3)	(2, 1, 1)	Luar	(67, 0, 0)	(1, 1, 2)
P12	Dalam	(0, 0, 3)	(5, 2, 1)	Luar	(73, 0, 0)	(5, 1, 2)



Gambar 2.6.25 Visualisasi Konfigurasi Awal dan Akhir Run 1-i100-p20 Genetic Algorithm



Gambar 2.6.26 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i100-p20 Genetic Algorithm

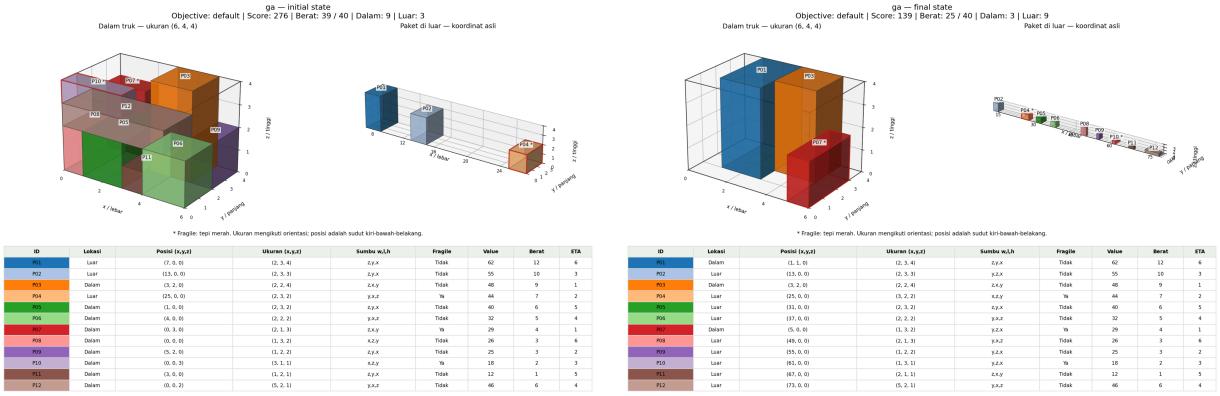
Run 2 - Populasi 20, Iterasi 100

Untuk run 2, berikut adalah hasil akhir yang didapatkan.

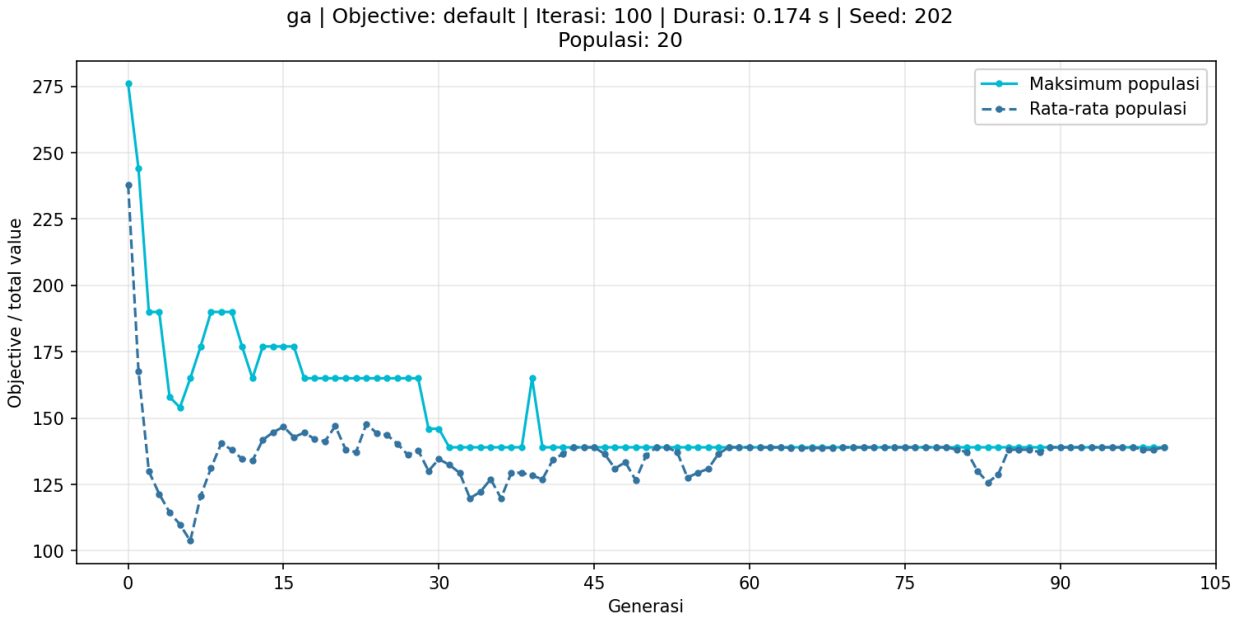
- Jumlah Populasi : 20
- Banyak Iterasi : 100
- Seed : 202
- Skor Awal : 276 (berat awal: 39/40)
- Skor Akhir : 139 (berat akhir: 25/40)
- Rata-rata Skor Populasi Akhir : 139.00
- Durasi Pencarian : 0.174 detik

Tabel 2.6.15 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i100-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 3, 4)	Dalam	(1, 1, 0)	(2, 3, 4)
P02	Luar	(13, 0, 0)	(2, 3, 3)	Luar	(13, 0, 0)	(2, 3, 3)
P03	Dalam	(3, 2, 0)	(2, 2, 4)	Dalam	(3, 2, 0)	(2, 2, 4)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Luar	(25, 0, 0)	(3, 2, 2)
P05	Dalam	(1, 0, 0)	(2, 3, 2)	Luar	(31, 0, 0)	(2, 3, 2)
P06	Dalam	(4, 0, 0)	(2, 2, 2)	Luar	(37, 0, 0)	(2, 2, 2)
P07	Dalam	(0, 3, 0)	(2, 1, 3)	Dalam	(5, 0, 0)	(1, 3, 2)
P08	Dalam	(0, 0, 0)	(1, 3, 2)	Luar	(49, 0, 0)	(2, 1, 3)
P09	Dalam	(5, 2, 0)	(1, 2, 2)	Luar	(55, 0, 0)	(1, 2, 2)
P10	Dalam	(0, 0, 3)	(3, 1, 1)	Luar	(61, 0, 0)	(1, 3, 1)
P11	Dalam	(3, 0, 0)	(1, 2, 1)	Luar	(67, 0, 0)	(2, 1, 1)
P12	Dalam	(0, 0, 2)	(5, 2, 1)	Luar	(73, 0, 0)	(5, 2, 1)



Gambar 2.6.27 Visualisasi Konfigurasi Awal dan Akhir Run 2-i100-p20 Genetic Algorithm



Gambar 2.6.28 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i100-p20 Genetic Algorithm

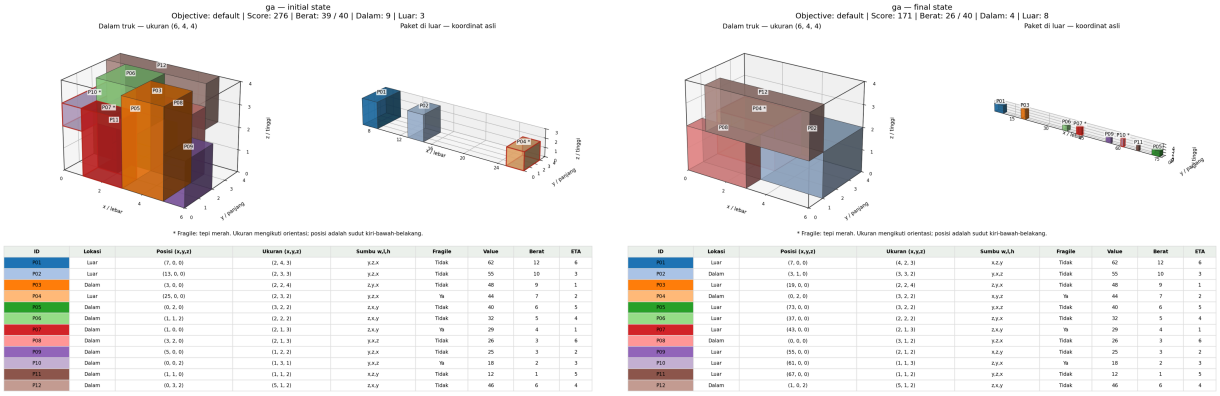
Run 3 - Populasi 20, Iterasi 100

Untuk run 3, berikut adalah hasil akhir yang didapatkan.

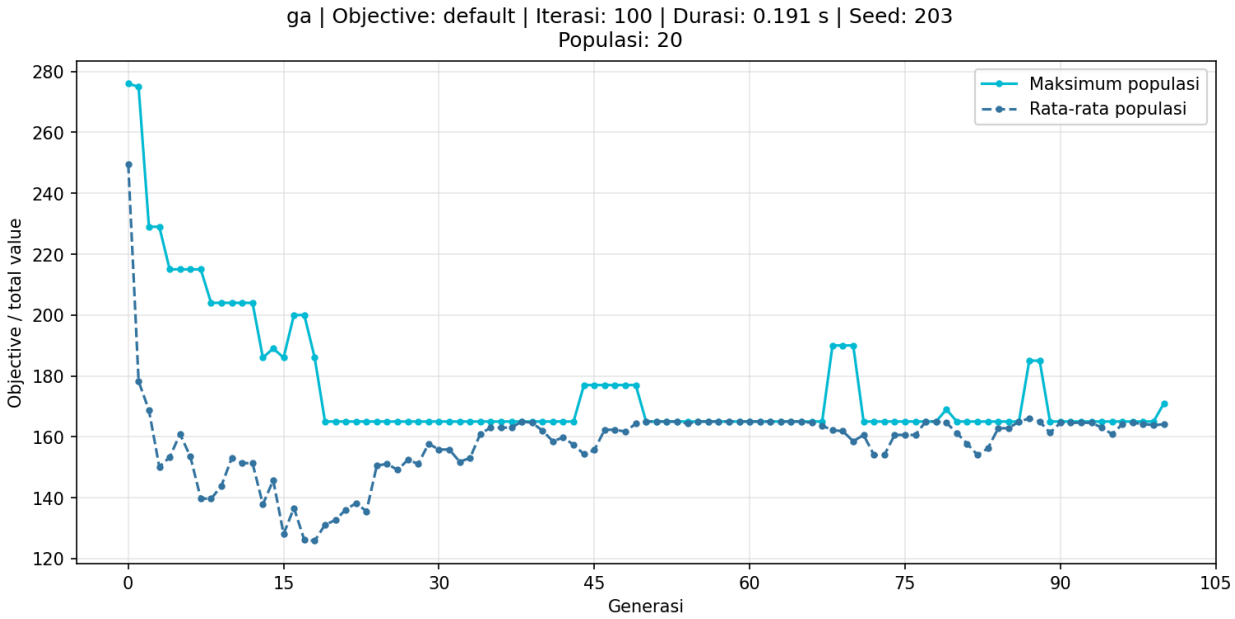
- Jumlah Populasi : 20
- Banyak Iterasi : 100
- Seed : 203
- Skor Awal : 276 (berat awal: 39/40)
- Skor Akhir : 171 (berat akhir: 26/40)
- Rata-rata Skor Populasi Akhir : 164.10
- Durasi Pencarian : 0.191 detik

Tabel 2.6.16 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i100-p20 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 4, 3)	Luar	(7, 0, 0)	(4, 2, 3)
P02	Luar	(13, 0, 0)	(2, 3, 3)	Dalam	(3, 1, 0)	(3, 3, 2)
P03	Dalam	(3, 0, 0)	(2, 2, 4)	Luar	(19, 0, 0)	(2, 2, 4)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Dalam	(0, 2, 0)	(3, 2, 2)
P05	Dalam	(0, 2, 0)	(3, 2, 2)	Luar	(73, 0, 0)	(3, 2, 2)
P06	Dalam	(1, 1, 2)	(2, 2, 2)	Luar	(37, 0, 0)	(2, 2, 2)
P07	Dalam	(1, 0, 0)	(2, 1, 3)	Luar	(43, 0, 0)	(2, 1, 3)
P08	Dalam	(3, 2, 0)	(2, 1, 3)	Dalam	(0, 0, 0)	(3, 1, 2)
P09	Dalam	(5, 0, 0)	(1, 2, 2)	Luar	(55, 0, 0)	(2, 1, 2)
P10	Dalam	(0, 0, 2)	(1, 3, 1)	Luar	(61, 0, 0)	(1, 1, 3)
P11	Dalam	(1, 1, 0)	(1, 1, 2)	Luar	(67, 0, 0)	(1, 1, 2)
P12	Dalam	(0, 3, 2)	(5, 1, 2)	Dalam	(1, 0, 2)	(5, 1, 2)



Gambar 2.6.29 Visualisasi Konfigurasi Awal dan Akhir Run 3-i100-p20 Genetic Algorithm



Gambar 2.6.30 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i100-p20 Genetic Algorithm

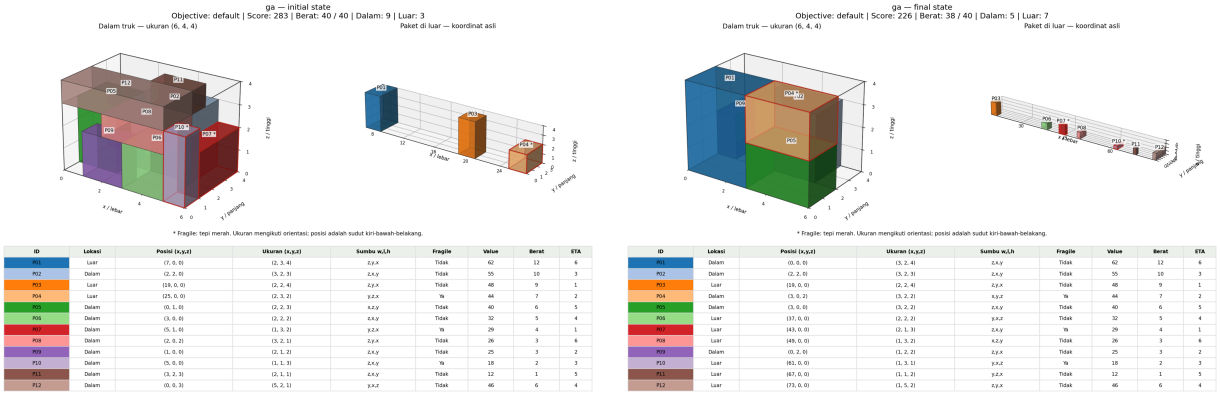
Run 1 - Populasi 40, Iterasi 100

Untuk run 1, berikut adalah hasil akhir yang didapatkan.

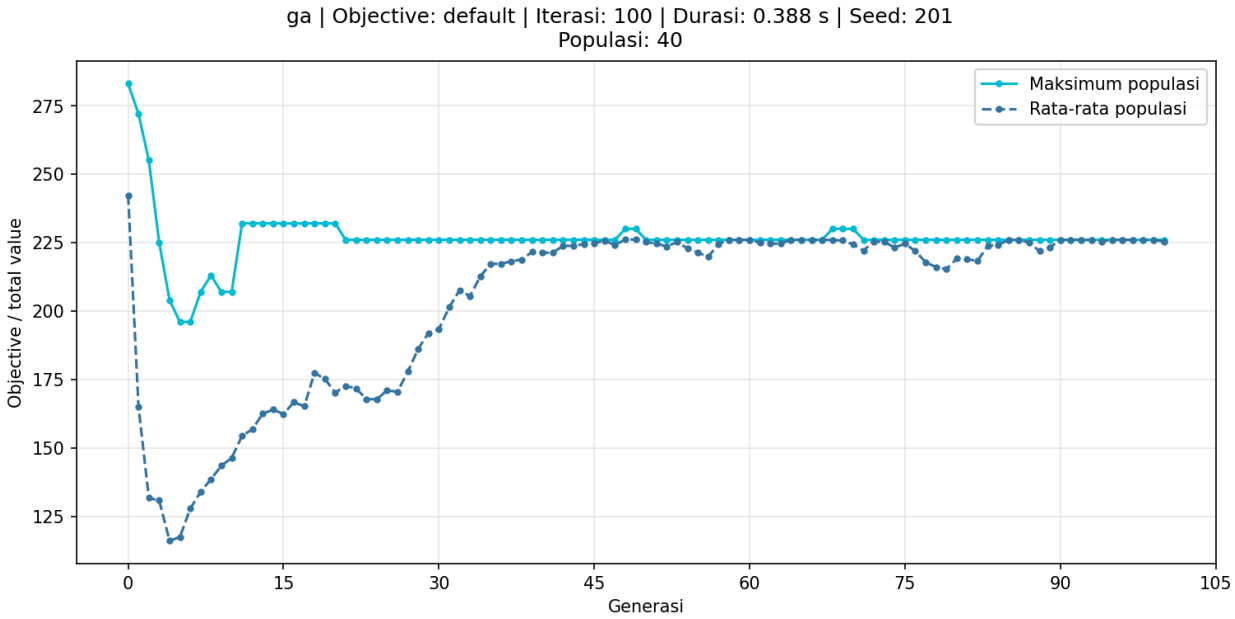
- Jumlah Populasi : 40
- Banyak Iterasi : 100
- Seed : 201
- Skor Awal : 283 (berat awal: 40/40)
- Skor Akhir : 226 (berat akhir: 38/40)
- Rata-rata Skor Populasi Akhir : 225.30
- Durasi Pencarian : 0.388 detik

Tabel 2.6.17 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1-i100-p40 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 3, 4)	Dalam	(0, 0, 0)	(3, 2, 4)
P02	Dalam	(2, 2, 0)	(3, 2, 3)	Dalam	(2, 2, 0)	(3, 2, 3)
P03	Luar	(19, 0, 0)	(2, 2, 4)	Luar	(19, 0, 0)	(2, 2, 4)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Dalam	(3, 0, 2)	(3, 2, 2)
P05	Dalam	(0, 1, 0)	(2, 2, 3)	Dalam	(3, 0, 0)	(3, 2, 2)
P06	Dalam	(3, 0, 0)	(2, 2, 2)	Luar	(37, 0, 0)	(2, 2, 2)
P07	Dalam	(5, 1, 0)	(1, 3, 2)	Luar	(43, 0, 0)	(2, 1, 3)
P08	Dalam	(2, 0, 2)	(3, 2, 1)	Luar	(49, 0, 0)	(1, 3, 2)
P09	Dalam	(1, 0, 0)	(2, 1, 2)	Dalam	(0, 2, 0)	(1, 2, 2)
P10	Dalam	(5, 0, 0)	(1, 1, 3)	Luar	(61, 0, 0)	(1, 3, 1)
P11	Dalam	(3, 2, 3)	(2, 1, 1)	Luar	(67, 0, 0)	(1, 1, 2)
P12	Dalam	(0, 0, 3)	(5, 2, 1)	Luar	(73, 0, 0)	(1, 5, 2)



Gambar 2.6.31 Visualisasi Konfigurasi Awal dan Akhir Run 1-i100-p40 Genetic Algorithm



Gambar 2.6.32 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1-i100-p40 Genetic Algorithm

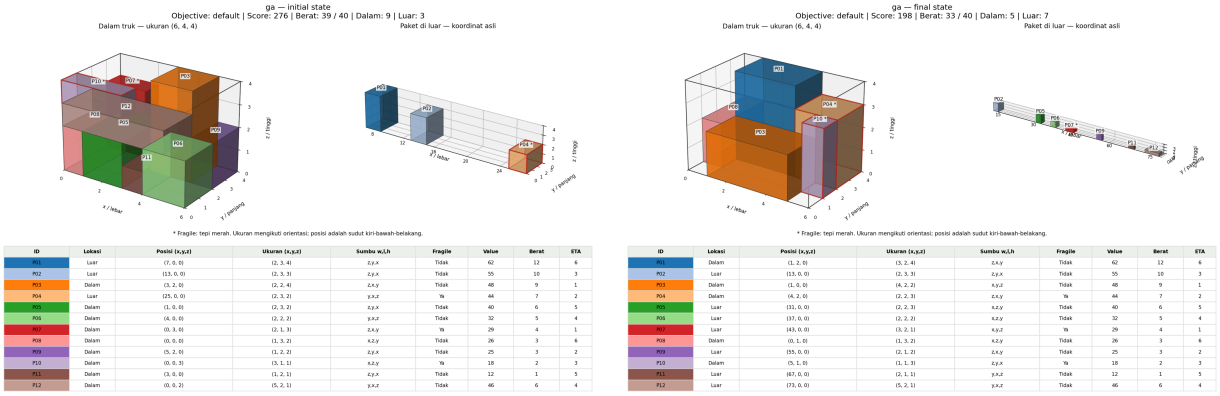
Run 2 - Populasi 40, Iterasi 100

Untuk run 2, berikut adalah hasil akhir yang didapatkan.

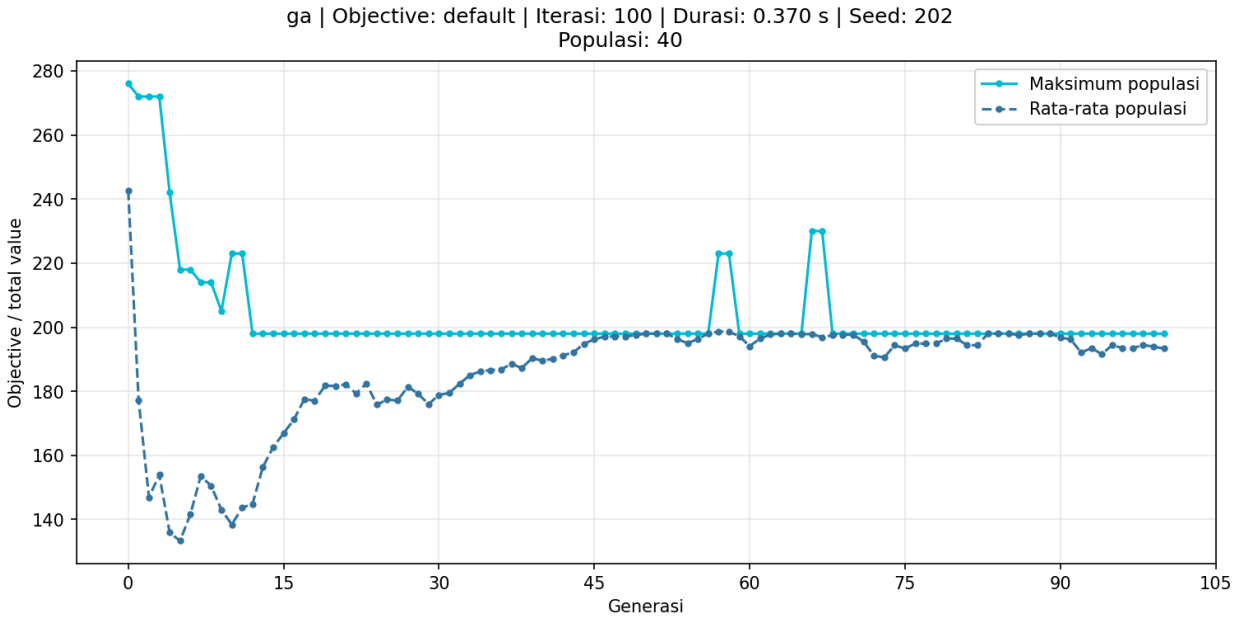
- Jumlah Populasi : 40
- Banyak Iterasi : 100
- Seed : 202
- Skor Awal : 276 (berat awal: 39/40)
- Skor Akhir : 198 (berat akhir: 33/40)
- Rata-rata Skor Populasi Akhir : 193.35
- Durasi Pencarian : 0.370 detik

Tabel 2.6.18 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2-i100-p40 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 3, 4)	Dalam	(1, 2, 0)	(3, 2, 4)
P02	Luar	(13, 0, 0)	(2, 3, 3)	Luar	(13, 0, 0)	(2, 3, 3)
P03	Dalam	(3, 2, 0)	(2, 2, 4)	Dalam	(1, 0, 0)	(4, 2, 2)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Dalam	(4, 2, 0)	(2, 2, 3)
P05	Dalam	(1, 0, 0)	(2, 3, 2)	Luar	(31, 0, 0)	(2, 2, 3)
P06	Dalam	(4, 0, 0)	(2, 2, 2)	Luar	(37, 0, 0)	(2, 2, 2)
P07	Dalam	(0, 3, 0)	(2, 1, 3)	Luar	(43, 0, 0)	(3, 2, 1)
P08	Dalam	(0, 0, 0)	(1, 3, 2)	Dalam	(0, 1, 0)	(1, 3, 2)
P09	Dalam	(5, 2, 0)	(1, 2, 2)	Luar	(55, 0, 0)	(2, 1, 2)
P10	Dalam	(0, 0, 3)	(3, 1, 1)	Dalam	(5, 1, 0)	(1, 1, 3)
P11	Dalam	(3, 0, 0)	(1, 2, 1)	Luar	(67, 0, 0)	(2, 1, 1)
P12	Dalam	(0, 0, 2)	(5, 2, 1)	Luar	(73, 0, 0)	(5, 2, 1)



Gambar 2.6.33 Visualisasi Konfigurasi Awal dan Akhir Run 2-i100-p40 Genetic Algorithm



Gambar 2.6.34 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2-i100-p40 Genetic Algorithm

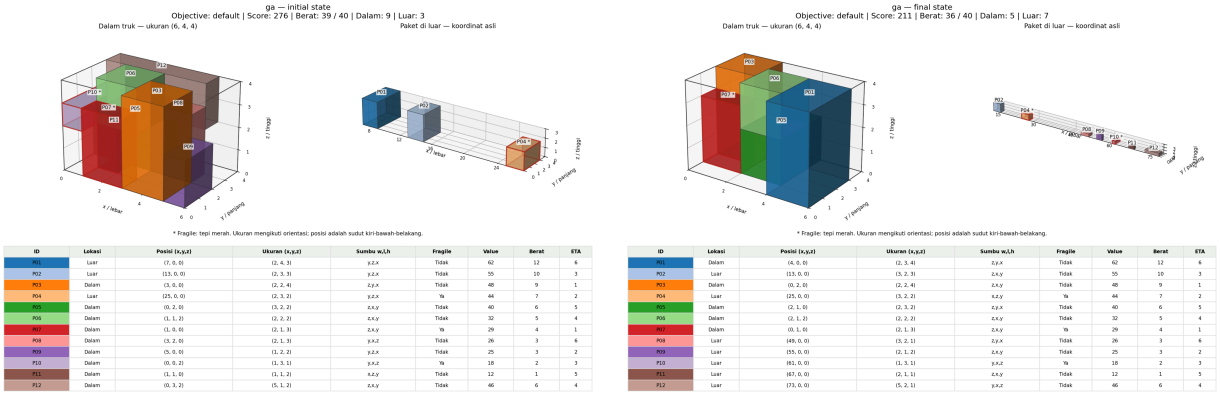
Run 3 - Populasi 40, Iterasi 100

Untuk run 3, berikut adalah hasil akhir yang didapatkan.

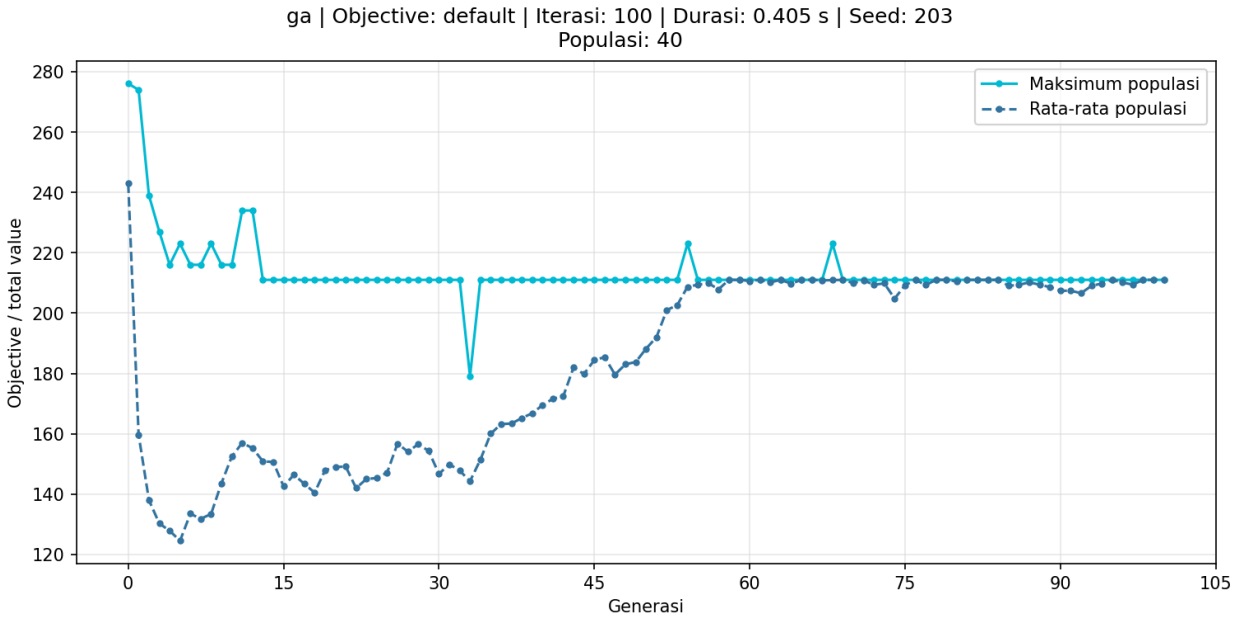
- Jumlah Populasi : 40
- Banyak Iterasi : 100
- Seed : 203
- Skor Awal : 276 (berat awal: 39/40)
- Skor Akhir : 211 (berat akhir: 36/40)
- Rata-rata Skor Populasi Akhir : 211.00
- Durasi Pencarian : 0.405 detik

Tabel 2.6.19 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3-i100-p40 Genetic Algorithm

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7, 0, 0)	(2, 4, 3)	Dalam	(4, 0, 0)	(2, 3, 4)
P02	Luar	(13, 0, 0)	(2, 3, 3)	Luar	(13, 0, 0)	(3, 2, 3)
P03	Dalam	(3, 0, 0)	(2, 2, 4)	Dalam	(0, 2, 0)	(2, 2, 4)
P04	Luar	(25, 0, 0)	(2, 3, 2)	Luar	(25, 0, 0)	(3, 2, 2)
P05	Dalam	(0, 2, 0)	(3, 2, 2)	Dalam	(2, 1, 0)	(2, 3, 2)
P06	Dalam	(1, 1, 2)	(2, 2, 2)	Dalam	(2, 1, 2)	(2, 2, 2)
P07	Dalam	(1, 0, 0)	(2, 1, 3)	Dalam	(0, 1, 0)	(2, 1, 3)
P08	Dalam	(3, 2, 0)	(2, 1, 3)	Luar	(49, 0, 0)	(3, 2, 1)
P09	Dalam	(5, 0, 0)	(1, 2, 2)	Luar	(55, 0, 0)	(2, 1, 2)
P10	Dalam	(0, 0, 2)	(1, 3, 1)	Luar	(61, 0, 0)	(1, 3, 1)
P11	Dalam	(1, 1, 0)	(1, 1, 2)	Luar	(67, 0, 0)	(2, 1, 1)
P12	Dalam	(0, 3, 2)	(5, 1, 2)	Luar	(73, 0, 0)	(5, 2, 1)



Gambar 2.6.35 Visualisasi Konfigurasi Awal dan Akhir Run 3-i100-p40 Genetic Algorithm



Gambar 2.6.36 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3-i100-p40 Genetic Algorithm

Bonus - Fungsi Objektif

Deskripsi Fungsi/Kelas

```
def score_bonus(problem: Problem, state: State) -> int | float:
    _require_valid(problem, state)
    packages = {package.id: package for package in problem.packages}
    boxes = {
        package.id: _bounds(package, state.placements[package.id])
        for package in problem.packages
        if is_inside(problem, package, state.placements[package.id])
    }
    if not boxes:
        return 0

    total = 0.0
    for key, box in boxes.items():
        package = packages[key]
        placement = state.placements[key]
        truck = problem.trucks[placement.truck]
        _, l_truck, h_truck = truck.dimensions.as_tuple()
        capacity = truck.max_capacity
        x1, y1, z1, x2, y2, _ = box
        base_area = (x2 - x1) * (y2 - y1)

        urgency = 1.0 / (1.0 + package.eta)
        access = y2 / l_truck
        eta_component = urgency * (1.0 + access)

        weight_ratio = (package.weight / capacity) if capacity > 0 else 0.0
        height_factor = 1.0 - (z1 / h_truck)
        cog_component = weight_ratio * height_factor

        if z1 == 0:
            support_ratio = 1.0
        else:
            supported_area = sum(
                max(0, min(x2, boxes[other][3]) - max(x1, boxes[other][0]))
                * max(0, min(y2, boxes[other][4]) - max(y1, boxes[other][1]))
                for other in boxes
                if other != key
                and state.placements[other].truck == placement.truck
                and boxes[other][5] == z1
            )
            support_ratio = supported_area / base_area

        total += (
            package.value
            + 2.0 * eta_component
            + 2.0 * cog_component
            + 1.0 * support_ratio
        )
    return round(total, 6)
```

Fungsi `score_bonus` mengevaluasi kualitas penataan paket menggunakan formulasi multifaktor logistik untuk seluruh paket yang berada di dalam truk. Selain menjumlahkan nilai

dasar kargo (`package.value`), fungsi ini memberikan bobot tambahan pada tiga aspek fisik dan operasional, yaitu komponen aksesibilitas bongkar-muat (`eta_component`) yang mendorong paket dengan `eta` kecil ditempatkan dekat pintu belakang truk (y_2 mendekati L_{truck}), komponen titik berat (`cog_component`) yang memprioritaskan paket berat berada di dekat lantai truk (z_1 rendah) , dan rasio luas alas yang tertopang (`support_ratio`) yang mengukur seberapa besar bidang alas paket ditopang oleh lantai atau permukaan atas paket lain pada truk yang sama.

```
OBJECTIVES = {
    "default": score_default,
    "bonus": score_bonus,
}


def set_objective(objective: str) -> str:
    global _ACTIVE_OBJECTIVE
    if objective not in OBJECTIVES:
        raise ValueError(f"Objective harus salah satu dari: {', '.join(OBJECTIVES)}.")
    _ACTIVE_OBJECTIVE = objective
    return _ACTIVE_OBJECTIVE


def get_objective() -> str:
    return _ACTIVE_OBJECTIVE


def score(
    problem: Problem, state: State, *, objective: str | None = None,
) -> int | float:
    chosen = _ACTIVE_OBJECTIVE if objective is None else objective
    if chosen not in OBJECTIVES:
        raise ValueError(f"Objective harus salah satu dari: {', '.join(OBJECTIVES)}.")
    return OBJECTIVES[chosen](problem, state)


def objective_upper_bound(
    problem: Problem, *, objective: str | None = None,
) -> int | float:
    chosen = _ACTIVE_OBJECTIVE if objective is None else objective
    if chosen not in OBJECTIVES:
        raise ValueError(f"Objective harus salah satu dari: {', '.join(OBJECTIVES)}.")
    if chosen == "default":
        return sum(package.value for package in problem.packages)
    min_positive_capacity = min(
        (truck.max_capacity for truck in problem.trucks if truck.max_capacity > 0),
        default=0,
    )
    total = sum(
        package.value
        + 2.0 * (2.0 / (1.0 + package.eta))
        + 2.0 * ((package.weight / min_positive_capacity) if min_positive_capacity > 0 else
0.0)
        + 1.0
        for package in problem.packages
```

```
    )
    return round(total, 6)
```

Keempat fungsi ini mengelola pemilihan dan pemanggilan fungsi objektif secara dinamis di seluruh modul algoritma pencarian. Fungsi `set_objective` dan `get_objective` mengatur serta mengambil mode objektif yang diinginkan pengguna, sedangkan fungsi `score` menjadi fitur yang meneruskan evaluasi state ke `score_default` atau `score_bonus`. Sementara itu, `objective_upper_bound` menghitung batas atas teoritis skor maksimum apabila seluruh paket berhasil dimuat secara ideal di dalam armada truk sehingga algoritma dapat melakukan penghentian lebih awal apabila nilai optimal global telah tercapai.

Sebagai catatan, fungsi objektif pada spesifikasi bonus yang digunakan menggunakan 4 parameter logistik pada kondisi ini yang dapat ditulis ulang sebagai berikut

$$\text{Score}_{\text{bonus}} = \sum_{i \in \text{truk}} \left(v_i + 2 \cdot \frac{1}{1 + \text{eta}_i} \left(1 + \frac{y_{\text{akhir}}}{l_{\text{truk}}} \right) + 2 \cdot \frac{w_i}{\text{cap}} \left(1 - \frac{z_{\text{awal}}}{h_{\text{truk}}} \right) + \text{rasio_support}_i \right)$$

Paket dengan ETA kecil diposisikan dekat pintu belakang supaya proses bongkar muat lebih efisien. Kemudian, paket berbobot berat diletakkan menapak di lantai agar tidak menimpa paket dengan bobot lebih ringan. Terakhir, fungsi objektif juga memerhatikan luas kontak dasar paket dengan lantai di bawahnya.

Tabel 2.7.1 Hasil Eksperimen Fungsi Objektif Bonus dengan Algoritma Simulated Annealing

Run (Seed)	Score Awal	Score Akhir	Score Terbaik	Nilai Kargo Akhir	Berat Akhir	Iterasi	Durasi Pencarian (s)	Episode Stuck	Iterasi Stuck
1 (11)	241.488	232.983	252.555	219	33/40	225	11.424	0	0
2 (22)	258.621	260.109	260.109	243	34/40	225	12.167	0	0
3 (33)	261.761	268.090	268.090	252	40/40	225	5.441	0	0
Rata-rata	253.957	253.727	260.251	238.0		225	9.677	0.0	0.0

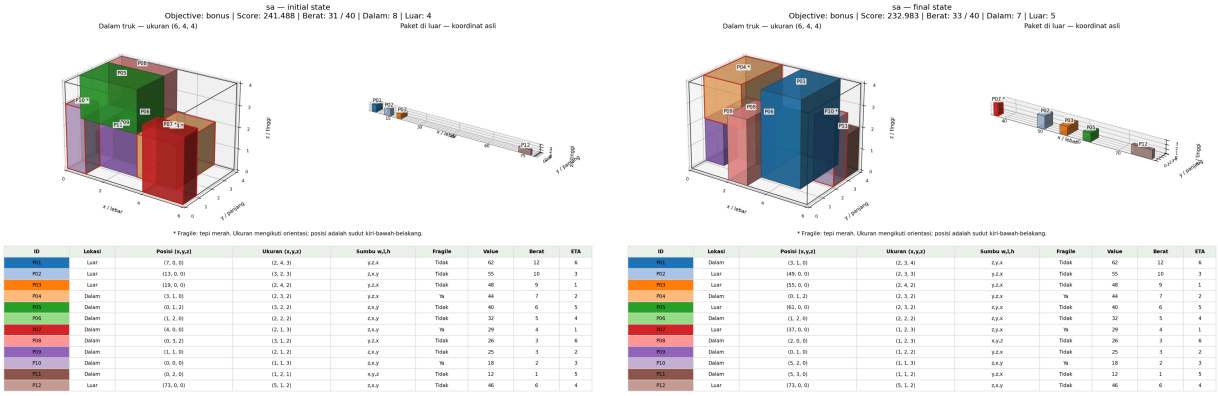
Run 1

Untuk run 1, berikut adalah hasil akhir yang didapatkan.

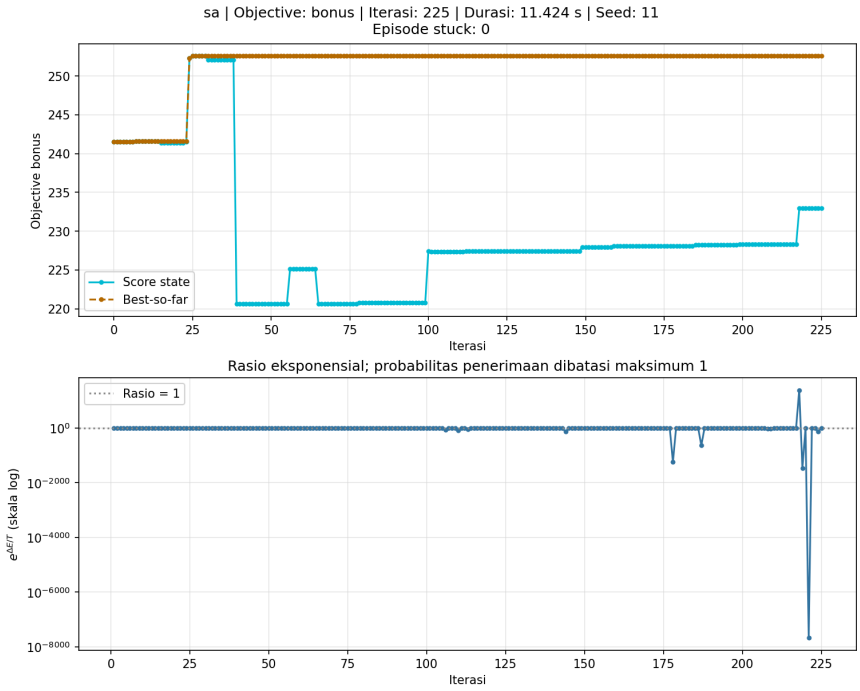
- Score Awal : 241.4881 (nilai kargo: 226, berat: 31/40)
- Score Akhir : 232.9833 (nilai kargo: 219, berat: 33/40)
- Score Terbaik : 252.5548 (nilai kargo: 237, berat: 34/40)
- Durasi Pencarian : 11.424 detik
- Statistik Transisi Langkah
    - Langkah perbaikan diterima : 14
    - Langkah sideways diterima : 187
    - Langkah penurunan diterima : 6
    - Langkah kandidat ditolak : 18

Tabel 2.7.2 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Simulated Annealing dengan Fungsi Objektif Bonus

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(4,2,3)	Luar	(55,0,0)	(4,2,3)
P02	Dalam	(1,0,0)	(3,2,3)	Dalam	(1,0,0)	(3,2,3)
P03	Luar	(19,0,0)	(4,2,2)	Luar	(25,0,0)	(4,2,2)
P04	Luar	(25,0,0)	(2,2,3)	Luar	(31,0,0)	(2,3,2)
P05	Luar	(31,0,0)	(2,2,3)	Luar	(19,0,0)	(2,3,2)
P06	Dalam	(4,2,2)	(2,2,2)	Dalam	(3,2,2)	(2,2,2)
P07	Dalam	(2,3,0)	(2,1,3)	Dalam	(4,0,0)	(2,1,3)
P08	Dalam	(0,1,0)	(1,2,3)	Dalam	(0,1,0)	(1,3,2)
P09	Dalam	(4,0,0)	(1,2,2)	Dalam	(1,3,0)	(2,1,2)
P10	Dalam	(5,1,0)	(1,1,3)	Dalam	(0,1,2)	(1,3,1)
P11	Dalam	(1,1,3)	(1,2,1)	Dalam	(1,0,3)	(1,2,1)
P12	Dalam	(1,2,0)	(5,1,2)	Dalam	(1,2,0)	(5,1,2)



Gambar 2.7.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Simulated Annealing dengan Fungsi Objektif Bonus



Gambar 2.7.2 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1 Simulated Annealing dengan Fungsi Objektif Bonus

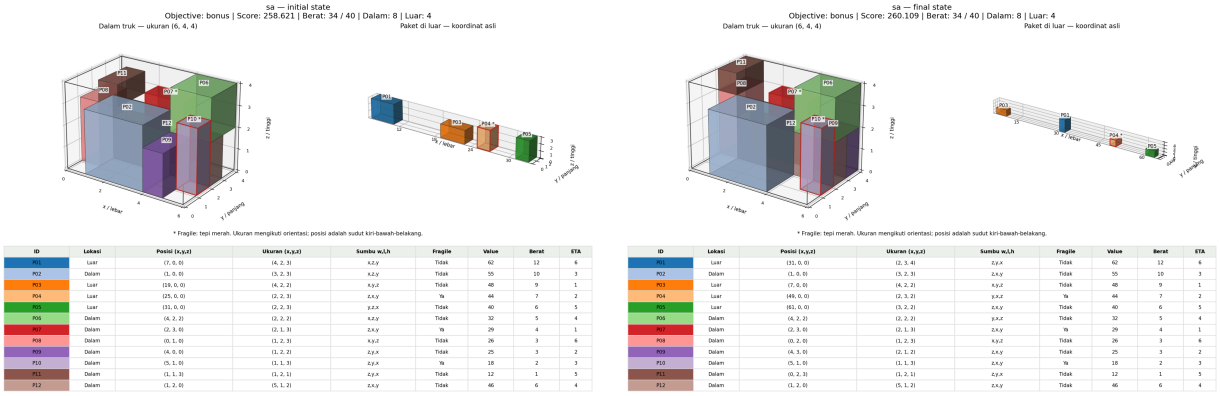
Run 2

Untuk run 2, berikut adalah hasil akhir yang didapatkan.

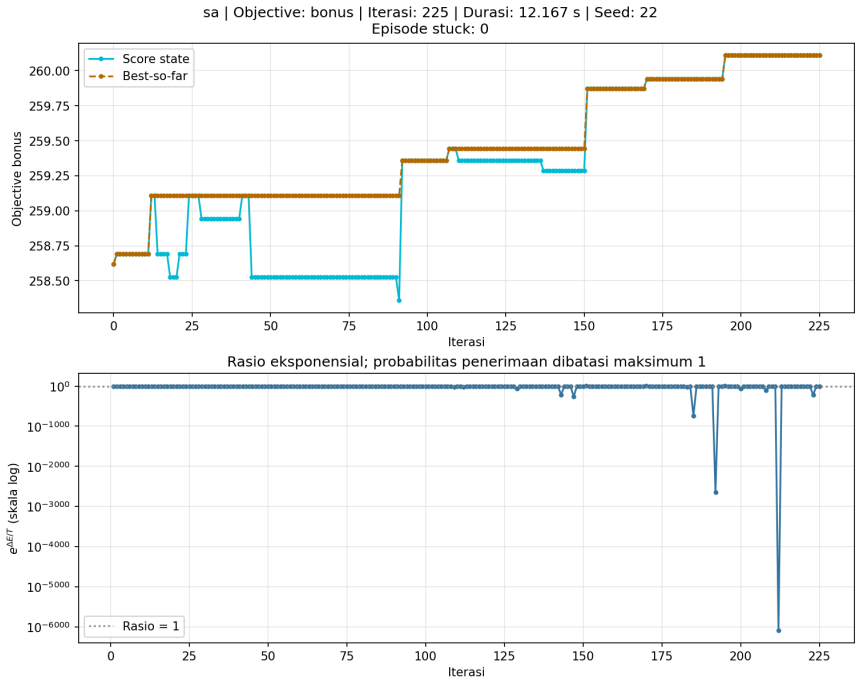
- Score Awal : 258.6208 (nilai kargo: 243, berat: 34/40)
- Score Akhir : 260.1089 (nilai kargo: 243, berat: 34/40)
- Score Terbaik : 260.1089 (nilai kargo: 243, berat: 34/40)
- Durasi Pencarian : 12.167 detik
- Statistik Transisi Langkah
    - Langkah perbaikan diterima : 10
    - Langkah sideways diterima : 189
    - Langkah penurunan diterima : 7
    - Langkah kandidat ditolak : 19

Tabel 2.7.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Simulated Annealing dengan Fungsi Objektif Bonus

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(4,2,3)	Luar	(31,0,0)	(2,3,4)
P02	Dalam	(1,0,0)	(3,2,3)	Dalam	(1,0,0)	(3,2,3)
P03	Luar	(19,0,0)	(4,2,2)	Luar	(7,0,0)	(4,2,2)
P04	Luar	(25,0,0)	(2,2,3)	Luar	(49,0,0)	(2,3,2)
P05	Luar	(31,0,0)	(2,2,3)	Luar	(61,0,0)	(3,2,2)
P06	Dalam	(4,2,2)	(2,2,2)	Dalam	(4,2,2)	(2,2,2)
P07	Dalam	(2,3,0)	(2,1,3)	Dalam	(2,3,0)	(2,1,3)
P08	Dalam	(0,1,0)	(1,2,3)	Dalam	(0,2,0)	(1,2,3)
P09	Dalam	(4,0,0)	(1,2,2)	Dalam	(4,3,0)	(2,1,2)
P10	Dalam	(5,1,0)	(1,1,3)	Dalam	(5,1,0)	(1,1,3)
P11	Dalam	(1,1,3)	(1,2,1)	Dalam	(0,2,3)	(1,2,1)
P12	Dalam	(1,2,0)	(5,1,2)	Dalam	(1,2,0)	(5,1,2)



Gambar 2.7.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Simulated Annealing dengan Fungsi Objektif Bonus



Gambar 2.7.4 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2 Simulated Annealing dengan Fungsi Objektif Bonus

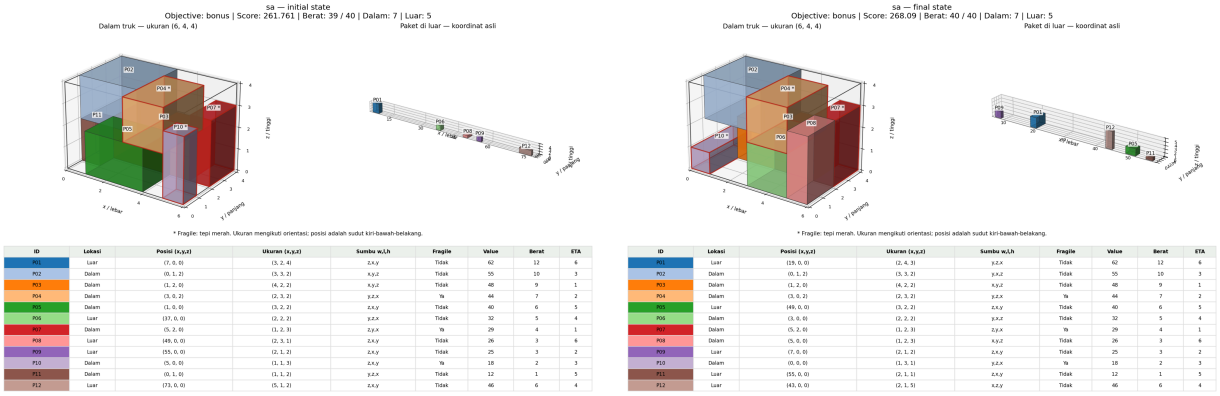
Run 3

Untuk run 3, berikut adalah hasil akhir yang didapatkan.

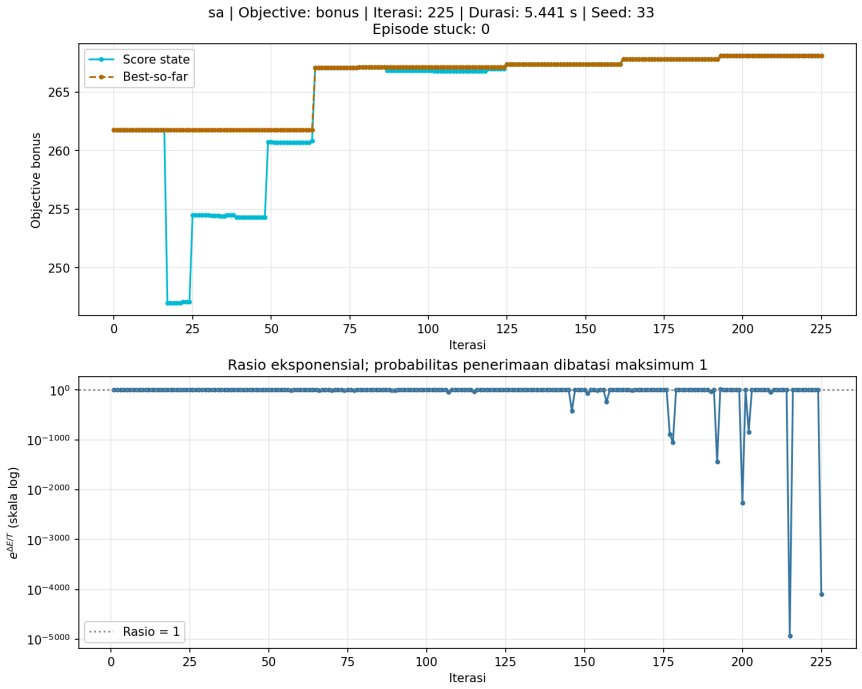
- Score Awal : 261.7611 (nilai kargo: 246, berat: 39/40)
- Score Akhir : 268.0897 (nilai kargo: 252, berat: 40/40)
- Score Terbaik : 268.0897 (nilai kargo: 252, berat: 40/40)
- Durasi Pencarian : 5.441 detik
- Statistik Transisi Langkah
    - Langkah perbaikan diterima : 12
    - Langkah sideways diterima : 177
    - Langkah penurunan diterima : 9
    - Langkah kandidat ditolak : 27

Tabel 2.7.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Simulated Annealing dengan Fungsi Objektif Bonus

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(3,2,4)	Luar	(19,0,0)	(2,4,3)
P02	Dalam	(0,1,2)	(3,3,2)	Dalam	(0,1,2)	(3,3,2)
P03	Dalam	(1,2,0)	(4,2,2)	Dalam	(1,2,0)	(4,2,2)
P04	Dalam	(3,0,2)	(2,3,2)	Dalam	(3,0,2)	(2,3,2)
P05	Dalam	(1,0,0)	(3,2,2)	Luar	(49,0,0)	(3,2,2)
P06	Luar	(37,0,0)	(2,2,2)	Dalam	(3,0,0)	(2,2,2)
P07	Dalam	(5,2,0)	(1,2,3)	Dalam	(5,2,0)	(1,2,3)
P08	Luar	(49,0,0)	(2,3,1)	Dalam	(5,0,0)	(1,2,3)
P09	Luar	(55,0,0)	(2,1,2)	Luar	(7,0,0)	(2,1,2)
P10	Dalam	(5,0,0)	(1,1,3)	Dalam	(0,0,0)	(1,3,1)
P11	Dalam	(0,1,0)	(1,2,1)	Luar	(55,0,0)	(2,1,1)
P12	Luar	(73,0,0)	(5,1,2)	Luar	(43,0,0)	(2,1,5)



Gambar 2.7.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Simulated Annealing dengan Fungsi Objektif Bonus



Gambar 2.7.6 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3 Simulated Annealing dengan Fungsi Objektif Bonus

Analisis Perbandingan Fungsi Objektif Wajib dan Bonus

Pada objective default, pergeseran posisi internal paket di dalam truk bernilai $\Delta E = 0$ sehingga SA terjebak dalam 2 hingga 7 episode stuck. Pada objective bonus, setiap pergeseran posisi ke arah pintu belakang (*y*) atau lantai (*z*) memberikan perbaikan kontinu $\Delta E > 0$ sehingga episode stuck turun menjadi tepat 0. Kemudian, karena tidak ada kondisi stagnan, pemeriksaan `_is_stuck` selalu menemukan tetangga yang lebih baik dengan sangat cepat, hal ini memangkas durasi dari rata-rata 33.46 s menjadi hanya 9.68 s dengan fungsi objektif bonus.

Bonus - Multitruk

Deskripsi Fungsi/Kelas

```
@dataclass(frozen=True)
class Problem:
    truck: Truck | None = None
    packages: tuple[Package, ...] = ()
    trucks: tuple[Truck, ...] = ()

    def __post_init__(self) -> None:
        if self.trucks:
            trucks = tuple(self.trucks)
        elif self.truck is not None:
            trucks = (self.truck,)
        else:
            raise ValueError("truck harus berupa Truck.")
        if any(not isinstance(item, Truck) for item in trucks):
            raise ValueError("trucks harus berisi Truck.")
        object.__setattr__(self, "trucks", trucks)
        object.__setattr__(self, "truck", trucks[0])
        object.__setattr__(self, "packages", tuple(self.packages))
        if any(not isinstance(package, Package) for package in self.packages):
            raise ValueError("packages harus berisi Package.")
        ids = [package.id for package in self.packages]
        if len(ids) != len(set(ids)):
            raise ValueError("ID paket harus unik.")

@dataclass(frozen=True)
class Placement:
    position: Position
    orientation: Orientation = ("x", "y", "z")
    truck: int = 0

    def __post_init__(self) -> None:
        if not isinstance(self.position, (tuple, list)) or len(self.position) != 3:
            raise ValueError("Posisi harus berisi tiga koordinat integer.")
        if any(type(coordinate) is not int for coordinate in self.position):
            raise ValueError("Koordinat harus berupa bilangan bulat.")
        if not isinstance(self.orientation, (tuple, list)):
            raise ValueError("Orientasi harus berupa pemetaan sumbu untuk w, l, h.")
        if type(self.truck) is not int or self.truck < 0:
            raise ValueError("Indeks truk harus berupa bilangan bulat nonnegatif.")
        object.__setattr__(self, "position", tuple(self.position))
        object.__setattr__(self, "orientation", tuple(self.orientation))
        if self.orientation not in ORIENTATIONS:
            raise ValueError("Orientasi harus berupa permutasi x, y, z.")
```

Kelas data immutable `Problem` dan `Placement` memperluas representasi persoalan dari satu truk menjadi armada multi-truk tanpa merusak kompatibilitas kode satu truk. Kelas `Problem` menyimpan koleksi armada pada atribut tuple `trucks` sekaligus menjaga atribut `truck` merujuk pada truk pertama (`trucks[0]`), sedangkan kelas `Placement` menambahkan atribut indeks truck (bernilai bilangan bulat nonnegatif, default `0`) untuk mencatat di truk mana suatu paket ditempatkan beserta koordinat (x, y, z) dan orientasinya.

```
def is_inside(problem: Problem, package: Package, placement: Placement) -> bool:
    if not 0 <= placement.truck < len(problem.trucks):
        return False
    dimensions = problem.trucks[placement.truck].dimensions.as_tuple()
    return _inside(_bounds(package, placement), dimensions)


def validate_state(problem: Problem, state: State) -> list[str]:
    errors = []
    packages = {package.id: package for package in problem.packages}
    missing = packages.keys() - state.placements.keys()
    extra = state.placements.keys() - packages.keys()
    if missing:
        errors.append(f"Paket tanpa placement: {', '.join(sorted(missing))}.")
    if extra:
        errors.append(f"ID paket tidak dikenal: {', '.join(sorted(extra))}.")
    if errors:
        return errors

    num_trucks = len(problem.trucks)
    for key, placement in state.placements.items():
        if not 0 <= placement.truck < num_trucks:
            errors.append(f"Indeks truk untuk paket {key} tidak valid: {placement.truck}.")
    if errors:
        return errors

    boxes = {key: _bounds(package, state.placements[key]) for key, package in
packages.items()}
    loaded_by_truck: list[list[str]] = [[] for _ in range(num_trucks)]
    outside: list[str] = []

    for key, box in boxes.items():
        truck_idx = state.placements[key].truck
        dimensions = problem.trucks[truck_idx].dimensions.as_tuple()
        if _inside(box, dimensions):
            loaded_by_truck[truck_idx].append(key)
        elif not any(box[axis + 3] <= 0 or box[axis] >= dimensions[axis] for axis in
range(3)):
            errors.append(f"Paket {key} menembus dinding truk.")
        else:
            outside.append(key)

    for truck_loaded in loaded_by_truck:
        for first, second in combinations(truck_loaded, 2):
            if _overlap(boxes[first], boxes[second], (0, 1, 2)):
                errors.append(f"Paket {first} dan {second} overlap.")

    for first, second in combinations(outside, 2):
        if _overlap(boxes[first], boxes[second], (0, 1, 2)):
            errors.append(f"Paket {first} dan {second} overlap.")

    for truck_idx, truck_loaded in enumerate(loaded_by_truck):
        for first in truck_loaded:
            for second in outside:
                if state.placements[second].truck == truck_idx and _overlap(boxes[first],
boxes[second], (0, 1, 2)):
```

```
                    errors.append(f"Paket {first} dan {second} overlap.")

    for truck_idx, (truck, truck_loaded) in enumerate(zip(problem.trucks, loaded_by_truck)):
        total_weight = _weight_sum(packages[key] for key in truck_loaded)
        if total_weight > Decimal(str(truck.max_capacity)):
            if num_trucks == 1:
                errors.append("Total berat melebihi kapasitas truk.")
            else:
                errors.append(f"Total berat pada truk {truck_idx + 1} melebihi kapasitas
truk.")

        for key in truck_loaded:
            box = boxes[key]
            if box[2] == 0:
                continue
            supports = [
                other for other in truck_loaded
                if other != key and boxes[other][5] == box[2]
                and _overlap(box, boxes[other], (0, 1))
            ]
            if not supports:
                errors.append(f"Paket {key} melayang.")
            for other in supports:
                if packages[other].is_fragile:
                    errors.append(f"Paket fragile {other} menjadi support untuk {key}.")
    return errors
```

Fungsi `is_inside` dan `validate_state` menangani pemeriksaan fisik untuk skenario satu maupun banyak truk. `is_inside` memeriksa apakah batas suatu paket berada sepenuhnya di dalam dimensi kontainer truk sesuai indeks `placement.truck`. Selanjutnya, `validate_state` mengelompokkan paket yang dimuat per truk ke dalam loaded_by_truck dan memastikan bahwa setiap truk mematuhi batasan secara independen, yaitu tidak ada paket yang menembus dinding truk, tidak ada tumpang-tindih antarpaket di dalam truk yang sama maupun di slot luar, total berat muatan masing-masing truk tidak melampaui `max_capacity` truk tersebut, tidak ada paket yang melayang, serta tidak ada paket *fragile* yang menopang paket lain di atasnya.

```
def outside_slots(problem: Problem) -> tuple[Position, ...]:
    spacing = 1 + max(
        (side for package in problem.packages for side in package.dimensions.as_tuple()),
        default=0,
    )
    max_width = max((truck.dimensions.w for truck in problem.trucks), default=0)
    return tuple(
        (max_width + 1 + index * spacing, 0, 0)
        for index in range(len(problem.packages))
    )


def move_package(
    state: State, package_id: str, position: Position, truck: int | None = None,
) -> State:
```

```
    placements = dict(state.placements)
    current = placements[package_id]
    target_truck = current.truck if truck is None else truck
    placements[package_id] = replace(current, position=position, truck=target_truck)
    return State(placements)


def swap_packages(state: State, first_id: str, second_id: str) -> State:
    placements = dict(state.placements)
    first, second = placements[first_id], placements[second_id]
    placements[first_id] = replace(first, position=second.position, truck=second.truck)
    placements[second_id] = replace(second, position=first.position, truck=first.truck)
    return State(placements)


def iter_neighbors(problem: Problem, state: State) -> Iterator[State]:
    _require_valid(problem, state)
    slots = outside_slots(problem)
    for package in problem.packages:
        current_placement = state.placements[package.id]
        for truck_idx, truck in enumerate(problem.trucks):
            w, l, h = truck.dimensions.as_tuple()
            for position in product(range(w), range(l), range(h)):
                if position != current_placement.position or truck_idx !=
current_placement.truck:
                    candidate = move_package(state, package.id, position, truck=truck_idx)
                    if not validate_state(problem, candidate):
                        yield candidate
        for position in slots:
            if position != current_placement.position or current_placement.truck != 0:
                candidate = move_package(state, package.id, position, truck=0)
                if not validate_state(problem, candidate):
                    yield candidate
        for axis in ("x", "y", "z"):
            candidate = rotate_package(state, package.id, axis)
            if not validate_state(problem, candidate):
                yield candidate
    for first, second in combinations(problem.packages, 2):
        candidate = swap_packages(state, first.id, second.id)
        if not validate_state(problem, candidate):
            yield candidate


def sample_neighbor(
    problem: Problem, state: State, rng: Random, max_attempts: int = 100,
) -> State | None:
    _positive_attempts(max_attempts)
    _require_valid(problem, state)
    if not problem.packages:
        return None
    truck_dims = [truck.dimensions.as_tuple() for truck in problem.trucks]
    truck_volumes = [w * l * h for w, l, h in truck_dims]
    total_volume = sum(truck_volumes)
    slots = outside_slots(problem)
    moves = ("move", "rotate", "swap") if len(problem.packages) > 1 else ("move", "rotate")
    for _ in range(max_attempts):
        package = rng.choice(problem.packages)
        operation = rng.choice(moves)
        if operation == "move":
            index = rng.randrange(total_volume + len(slots))
```

```
            if index >= total_volume:
                position = slots[index - total_volume]
                candidate = move_package(state, package.id, position, truck=0)
            else:
                rem = index
                target_truck = 0
                for truck_idx, vol in enumerate(truck_volumes):
                    if rem < vol:
                        target_truck = truck_idx
                        break
                    rem -= vol
                w, l, h = truck_dims[target_truck]
                position = (rem // (l * h), (rem // h) % l, rem % h)
                candidate = move_package(state, package.id, position, truck=target_truck)
        elif operation == "rotate":
            candidate = rotate_package(state, package.id, rng.choice(("x", "y", "z")))
        else:
            first, second = rng.sample(problem.packages, 2)
            candidate = swap_packages(state, first.id, second.id)
        if candidate != state and not validate_state(problem, candidate):
            return candidate
    return None
```

Kelima fungsi pada `src/neighbors.py` ini mendefinisikan transisi untuk perpindahan paket di dalam truk maupun lintas truk. Fungsi `outside_slots` menyediakan koordinat penempatan di luar kontainer yang aman dari lebar maksimum seluruh truk dalam armada. Operator `move_package` memindahkan satu paket ke koordinat dan indeks truk tujuan baru (`target_truck`), kemudian `swap_packages` menukar posisi sekaligus indeks truk dari dua paket yang berbeda. Selanjutnya, `iter_neighbors` membangkitkan seluruh kandidat tetangga yang valid secara sistematis dengan mengiterasi seluruh sel (x, y, z) pada setiap truk di armada, slot luar, rotasi sumbu, dan pertukaran pasang paket, sementara `sample_neighbor` melakukan pencuplikan satu tetangga valid secara acak dengan memetakan indeks volume gabungan seluruh armada (`total_volume`) ke truk target dan koordinat tiga dimensinya.

```
def total_capacity(problem: Problem) -> int | float:
    total = sum((Decimal(str(truck.max_capacity)) for truck in problem.trucks), Decimal(0))
    return int(total) if total == total.to_integral_value() else float(total)


def loaded_weight_for_truck(problem: Problem, state: State, truck_index: int) -> int |
float:
    _require_valid(problem, state)
    if not 0 <= truck_index < len(problem.trucks):
        raise ValueError("Indeks truk tidak valid.")
    total = _weight_sum(
        package for package in problem.packages
        if state.placements[package.id].truck == truck_index
        and is_inside(problem, package, state.placements[package.id])
    )
```

```
    return int(total) if total == total.to_integral_value() else float(total)


def build_occupancy(
    problem: Problem, state: State, *, truck_index: int = 0,
) -> dict[Position, str | None]:
    _require_valid(problem, state)
    if not 0 <= truck_index < len(problem.trucks):
        raise ValueError("Indeks truk tidak valid.")
    w, l, h = problem.trucks[truck_index].dimensions.as_tuple()
    occupancy = dict.fromkeys(product(range(w), range(l), range(h)))
    for package in problem.packages:
        placement = state.placements[package.id]
        if placement.truck == truck_index and is_inside(problem, package, placement):
            x, y, z, end_x, end_y, end_z = _bounds(package, placement)
            for cell in product(range(x, end_x), range(y, end_y), range(z, end_z)):
                occupancy[cell] = package.id
    return occupancy


def generate_initial_state(problem: Problem, rng: Random, max_attempts: int = 100) -> State:
    _positive_attempts(max_attempts)
    state = State({
        package.id: Placement(position, rng.choice(ORIENTATIONS), truck=0)
        for package, position in zip(problem.packages, outside_slots(problem))
    })
    order = list(problem.packages)
    rng.shuffle(order)
    num_trucks = len(problem.trucks)
    truck_limits = [truck.dimensions.as_tuple() for truck in problem.trucks]
    weights = [Decimal(0) for _ in problem.trucks]
    capacities = [Decimal(str(truck.max_capacity)) for truck in problem.trucks]

    for package in order:
        package_weight = Decimal(str(package.weight))
        eligible_trucks = []
        for truck_idx, limits in enumerate(truck_limits):
            if weights[truck_idx] + package_weight > capacities[truck_idx]:
                continue
            orientations = [
                orientation for orientation in ORIENTATIONS
                if all(size <= limit for size, limit in zip(oriented_dimensions(package,
orientation), limits))
            ]
            if orientations:
                eligible_trucks.append((truck_idx, orientations, limits))
        if not eligible_trucks:
            continue
        for _ in range(max_attempts):
            if num_trucks == 1:
                truck_idx, orientations, limits = eligible_trucks[0]
            else:
                truck_idx, orientations, limits = rng.choice(eligible_trucks)
            orientation = rng.choice(orientations)
            dimensions = oriented_dimensions(package, orientation)
            position = tuple(rng.randrange(limit - size + 1) for size, limit in
zip(dimensions, limits))
            placements = dict(state.placements)
            placements[package.id] = Placement(position, orientation, truck=truck_idx)
            candidate = State(placements)
```

```
            if not validate_state(problem, candidate):
                state = candidate
                weights[truck_idx] += package_weight
                break
    return state
```

Fungsi-fungsi pada `src/problem.py` ini mengelola kapasitas beberapa truk dan pembuatan solusi awal multitruk. Fungsi `total_capacity` menjumlahkan kapasitas berat maksimum dari seluruh truk yang ada, `loaded_weight_for_truck` menghitung total beban kargo yang termuat di satu truk spesifik (`truck_index`), dan `build_occupancy` membangun peta grid 3D sel demi sel untuk truk yang dipilih. Adapun `generate_initial_state` membangkitkan state awal yang valid secara acak dengan menempatkan seluruh paket di slot luar terlebih dahulu, mengacak urutan paket, menyaring daftar truk yang masih memiliki sisa kapasitas berat serta ukuran dimensi yang mencukupi (`eligible_trucks`), lalu mencoba memasukkan paket ke salah satu truk yang memenuhi syarat secara acak hingga ditemukan penempatan yang lolos `validate_state`.

```
def _truck_from_dict(raw: object) -> tuple[Truck, int]:
    if not isinstance(raw, dict):
        raise ValueError("truck harus berupa object JSON.")
    allowed = {"dimensions", "max_capacity"}
    count = 1
    if "count" in raw:
        allowed = {"dimensions", "max_capacity", "count"}
        count = raw["count"]
        if type(count) is not int or count <= 0:
            raise ValueError("count truk harus berupa bilangan bulat positif.")
    item = _object(raw, allowed, "truck")
    return Truck(_dimensions(item["dimensions"]), item["max_capacity"]), count


def problem_from_dict(data: dict) -> Problem:
    if not isinstance(data, dict):
        raise ValueError("input harus berupa object JSON.")
    keys = set(data)
    if not keys <= {"truck", "trucks", "packages"} or "packages" not in keys or not (keys &
{"truck", "trucks"}):
        raise ValueError("Field input harus memuat packages serta truck atau trucks.")
    trucks: list[Truck] = []
    if "trucks" in data:
        if not isinstance(data["trucks"], list) or not data["trucks"]:
            raise ValueError("trucks harus berupa array JSON yang tidak kosong.")
        for raw_truck in data["trucks"]:
            truck_obj, count = _truck_from_dict(raw_truck)
            trucks.extend([truck_obj] * count)
    elif "truck" in data:
        truck_obj, count = _truck_from_dict(data["truck"])
        trucks.extend([truck_obj] * count)
```

```
    if not isinstance(data["packages"], list):
        raise ValueError("packages harus berupa array JSON.")
    packages = []
    for raw in data["packages"]:
        item = _object(raw, {"id", "dimensions", "value", "weight", "is_fragile", "eta"},
"package")
        packages.append(Package(
            id=item["id"],
            dimensions=_dimensions(item["dimensions"]),
            value=item["value"],
            weight=item["weight"],
            is_fragile=item["is_fragile"],
            eta=item["eta"],
        ))
    return Problem(
        truck=trucks[0],
        packages=tuple(packages),
        trucks=tuple(trucks),
    )


def with_trucks(problem: Problem, num_trucks: int | None) -> Problem:
    if num_trucks is None:
        return problem
    if type(num_trucks) is not int or num_trucks <= 0:
        raise ValueError("Jumlah truk harus berupa bilangan bulat positif.")
    if num_trucks == len(problem.trucks):
        return problem
    if len(problem.trucks) == 1:
        trucks = (problem.truck,) * num_trucks
    else:
        trucks = tuple(problem.trucks[i % len(problem.trucks)] for i in range(num_trucks))
    return Problem(truck=trucks[0], packages=problem.packages, trucks=trucks)


def load_problem(path: str | Path, *, trucks: int | None = None) -> Problem:
    with Path(path).open(encoding="utf-8") as handle:
        problem = problem_from_dict(json.load(handle))
    return with_trucks(problem, trucks)
```

Rangkaian fungsi pada `src/experiments.py` ini menangani pembacaan dataset JSON dan penyesuaian jumlah armada truk dari CLI (`--trucks`). Fungsi `_truck_from_dict` dan `problem_from_dict` mendukung pembacaan spesifikasi truk tunggal (`truck`), properti `count` untuk menggandakan truk sejenis, maupun daftar truk yang berbeda (`trucks`). Selanjutnya, fungsi `with_trucks` dan `load_problem` memungkinkan pengguna mengubah jumlah truk secara fleksibel saat eksekusi eksperimen dengan mereplikasi spesifikasi truk dari file input sebanyak `num_trucks` yang diminta.

```
def plot_state(
    problem: Problem, state: State, title: str = "State paket", *, objective: str | None =
```

```
None,
) -> Figure:
    chosen = get_objective() if objective is None else objective
    value = score(problem, state, objective=chosen)
    weight = loaded_weight(problem, state)
    fleet_capacity = total_capacity(problem)
    packages = problem.packages
    num_trucks = len(problem.trucks)
    colors = {package.id: plt.get_cmap("tab20")(index % 20) for index, package in
enumerate(packages)}
    inside = [package for package in packages if is_inside(problem, package,
state.placements[package.id])]
    outside = [package for package in packages if package not in inside]

    num_panels = num_trucks + 1
    table_height = max(1.3, 0.3 * (len(packages) + 1))
    fig_width = max(15, 6 * num_panels)
    figure = plt.figure(figsize=(fig_width, 6 + table_height), layout="constrained")
    grid = figure.add_gridspec(2, num_panels, height_ratios=(5, table_height))
    table_ax = figure.add_subplot(grid[1, :])

    fleet_info = f" | Truk: {num_trucks}" if num_trucks > 1 else ""
    figure.suptitle(
        f"{title}\nObjective: {chosen} | Score: {value:g} | Berat: {weight:g} /
{fleet_capacity:g}"
        f"{fleet_info} | Dalam: {len(inside)} | Luar: {len(outside)}",
        fontsize=14,
    )

    for truck_idx, truck in enumerate(problem.trucks):
        truck_ax = figure.add_subplot(grid[0, truck_idx], projection="3d")
        dimensions = truck.dimensions.as_tuple()
        _configure_axes(truck_ax, (0, 0, 0), dimensions)
        _draw_truck(truck_ax, dimensions)
        if num_trucks == 1:
            truck_ax.set_title(f"Dalam truk — ukuran {dimensions}")
        else:
            t_weight = loaded_weight_for_truck(problem, state, truck_idx)
            truck_ax.set_title(
                f"Truk {truck_idx + 1} — ukuran {dimensions}\nBerat: {t_weight:g} /
{truck.max_capacity:g}"
            )
        for package in inside:
            if state.placements[package.id].truck == truck_idx:
                _draw_package(truck_ax, package, state.placements[package.id],
colors[package.id])
```

Fungsi `plot_state` membuat visualisasi tiga dimensi dari penataan paket untuk satu maupun banyak truk beserta tabel rincian penempatannya. Ketika `num_trucks > 1`, fungsi ini secara otomatis menyesuaikan lebar kanvas gambar (`fig_width`) dan membuat panel subplot 3D terpisah untuk masing-masing truk berdampingan dengan panel paket di luar truk, lengkap dengan informasi beban muatan spesifik tiap truk (`loaded_weight_for_truck`), dan label kolom lokasi truk tertentu pada tabel ringkasan paket di bagian bawah gambar.

Jumlah truk ditambahkan menjadi 2 truk dan 3 truk pada bonus kali ini. Operator untuk *neighbor* atau tetangga akan mencakup operasi perpindahan paket antartruk dan penukaran posisi paket antartruk. Validator akan memeriksa batas dimensi, kapasitas muatan, dan aturan tumpuan/fragile secara independen per truk

Tabel 2.8.1 Hasil Eksperimen Bonus Multitruk dengan Algoritma Simulated Annealing untuk 2 Truk

Run (Seed)	Score Awal	Score Akhir	Score Terbaik	Muatan Truk 1	Muatan Truk 2	Total Berat	Iterasi	Durasi Pencarian (s)	Episode Stuck	Iterasi Stuck
1 (11)	334	380	380	32/40	28/40	60/80	225	15.109	1	5
2 (22)	291	357	357	24/40	30/40	54/80	225	23.895	0	0
3 (33)	405	411	411	34/40	31/40	65/80	225	23.628	3	30
Rata-rata	343.33	382.67	382.67				225	20.877	1.33	11.67

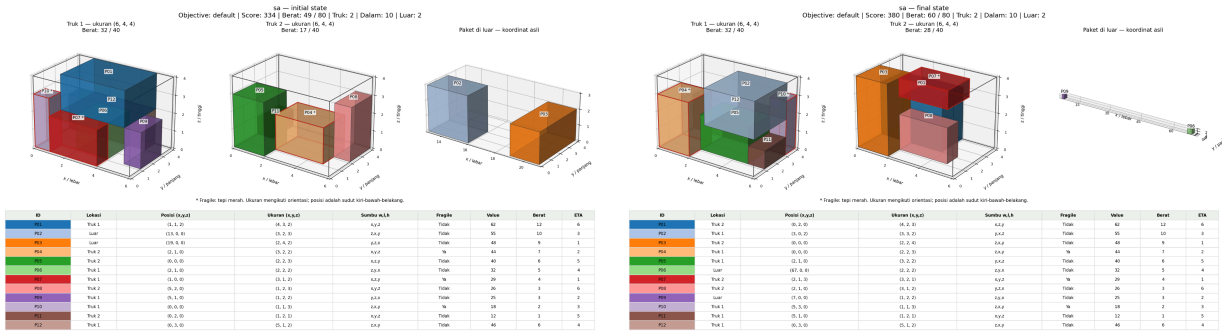
Tabel 2.8.2 Hasil Eksperimen Bonus Multitruk dengan Algoritma Simulated Annealing untuk 3 Truk

Run (Seed)	Score Awal	Score Akhir	Score Terbaik	Muatan Truk 1	Muatan Truk 2	Muatan Truk 3	Total Berat	Iterasi	Durasi Pencarian (s)	Episode Stuck	Iterasi Stuck
1 (11)	437	437	437	24/40	25/40	19/40	68/120	225	52.380	1	225
2 (22)	437	437	437	28/40	21/40	19/40	68/120	225	44.122	2	169
3 (33)	437	437	437	32/40	25/40	11/40	68/120	225	43.713	2	91
Rata-rata	437	437	437					225	46.738	1.67	161.67

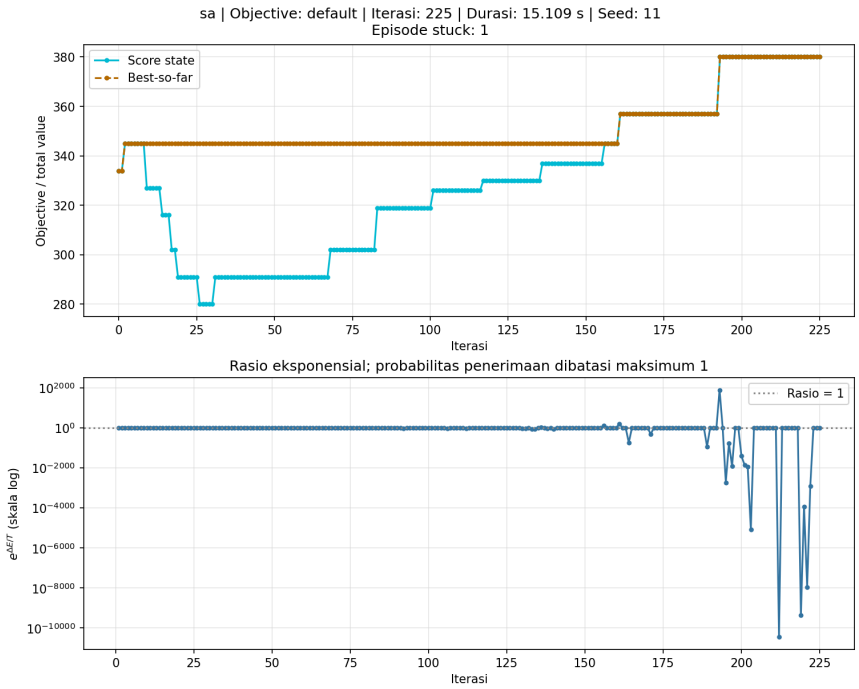
Run 1 – 2 Truk

Tabel 2.8.3 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Simulated Annealing dengan 2 Truk

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Truk 1	(1,1,2)	(4,3,2)	Truk 2	(0,2,0)	(4,2,3)
P02	Luar	(13,0,0)	(3,2,3)	Truk 1	(3,0,2)	(3,3,2)
P03	Luar	(19,0,0)	(2,4,2)	Truk 2	(0,0,0)	(2,2,4)
P04	Truk 2	(2,1,0)	(3,2,2)	Truk 1	(0,0,0)	(2,2,3)
P05	Truk 2	(0,0,0)	(2,2,3)	Truk 1	(2,1,0)	(3,2,2)
P06	Truk 1	(2,1,0)	(2,2,2)	Luar	(67,0,0)	(2,2,2)
P07	Truk 1	(1,0,0)	(3,1,2)	Truk 2	(2,1,3)	(3,2,1)
P08	Truk 2	(5,2,0)	(1,2,3)	Truk 2	(2,1,0)	(3,1,2)
P09	Truk 1	(5,1,0)	(1,2,2)	Luar	(7,0,0)	(1,2,2)
P10	Truk 1	(0,0,0)	(1,1,3)	Truk 1	(5,3,0)	(1,1,3)
P11	Truk 2	(0,2,0)	(1,2,1)	Truk 1	(5,1,0)	(1,2,1)
P12	Truk 1	(0,3,0)	(5,1,2)	Truk 1	(0,3,0)	(5,1,2)



Gambar 2.8.1 Visualisasi Konfigurasi Awal dan Akhir Run 1 Simulated Annealing dengan 2 Truk

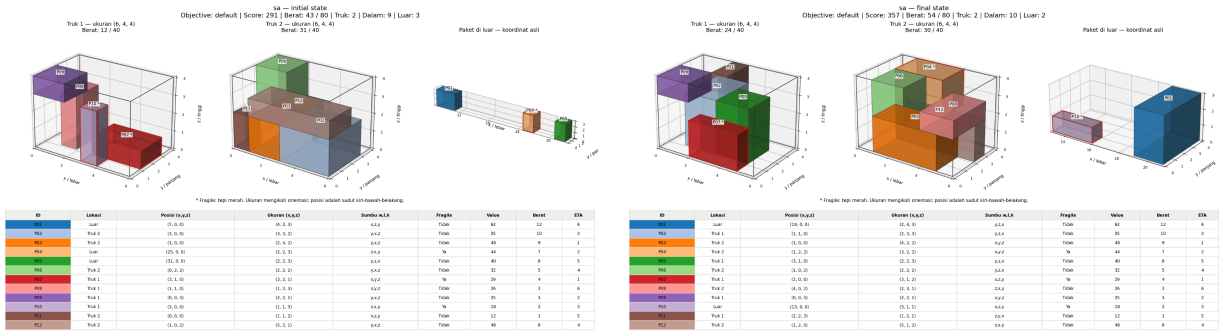


Gambar 2.8.2 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1 Simulated Annealing dengan 2 Truk

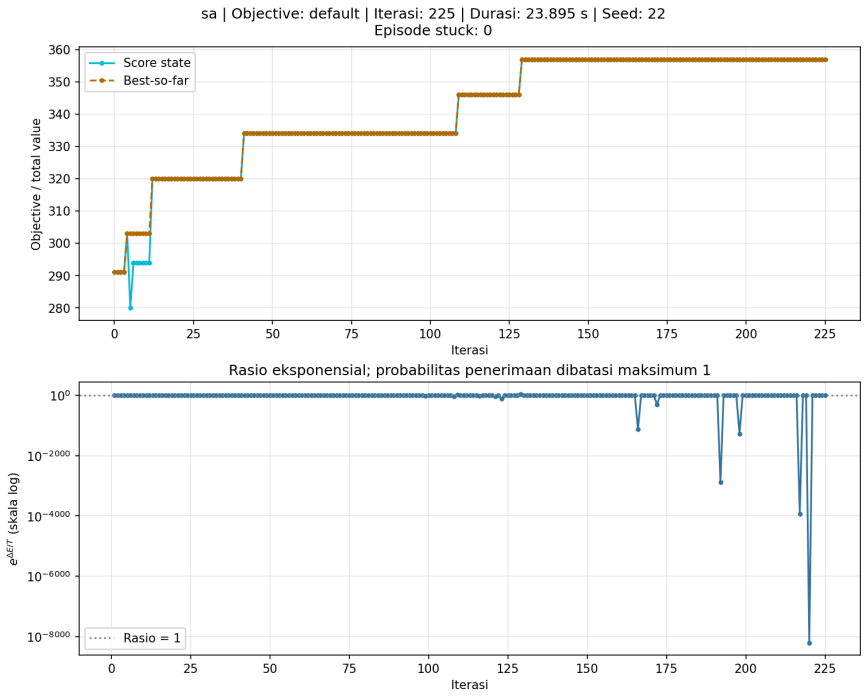
Run 2 – 2 Truk

Tabel 2.8.4 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Simulated Annealing dengan 2 Truk

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Luar	(7,0,0)	(4,2,3)	Luar	(19,0,0)	(2,4,3)
P02	Truk 2	(3,0,0)	(3,3,2)	Truk 1	(1,1,0)	(2,3,3)
P03	Truk 2	(1,0,0)	(2,4,2)	Truk 2	(1,0,0)	(4,2,2)
P04	Luar	(25,0,0)	(2,2,3)	Truk 2	(1,2,2)	(3,2,2)
P05	Luar	(31,0,0)	(2,2,3)	Truk 1	(3,1,0)	(2,2,3)
P06	Truk 2	(0,2,2)	(2,2,2)	Truk 2	(1,0,2)	(2,2,2)
P07	Truk 1	(3,1,0)	(3,2,1)	Truk 1	(2,0,0)	(3,1,2)
P08	Truk 1	(1,1,0)	(1,2,3)	Truk 2	(4,0,2)	(2,3,1)
P09	Truk 1	(0,0,3)	(2,2,1)	Truk 1	(0,0,3)	(2,2,1)
P10	Truk 1	(3,0,0)	(1,1,3)	Luar	(13,0,0)	(3,1,1)
P11	Truk 2	(0,0,0)	(1,1,2)	Truk 1	(2,2,3)	(1,2,1)
P12	Truk 2	(1,0,2)	(5,2,1)	Truk 2	(1,2,0)	(5,1,2)



Gambar 2.8.3 Visualisasi Konfigurasi Awal dan Akhir Run 2 Simulated Annealing dengan 2 Truk

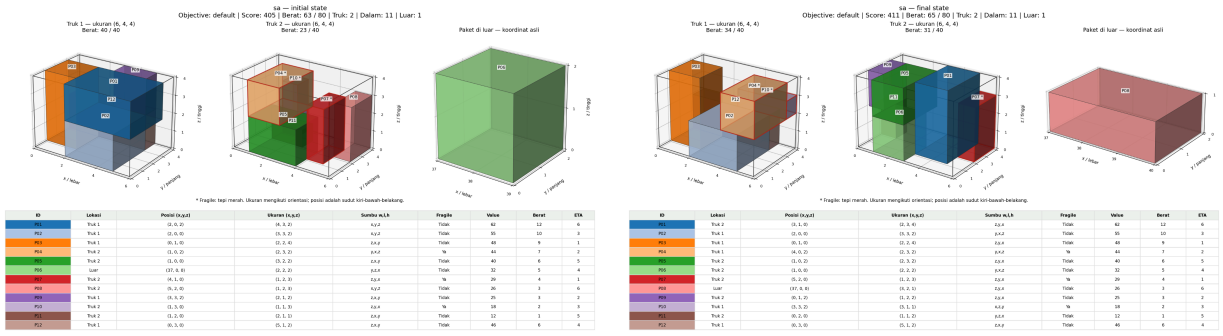


Gambar 2.8.4 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2 Simulated Annealing dengan 2 Truk

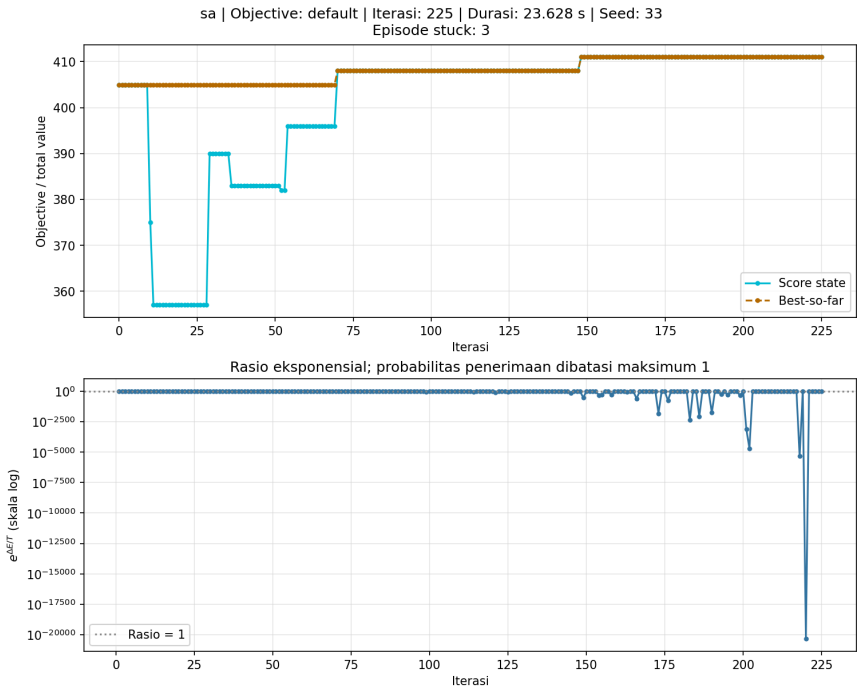
Run 3 – 2 Truk

Tabel 2.8.5 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 3 Simulated Annealing dengan 2 Truk

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Truk 1	(2,0,2)	(4,3,2)	Truk 2	(3,1,0)	(2,3,4)
P02	Truk 1	(2,0,0)	(3,3,2)	Truk 1	(2,0,0)	(3,3,2)
P03	Truk 1	(0,1,0)	(2,2,4)	Truk 1	(0,1,0)	(2,2,4)
P04	Truk 2	(1,0,2)	(2,3,2)	Truk 1	(4,0,2)	(2,3,2)
P05	Truk 2	(1,0,0)	(3,2,2)	Truk 2	(1,0,2)	(2,3,2)
P06	Luar	(37,0,0)	(2,2,2)	Truk 2	(1,0,0)	(2,2,2)
P07	Truk 2	(4,1,0)	(1,2,3)	Truk 2	(5,2,0)	(1,2,3)
P08	Truk 2	(5,2,0)	(1,2,3)	Luar	(37,0,0)	(3,2,1)
P09	Truk 1	(3,3,2)	(2,1,2)	Truk 2	(0,1,2)	(1,2,2)
P10	Truk 2	(1,3,0)	(1,1,3)	Truk 1	(3,3,2)	(3,1,1)
P11	Truk 2	(1,2,0)	(2,1,1)	Truk 2	(0,2,0)	(1,1,2)
P12	Truk 1	(0,3,0)	(5,1,2)	Truk 1	(0,3,0)	(5,1,2)



Gambar 2.8.5 Visualisasi Konfigurasi Awal dan Akhir Run 3 Simulated Annealing dengan 2 Truk

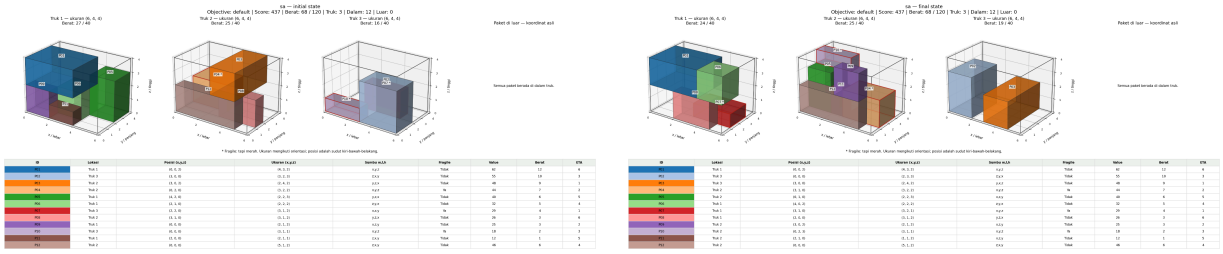


Gambar 2.8.6 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3 Simulated Annealing dengan 2 Truk

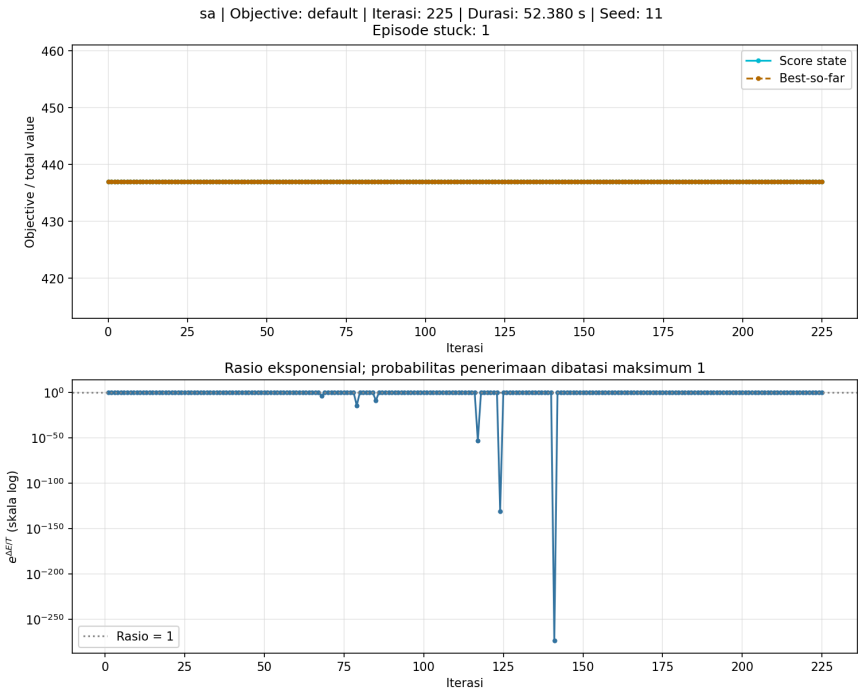
Run 1 – 3 Truk

Tabel 2.8.6 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 1 Simulated Annealing dengan 3 Truk

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Truk 1	(0,0,2)	(4,3,2)	Truk 1	(0,0,2)	(4,3,2)
P02	Truk 3	(3,0,0)	(3,2,3)	Truk 3	(0,0,0)	(2,3,3)
P03	Truk 2	(3,0,2)	(2,4,2)	Truk 3	(3,0,0)	(2,4,2)
P04	Truk 2	(0,2,0)	(3,2,2)	Truk 2	(3,1,0)	(3,2,2)
P05	Truk 1	(4,2,0)	(2,2,3)	Truk 2	(0,1,0)	(2,2,3)
P06	Truk 1	(2,1,0)	(2,2,2)	Truk 1	(4,0,2)	(2,2,2)
P07	Truk 3	(2,2,0)	(3,1,2)	Truk 1	(3,1,0)	(3,2,1)
P08	Truk 2	(3,1,0)	(3,1,2)	Truk 1	(2,0,0)	(3,1,2)
P09	Truk 1	(0,0,0)	(2,1,2)	Truk 2	(3,0,2)	(2,1,2)
P10	Truk 3	(0,0,0)	(3,1,1)	Truk 2	(0,2,3)	(3,1,1)
P11	Truk 1	(2,0,0)	(2,1,1)	Truk 2	(2,1,0)	(1,1,2)
P12	Truk 2	(0,0,0)	(5,1,2)	Truk 2	(0,0,0)	(5,1,2)



Gambar 2.8.7 Visualisasi Konfigurasi Awal dan Akhir Run 1 Simulated Annealing dengan 3 Truk

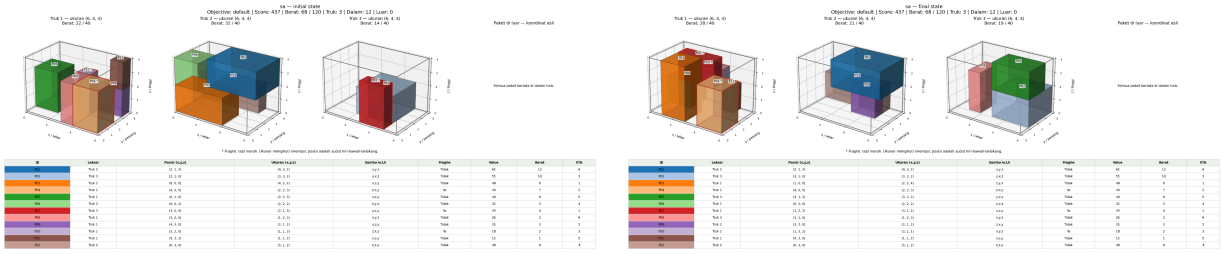


Gambar 2.8.8 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 1 Simulated Annealing dengan 2 Truk

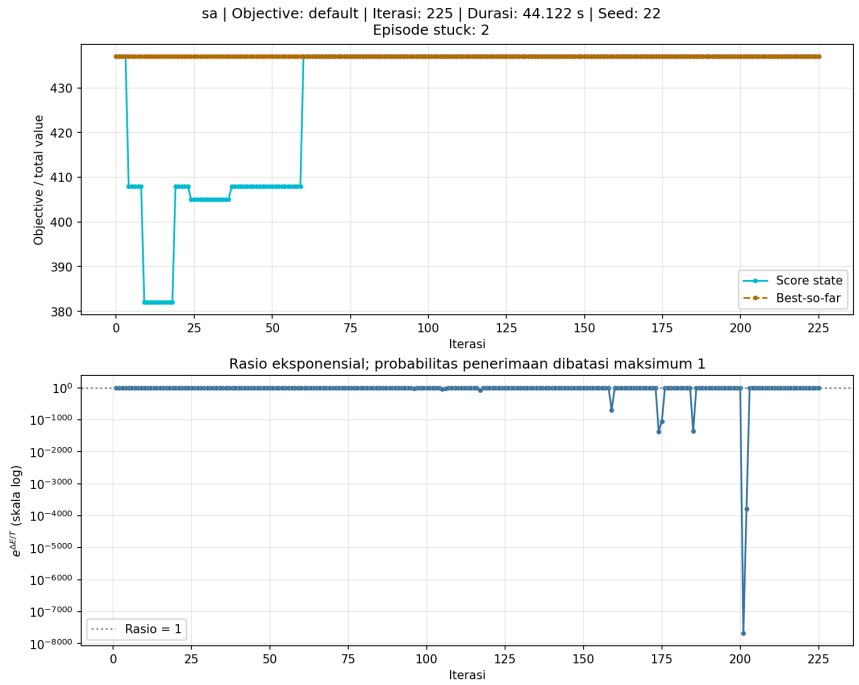
Run 2 – 3 Truk

Tabel 2.8.7 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Simulated Annealing dengan 3 Truk

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Truk 2	(2,1,2)	(4,3,2)	Truk 2	(2,1,2)	(4,3,2)
P02	Truk 3	(2,1,0)	(3,2,3)	Truk 3	(3,1,0)	(3,3,2)
P03	Truk 2	(0,0,0)	(2,4,2)	Truk 1	(1,0,0)	(2,2,4)
P04	Truk 1	(4,0,0)	(3,2,2)	Truk 1	(4,0,0)	(2,2,3)
P05	Truk 1	(0,1,0)	(2,2,3)	Truk 3	(3,1,2)	(3,2,2)
P06	Truk 2	(0,0,2)	(2,2,2)	Truk 1	(0,2,0)	(2,2,2)
P07	Truk 3	(3,0,0)	(3,1,2)	Truk 1	(1,2,2)	(3,1,2)
P08	Truk 1	(3,0,0)	(3,1,2)	Truk 3	(1,1,0)	(1,2,3)
P09	Truk 1	(4,3,0)	(2,1,2)	Truk 2	(3,2,0)	(2,1,2)
P10	Truk 1	(3,2,0)	(3,1,1)	Truk 1	(1,3,2)	(3,1,1)
P11	Truk 1	(5,3,2)	(2,1,1)	Truk 1	(4,3,0)	(1,1,2)
P12	Truk 2	(0,3,0)	(5,1,2)	Truk 2	(0,3,0)	(5,1,2)



Gambar 2.8.9 Visualisasi Konfigurasi Awal dan Akhir Run 2 Simulated Annealing dengan 3 Truk

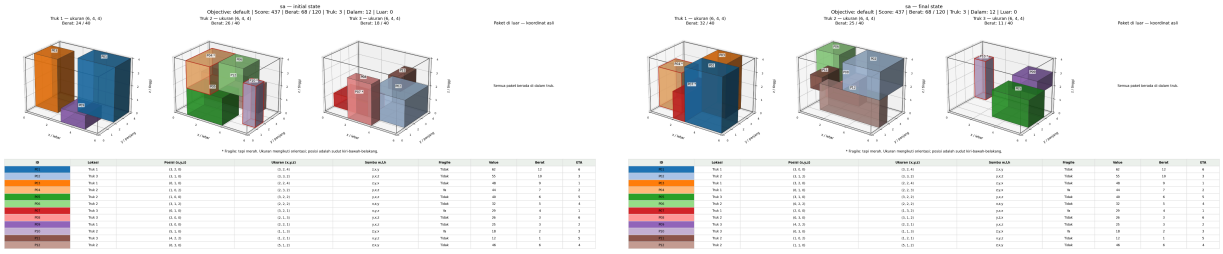


Gambar 2.8.10 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 2 Simulated Annealing dengan 3 Truk

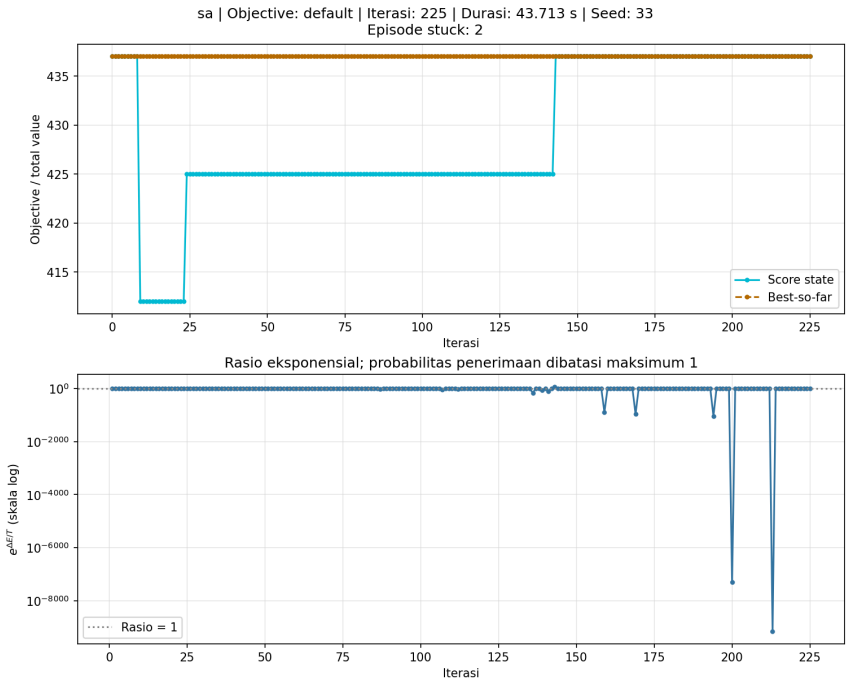
Run 3 – 3 Truk

Tabel 2.8.8 Perbandingan Konfigurasi Awal dan Konfigurasi Akhir Run 2 Simulated Annealing dengan 3 Truk

ID Paket	Awal			Akhir		
	State	Posisi	Orientasi	State	Posisi	Orientasi
P01	Truk 1	(3,2,0)	(3,2,4)	Truk 1	(3,0,0)	(3,2,4)
P02	Truk 3	(3,1,0)	(3,3,2)	Truk 2	(3,1,2)	(3,3,2)
P03	Truk 1	(0,1,0)	(2,2,4)	Truk 1	(3,2,0)	(2,2,4)
P04	Truk 2	(1,0,2)	(2,3,2)	Truk 1	(0,1,0)	(2,2,3)
P05	Truk 2	(1,0,0)	(3,2,2)	Truk 3	(3,1,0)	(3,2,2)
P06	Truk 2	(3,1,2)	(2,2,2)	Truk 2	(0,2,2)	(2,2,2)
P07	Truk 3	(0,1,0)	(3,2,1)	Truk 1	(2,0,0)	(1,3,2)
P08	Truk 3	(2,0,0)	(2,1,3)	Truk 2	(0,3,0)	(3,1,2)
P09	Truk 1	(3,0,0)	(2,2,1)	Truk 3	(4,2,2)	(2,2,1)
P10	Truk 2	(5,1,0)	(1,1,3)	Truk 3	(0,3,0)	(1,1,3)
P11	Truk 3	(4,2,2)	(1,2,1)	Truk 3	(1,0,2)	(1,2,1)
P12	Truk 2	(0,3,0)	(5,1,2)	Truk 2	(1,1,0)	(5,1,2)



Gambar 2.8.11 Visualisasi Konfigurasi Awal dan Akhir Run 3 Simulated Annealing dengan 3 Truk



Gambar 2.8.12 Plot Nilai Fungsi Objektif dan Rasio Penerimaan terhadap Iterasi Run 3 Simulated Annealing dengan 3 Truk

Analisis Perbandingan Hasil untuk Variasi Banyak Truk

Tabel 2.8.9 Perbandingan Hasil untuk Variasi Banyak Truk

Parameter	1 Truk	2 Truk	3 Truk
Kapasitas Total Armada	40 unit berat	80 unit berat	120 unit berat
Volume Total Armada	96 unit^3	192 unit^3	288 unit^3
Persentase Paket Masuk	58.3% – 66.7% (7 –8 paket)	83.3% – 91.7% (10 – 11 paket)	100% (12 paket penuh)
Rata-rata Score Akhir	244.00 (86.2%)	382.67 (87.6%)	437.00 (100%)
Score Maksimal yang Dicapai	253	411	437

Analisis dari Keseluruhan Hasil Eksperimen

1. Seberapa dekat tiap-tiap algoritma bisa mendekati *global optima* dan mengapa hasilnya demikian?

Untuk mengetahui seberapa dekat tiap algoritma ke global optima, kita harus tau terlebih dahulu global optimanya berapa. Dengan menggunakan bruteforce, didapatkan global optima adalah 283. Kita akan menggunakan ini untuk menganalisis kedekatan tiap algoritma ke global optima.

Tabel 2.9.1 Kedekatan dan Pencapaian Algoritma terhadap Global Optima

Algoritma	Score Run 1	Score Run 2	Score Run 3	Rata-rata Score Akhir	Rata-rata Pencapaian terhadap Global Optima	Run dengan Hasil Akhir Optimal
Steepest Ascent	283	243	246	257,33	90,93%	1 dari 3
Sideways Move	283	266	252	267,00	94,35%	1 dari 3
Random Restart	283	283	276	280,67	99,18%	2 dari 3
Stochastic	283	243	246	257,33	90,93%	1 dari 3
Simulated Annealing	236	243	253	244,00	86,22%	0 dari 3
GA, populasi 20 dan 100 generasi	223	216	179	206,00	72,79%	0 dari 3

Berdasarkan pada tabel, terlihat dengan jelas yang paling mendekati global optima adalah Random Restart dan yang paling jauh adalah Genetic Algorithm. Hal ini karena Random Restart menjalankan pencarian dari beberapa konfigurasi awal yang berbeda, sehingga memiliki lebih banyak kesempatan menemukan solusi yang lebih baik. Solusi terbaik dari seluruh percobaan juga selalu disimpan, sehingga restart berikutnya tidak menurunkan kualitas hasil akhir. Mekanisme tersebut menghasilkan rata-rata score 280,67, mendekati global optimum 283. Sementara itu, implementasi Genetic Algorithm menggunakan pergantian seluruh populasi tanpa elitism, sehingga individu terbaik dari generasi sebelumnya tidak dijamin dipertahankan. Proses pembentukan keturunan, mutasi, dan repair dapat menghasilkan populasi dengan score lebih rendah.

Untuk algoritma lainnya, Sideways Move memperoleh rata-rata score 267,00 karena dapat menerima perpindahan dengan score yang sama. Perubahan konfigurasi tersebut membuka kesempatan menemukan neighbor yang lebih baik, seperti pada seed 22 dan 33. Steepest Ascent dan Stochastic menghasilkan rata-rata score yang sama, yaitu 257,33, karena keduanya hanya menerima perpindahan yang langsung meningkatkan score. Pada seed 22 dan 33, konfigurasi awal tidak memiliki neighbor valid dan juga tidak memiliki kandidat valid dengan score lebih tinggi, sehingga kedua algoritma ini terjebak pada state awal. Simulated Annealing memperoleh rata-rata score 244,00. Algoritma ini dapat menerima perpindahan bernilai sama maupun penurunan score untuk melanjutkan eksplorasi, tetapi mekanisme tersebut tidak menjamin pencapaian optimum dalam jumlah iterasi terbatas dan pada eksperimen ini, pencariannya terhenti setelah 225 iterasi dan kebetulan pada titik tersebut masih belum ada yang berhasil mencapai global optimum.

2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma *local search* yang lain?

Pada Run 1 dengan seed 11, seluruh varian HC memperoleh score akhir 283, sedangkan SA memperoleh 236. Steepest Ascent mencapai hasil tersebut melalui tiga perpindahan yang meningkatkan score. Sideways Move melanjutkan pencarian dengan 100 perpindahan bernilai sama tanpa peningkatan tambahan. Random Restart juga tidak menemukan hasil yang melampaui solusi percobaan awal, sedangkan Stochastic mencapai score yang sama melalui empat perpindahan perbaikan. Dengan demikian, kemampuan eksplorasi tambahan belum memberikan keuntungan kualitas pada seed ini.

Pada Run 2 dengan seed 22, Random Restart memperoleh hasil tertinggi sebesar 283, diikuti Sideways Move sebesar 266. Steepest Ascent, Stochastic, dan SA masing-masing memperoleh 243. Sideways menemukan perbaikan setelah dua perpindahan dengan score sama, sedangkan Random Restart menemukan solusi optimal pada restart ke-5. Hasil ini memperlihatkan manfaat mekanisme yang memungkinkan pencarian melanjutkan eksplorasi dari konfigurasi berbeda.

Pada Run 3 dengan seed 33, Random Restart kembali memberikan hasil tertinggi sebesar 276, diikuti SA sebesar 253, Sideways Move sebesar 252, serta Steepest Ascent dan Stochastic sebesar 246. Sideways membutuhkan 56 perpindahan bernilai sama sebelum menemukan peningkatan pada iterasi ke-57. SA sedikit mengungguli Sideways pada run ini, sehingga keunggulan rata-rata suatu algoritma tidak berarti algoritma tersebut selalu unggul pada setiap seed.

Secara rata-rata, urutan kualitas hasil adalah Random Restart (280,67), Sideways Move (267,00), Steepest Ascent dan Stochastic (257,33), kemudian SA (244,00). Random Restart memberikan peningkatan rata-rata sebesar 42,33 terhadap initial state percobaan pertama, sedangkan Sideways memberikan peningkatan 28,67 dan Steepest Ascent serta Stochastic memberikan peningkatan 19,00.

GA dengan populasi 20 dan 100 generasi pada seed 101–103 memperoleh rata-rata score akhir 206,00. Implementasi GA menggunakan pergantian seluruh populasi tanpa elitism, sehingga individu terbaik tidak otomatis dipertahankan. Pada seed 102 dan 103, populasi awal bahkan sudah memiliki individu optimal bernilai 283, tetapi score akhir masing-masing turun menjadi 216 dan 179. Dengan demikian, hasil akhir GA perlu dibedakan dari score terbaik yang pernah muncul selama proses evolusi.

3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?

Tabel 2.9.2 Perbandingan Durasi Proses Pencarian Setiap Algoritma

Algoritma	Durasi Run 1 (s)	Durasi Run 2 (s)	Durasi Run 3 (s)	Rata-rata Durasi (s)
Steepest Ascent	0,320	0,081	0,078	0,160
Sideways Move	37,651	8,283	12,781	19,571
Random Restart	1,688	1,718	1,360	1,589
Stochastic	0,315	0,367	0,287	0,323
Simulated Annealing	29,853	35,140	35,386	33,460
GA, populasi 20 dan 100 generasi	0,185	0,174	0,183	0,181

Steepest Ascent mempunyai rata-rata durasi paling singkat pada perbandingan ini. Dua run langsung berhenti setelah satu pemeriksaan neighborhood, sedangkan Run 1 hanya memerlukan empat iterasi. Penghentian yang cepat menghemat waktu, tetapi pada seed 22 dan 33 juga membatasi kualitas hasil karena pencarian berhenti pada konfigurasi awal.

Sideways Move membutuhkan waktu jauh lebih lama karena terus memeriksa seluruh neighbor selama perpindahan bernilai sama. Jumlah iterasinya mencapai 103, 103, dan 157. Run 1 dan Run 2 sama-sama memiliki 103 iterasi, tetapi durasinya berbeda cukup besar. Jumlah neighbor valid yang dievaluasi, biaya validasi konfigurasi, serta kondisi perangkat saat eksekusi dapat memengaruhi durasi. Kemungkinan sangat besar dipengaruhi kondisi perangkat setelah dites ulang, durasi Run 1 hanya sekitar 8,5 detik.

Random Restart membutuhkan rata-rata 1,589 detik, lebih lama daripada Steepest Ascent dan Stochastic, tetapi lebih singkat daripada Sideways Move pada hasil yang tercatat. Setiap run menjalankan 11 percobaan, tetapi total pemeriksaan neighborhood hanya 20, 20, dan 16 iterasi karena banyak percobaan segera mencapai kondisi berhenti lokal. Tambahan waktu tersebut menghasilkan peningkatan kualitas rata-rata yang paling tinggi di antara varian HC.

Stochastic menjalankan 1000 iterasi dengan rata-rata durasi 0,323 detik. Setiap iterasi hanya mengambil satu kandidat acak, sehingga biayanya berbeda dari satu iterasi Steepest Ascent atau Sideways yang memeriksa seluruh neighborhood. Oleh karena itu, meski jumlah iterasinya jauh lebih besar, durasinya bisa saja lebih singkat.

SA mempunyai rata-rata durasi 33,460 detik meskipun hanya menjalankan 225 iterasi. Selain sampling dan keputusan penerimaan kandidat, implementasinya melakukan pemeriksaan kondisi stuck melalui enumerasi neighbor pada state yang belum tersimpan dalam cache. Pemeriksaan tambahan ini ikut menambah biaya pencarian. Sementara itu, GA mengevaluasi populasi dan menghasilkan keturunan, sehingga satu generasi GA juga tidak setara dengan satu iterasi HC.

4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?

Konsistensi hasil akhir kita uji menggunakan rentang dan simpangan baku score dari 3 kali pengulangan. Semakin kecil rentang dan simpangan baku, maka semakin konsisten hasil akhirnya.

Tabel 2.9.3 Perbandingan Rentang dan Simpangan Baku Hasil Akhir Eksperimen

Algoritma	Rata-rata Score Akhir	Score Minimum	Score Maksimum	Rentang	Simpangan Baku Sampel

Steepest Ascent	257,33	243	283	40	22,28
Sideways Move	267,00	252	283	31	15,52
Random Restart	280,67	276	283	7	4,04
Stochastic	257,33	243	283	40	22,28
Simulated Annealing	244,00	236	253	17	8,54
GA, populasi 20 dan 100 generasi	206,00	179	223	44	23,64

Berdasarkan tabel, Random Restart merupakan algoritma dengan hasil paling konsisten, ditunjukkan oleh simpangan baku terkecil sebesar 4,04 dan rentang score 7. Selanjutnya, Simulated Annealing memiliki simpangan baku 8,54 dan rentang 17, diikuti Sideways Move dengan simpangan baku 15,52 dan rentang 31. Steepest Ascent dan Stochastic memiliki tingkat konsistensi yang sama, dengan simpangan baku 22,28 dan rentang 40. Sementara itu, GA pada konfigurasi populasi 20 dan 100 generasi memiliki variasi hasil terbesar, yaitu simpangan baku 23,64 dan rentang 44, sehingga paling kurang konsisten pada perbandingan ini.

5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

Tabel 2.9.4 Pengaruh banyak iterasi dengan populasi tetap 20

Jumlah Iterasi (Generasi)	Score Akhir Seed 101	Score Akhir Seed 102	Score Akhir Seed 103	Rata-rata Score Akhir Terbaik	Rata-rata Fitness Populasi Akhir	Rata-rata Durasi (s)
50	216	216	179	203,67	197,17	0,096
100	223	216	179	206,00	201,05	0,181
200	223	216	179	206,00	206,00	0,347

Penambahan iterasi dari 50 menjadi 100 meningkatkan rata-rata score akhir terbaik dari 203,67 menjadi 206,00, tetapi penambahan hingga 200 iterasi tidak memberikan peningkatan lagi.

Rata-rata fitness populasi tetap meningkat, sedangkan durasi pencarian bertambah. Artinya, lebih banyak iterasi belum tentu menghasilkan individu terbaik yang lebih baik.

Tabel 2.9.5 Pengaruh jumlah populasi dengan iterasi tetap 100

Populasi	Score Akhir Seed 201	Score Akhir Seed 202	Score Akhir Seed 203	Rata-rata Score Akhir Terbaik	Rata-rata Fitness Populasi Akhir	Rata-rata Durasi (s)
10	102	110	146	119,33	117,73	0,082
20	225	139	171	178,33	174,47	0,185
40	226	198	211	211,67	209,88	0,388

Peningkatan populasi dari 10, 20, hingga 40 meningkatkan rata-rata score akhir terbaik menjadi 119,33; 178,33; dan 211,67. Populasi lebih besar menyediakan lebih banyak individu untuk dieksplorasi, dengan konsekuensi durasi yang lebih panjang. Pada eksperimen ini, peningkatan populasi memberikan perbaikan lebih besar daripada penambahan iterasi, tetapi implementasi tanpa elitism tetap memungkinkan hilangnya individu terbaik. Karena kedua perbandingan ini memakai kelompok seed yang berbeda, maka hasil kedua serinya juga dianalisis secara terpisah.

Kesimpulan dan Saran

Kesimpulan

Berdasarkan eksperimen pada dataset satu truk dengan objective total value, Random Restart Hill-Climbing memberikan hasil rata-rata tertinggi dan paling konsisten, yaitu score 280,67 dengan simpangan baku 4,04, serta mencapai global optimum 283 pada dua dari tiga run. Sideways Move meningkatkan kualitas hasil dibandingkan Steepest Ascent dan Stochastic dengan memungkinkan perpindahan pada nilai yang sama sebelum menemukan perbaikan. Steepest Ascent memiliki rata-rata durasi paling singkat, tetapi hasilnya bergantung pada konfigurasi awal.

Pada konfigurasi yang diuji, Simulated Annealing dan Genetic Algorithm menghasilkan rata-rata score akhir lebih rendah daripada seluruh varian HC. Pada GA, peningkatan jumlah populasi memberikan perbaikan hasil yang lebih besar daripada penambahan iterasi dengan konsekuensi bertambahnya durasi pencarian. Namun, implementasi ini memungkinkan individu terbaik hilang selama pergantian populasi. Hasil eksperimen menunjukkan bahwa penambahan waktu atau jumlah iterasi belum tentu meningkatkan kualitas solusi. Mekanisme eksplorasi dan pemeliharaan solusi terbaik juga menentukan keberhasilan pencarian.

Saran

Pada pengembangan selanjutnya, kami menyarankan penambahan mekanisme *elitism* pada Genetic Algorithm agar individu terbaik tetap dipertahankan. Kemudian evaluasi parameter Simulated Annealing, seperti suhu awal dan laju pendinginan, dapat dilakukan untuk memperoleh keseimbangan eksplorasi dan kualitas solusi. Batas *sideways*, jumlah restart, dan batas iterasi juga dapat divariasikan untuk menilai pengaruhnya terhadap hasil serta durasi pencarian. Selain itu, eksperimen sebaiknya diperluas dengan lebih banyak seed dan dataset yang beragam pada lingkungan eksekusi yang sama agar kesimpulan mengenai kualitas, konsistensi, dan efisiensi setiap algoritma lebih kuat.

Pembagian Tugas

NIM	Nama	Tugas
13524019	Kevin Wirya Valerian	• Simulated Annealing • Fungsi Objektif Tambahan • Multitruck
13524043	Emilio Justin	• Setup project • Visualization, models, validation, problem, neighbors • Genetic Algorithm • Integration
13524046	Farrell Limjaya	• Hill-Climbing Search (steepest-ascent, sideways, random restart, stochastic)

Referensi

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Oct. 17, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Oct. 17, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Oct. 17, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Oct. 17, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Oct. 17, 2026].

[6] B. Jason, “Chapter 2: Temperature and Cooling Schedules”. https://algorithmafternoon.com/books/simulated_annealing/chapter02/ [Accessed: Oct. 17, 2026]

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Kevin Wirya Valeria/13524019
Emilio Justin/13524043
Farrell Limjaya/13524046
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

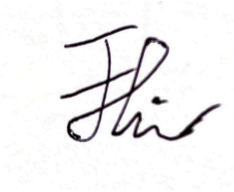
Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	Sedang
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Sedang
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 18 Oktober 2026

Kevin Wirya Valerian
13524019

Emilio Justin
13524043



Farell Limjaya
13524046